

计算机网络面试题

 面试鸭 mianshiya.com

程序员面试 刷题神器

八股文一网打尽 面试从未如此轻松

- ✓ 9000+ 中大厂高频面试题
- ✓ 大厂面试官团队原创优质题解
- ✓ 网页版+小程序+IDE、VSCode插件
- ✓ 真题命中率高，持续更新

微信扫一扫 >>

刷高频题拿高薪offer

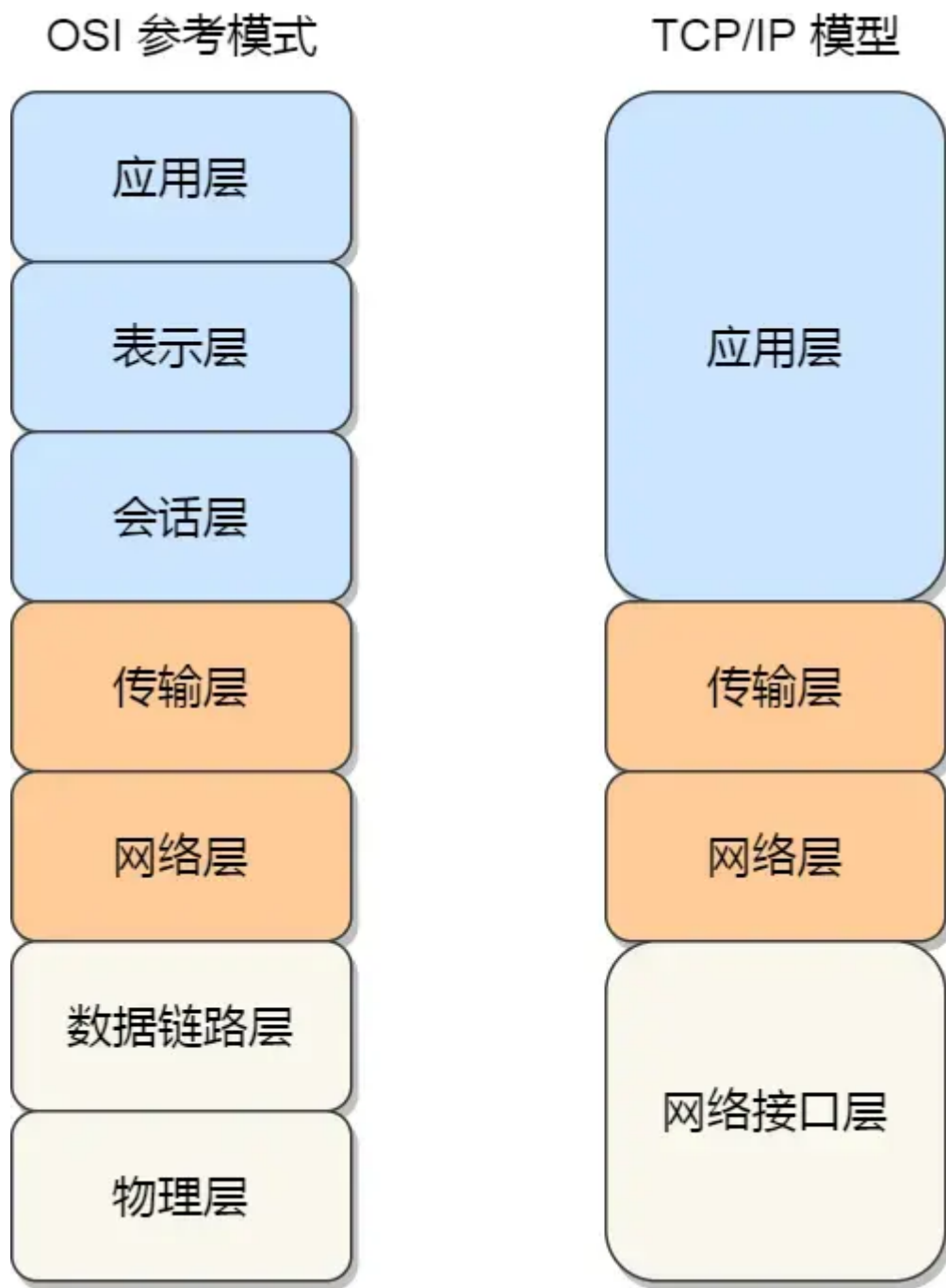


网络模型

网络OSI模型和TCP/IP模型分别介绍一下

OSI七层模型

为了使得多种设备能通过网络相互通信，和为了解决各种不同设备在网络互联中的兼容性问题，国际标准化组织制定了开放式系统互联通信参考模型（*Open System Interconnection Reference Model*），也就是 OSI 网络模型，该模型主要有 7 层，分别是应用层、表示层、会话层、传输层、网络层、数据链路层以及物理层。



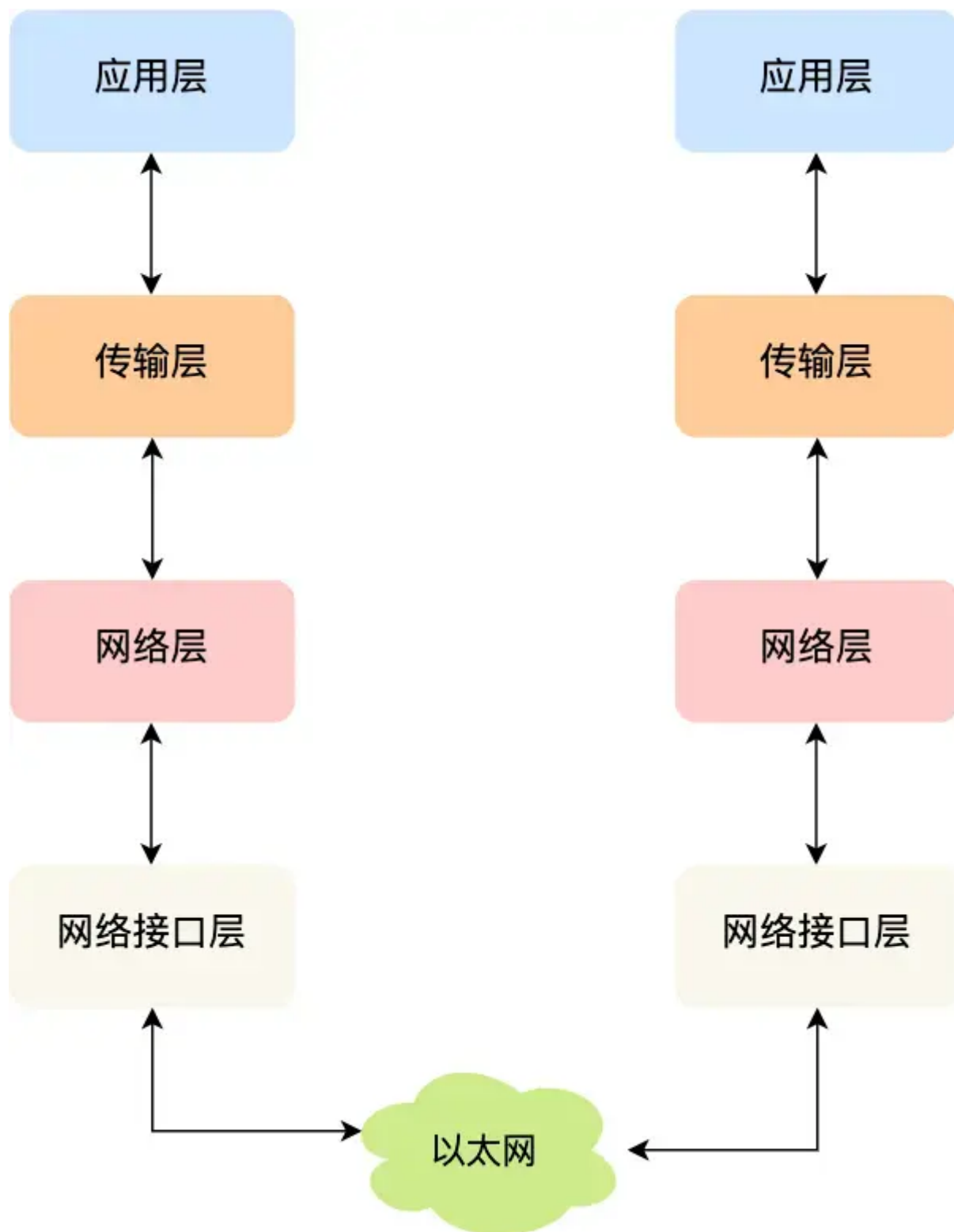
每一层负责的职能都不同，如下：

- 应用层，负责给应用程序提供统一的接口；
- 表示层，负责把数据转换成兼容另一个系统能识别的格式；
- 会话层，负责建立、管理和终止表示层实体之间的通信会话；
- 传输层，负责端到端的数据传输；
- 网络层，负责数据的路由、转发、分片；
- 数据链路层，负责数据的封装和差错检测，以及 MAC 寻址；
- 物理层，负责在物理网络中传输数据帧；

由于 OSI 模型实在太复杂，提出的也只是概念理论上的分层，并没有提供具体的实现方案。

事实上，我们比较常见，也比较实用的是四层模型，即 TCP/IP 网络模型，Linux 系统正是按照这套网络模型来实现网络协议栈的。

TCP/IP协议被组织成四个概念层，其中有三层对应于ISO参考模型中的相应层。TCP/IP协议族并不包含物理层和数据链路层，因此它不能独立完成整个计算机网络系统的功能，必须与许多其他的协议协同工作。TCP/IP 网络通常是由上到下分成 4 层，分别是**应用层**，**传输层**，**网络层**和**网络接口层**。



- 应用层 支持 HTTP、SMTP 等最终用户进程
- 传输层 处理主机到主机的通信 (TCP、UDP)
- 网络层 寻址和路由数据包 (IP 协议)

- 链路层 通过网络的物理电线、电缆或无线信道移动比特

tcp、ip分别位于哪一层？

- tcp 在传输层
- ip 在网络层

应用层

应用层有哪些协议？

HTTP、HTTPS、CDN、DNS、FTP 都是应用层协议

HTTP报文有哪些部分？

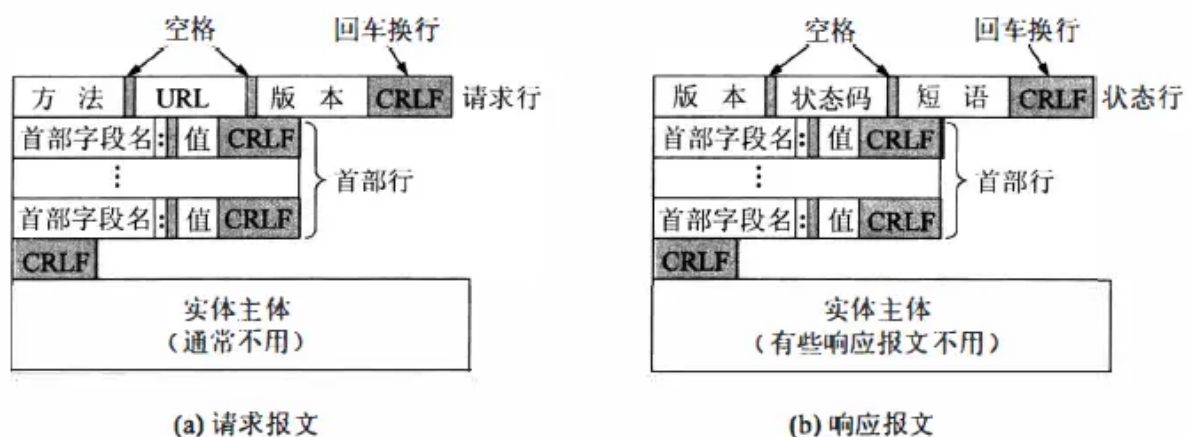


图 6-12 HTTP 的报文结构

分请求报文和响应报文来说明。

请求报文：

- 请求行：包含请求方法、请求目标（URL或URI）和HTTP协议版本。
- 请求头部：包含关于请求的附加信息，如Host、User-Agent、Content-Type等。
- 空行：请求头部和请求体之间用空行分隔。
- 请求体：可选，包含请求的数据，通常用于POST请求等需要传输数据的情况。

响应报文：

- 状态行：包含HTTP协议版本、状态码和状态信息。
- 响应头部：包含关于响应的附加信息，如Content-Type、Content-Length等。
- 空行：响应头部和响应体之间用空行分隔。
- 响应体：包含响应的数据，通常是服务器返回的HTML、JSON等内容。

HTTP常用的状态码？

五大类 HTTP 状态码

	具体含义	常见的状态码
1xx	提示信息，表示目前是协议处理的中间状态，还需要后续的操作；	
2xx	成功，报文已经收到并被正确处理；	200、204、206
3xx	重定向，资源位置发生变动，需要客户端重新发送请求；	301、302、304
4xx	客户端错误，请求报文有误，服务器无法处理；	400、403、404
5xx	服务器错误，服务器在处理请求时内部发生了错误。	500、501、502、503

HTTP 状态码分为 5 大类

- 1xx 类状态码属于**提示信息**，是协议处理中的一种中间状态，实际用到的比较少。
- 2xx 类状态码表示服务器**成功**处理了客户端的请求，也是我们最愿意看到的状态。
- 3xx 类状态码表示客户端请求的资源发生了变动，需要客户端用新的 URL 重新发送请求获取资源，也就是**重定向**。
- 4xx 类状态码表示客户端发送的**报文有误**，服务器无法处理，也就是错误码的含义。
- 5xx 类状态码表示客户端请求报文正确，但是**服务器处理时内部发生了错误**，属于服务器端的错误码。

其中常见的具体状态码有：

- 200：请求成功；
- 301：永久重定向；302：临时重定向；
- 404：无法找到此页面；405：请求的方法类型不支持；
- 500：服务器内部出错。

HTTP返回状态301 302分别是什么？

3xx 类状态码表示客户端请求的资源发生了变动，需要客户端用新的 URL 重新发送请求获取资源，也就是**重定向**。

- 「**301 Moved Permanently**」表示永久重定向，说明请求的资源已经不存在了，需改用新的 URL 再次访问。
- 「**302 Found**」表示临时重定向，说明请求的资源还在，但暂时需要用另一个 URL 来访问。

301 和 302 都会在响应头里使用字段 Location，指明后续要跳转的 URL，浏览器会自动重定向新的 URL。

http 502和 504 的区别？

- 502 Bad Gateway：作为网关或者代理工作的服务器尝试执行请求时，从上游服务器接收到无效的响应。
- 504 Gateway Time-out：作为网关或者代理工作的服务器尝试执行请求时，未能及时从上游服务器收到响应。

举一个例子，假设 nginx 是代理服务器，收到客户端的请求后，将请求转发到后端服务器（tomcat 等）。

- 当nginx收到了无效的响应时，就返回502。

- 当nginx超过自己配置的超时时间，还没有收到请求时，就返回504错误。

HTTP层请求的类型有哪些？

- GET：用于请求获取指定资源，通常用于获取数据。
- POST：用于向服务器提交数据，通常用于提交表单数据或进行资源的创建。
- PUT：用于向服务器更新指定资源，通常用于更新已存在的资源。
- DELETE：用于请求服务器删除指定资源。
- HEAD：类似于GET请求，但只返回资源的头部信息，用于获取资源的元数据而不获取实际内容。

GET和POST的使用场景，有哪些区别？

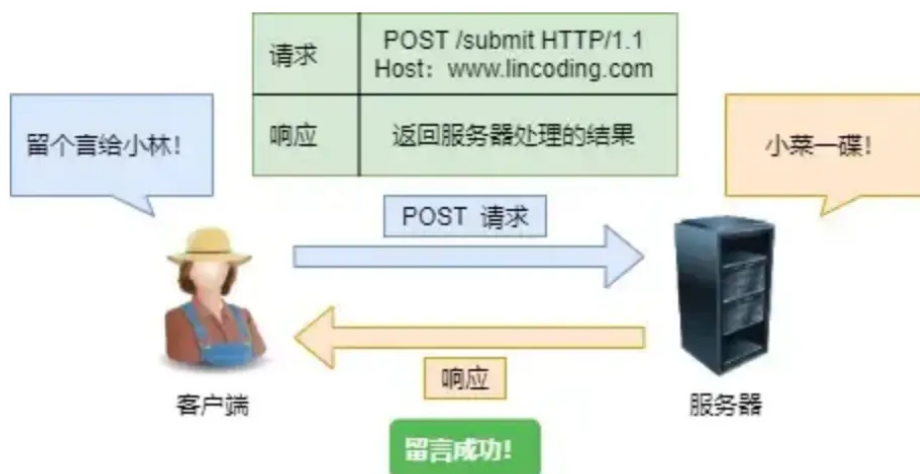
根据 RFC 规范，**GET 的语义是从服务器获取指定的资源**，这个资源可以是静态的文本、页面、图片视频等。GET 请求的参数位置一般是写在 URL 中，URL 规定只能支持 ASCII，所以 GET 请求的参数只允许 ASCII 字符，而且浏览器会对 URL 的长度有限制（HTTP协议本身对 URL长度并没有做任何规定）。

比如，你打开我的文章，浏览器就会发送 GET 请求给服务器，服务器就会返回文章的所有文字及资源。



根据 RFC 规范，**POST 的语义是根据请求负荷（报文body）对指定的资源做出处理**，具体的处理方式视资源类型而不同。POST 请求携带数据的位置一般是写在报文 body 中，body 中的数据可以是任意格式的数据，只要客户端与服务端协商好即可，而且浏览器不会对 body 大小做限制。

比如，你在我文章底部，敲入了留言后点击「提交」（暗示你们留言），浏览器就会执行一次 POST 请求，把你的留言文字放进了报文 body 里，然后拼接好 POST 请求头，通过 TCP 协议发送给服务器。



如果从 RFC 规范定义的语义来看：

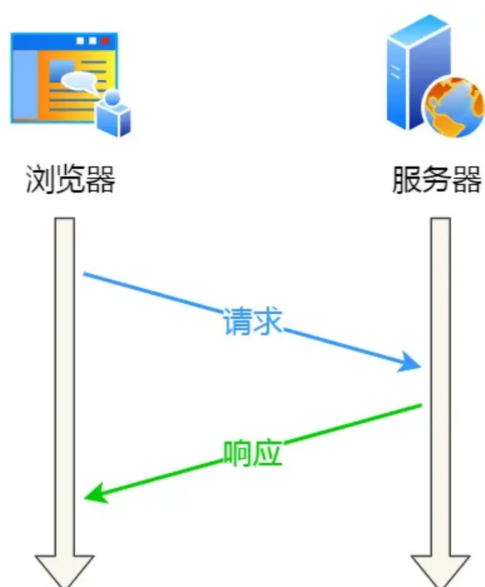
- **GET 方法就是安全且幂等的**，因为它是「只读」操作，无论操作多少次，服务器上的数据都是安全的，且每次的结果都是相同的。所以，**可以对 GET 请求的数据做缓存，这个缓存可以做到浏览器本身上（彻底避免浏览器发请求），也可以做到代理上（如nginx），而且在浏览器中 GET 请求可以保存为书签。**
- **POST** 因为是「新增或提交数据」的操作，会修改服务器上的资源，所以是**不安全的**，且多次提交数据就会创建多个资源，所以**不是幂等的**。所以，**浏览器一般不会缓存 POST 请求，也不能把 POST 请求保存为书签。**

但是实际过程中，开发者不一定会按照 RFC 规范定义的语义来实现 GET 和 POST 方法。比如：

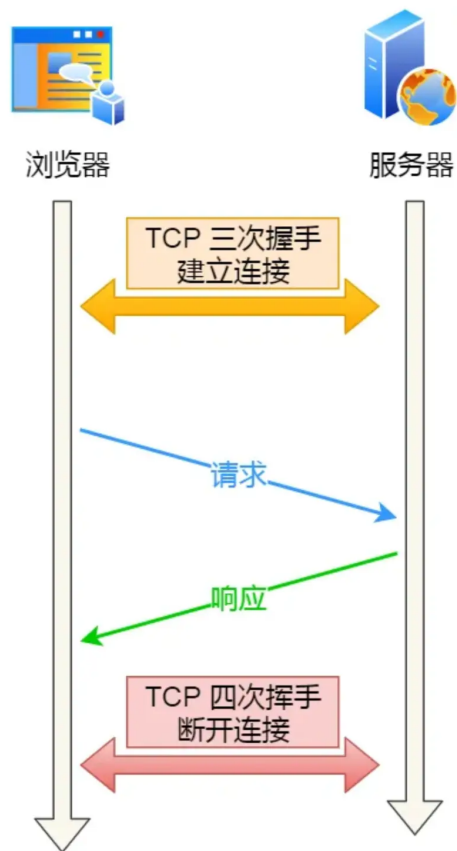
- 可以用 GET 方法实现新增或删除数据的请求，这样实现的 GET 方法自然就不是安全和幂等。
- 可以用 POST 方法实现查询数据的请求，这样实现的 POST 方法自然就是安全和幂等。

HTTP的长连接是什么？

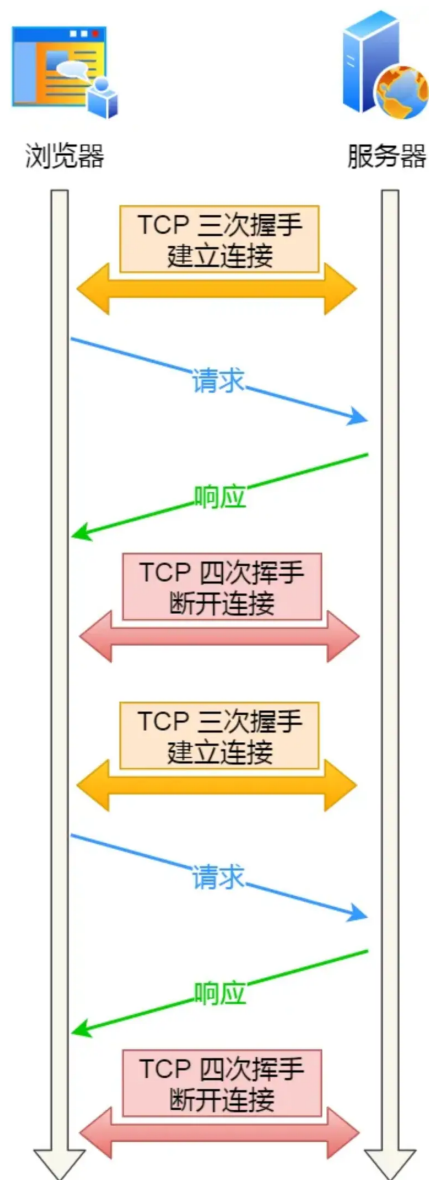
HTTP 协议采用的是「请求-应答」的模式，也就是客户端发起了请求，服务端才会返回响应，一来一回这样子。



由于 HTTP 是基于 TCP 传输协议实现的，客户端与服务端要进行 HTTP 通信前，需要先建立 TCP 连接，然后客户端发送 HTTP 请求，服务端收到后就返回响应，至此「请求-应答」的模式就完成了，随后就会释放 TCP 连接。



如果每次请求都要经历这样的过程：建立 TCP -> 请求资源 -> 响应资源 -> 释放连接，那么此方式就是 **HTTP 短连接**，如下图：



这样实在太累人了，一次连接只能请求一次资源。

能不能在第一个 HTTP 请求完后，先不断开 TCP 连接，让后续的 HTTP 请求继续使用此连接？

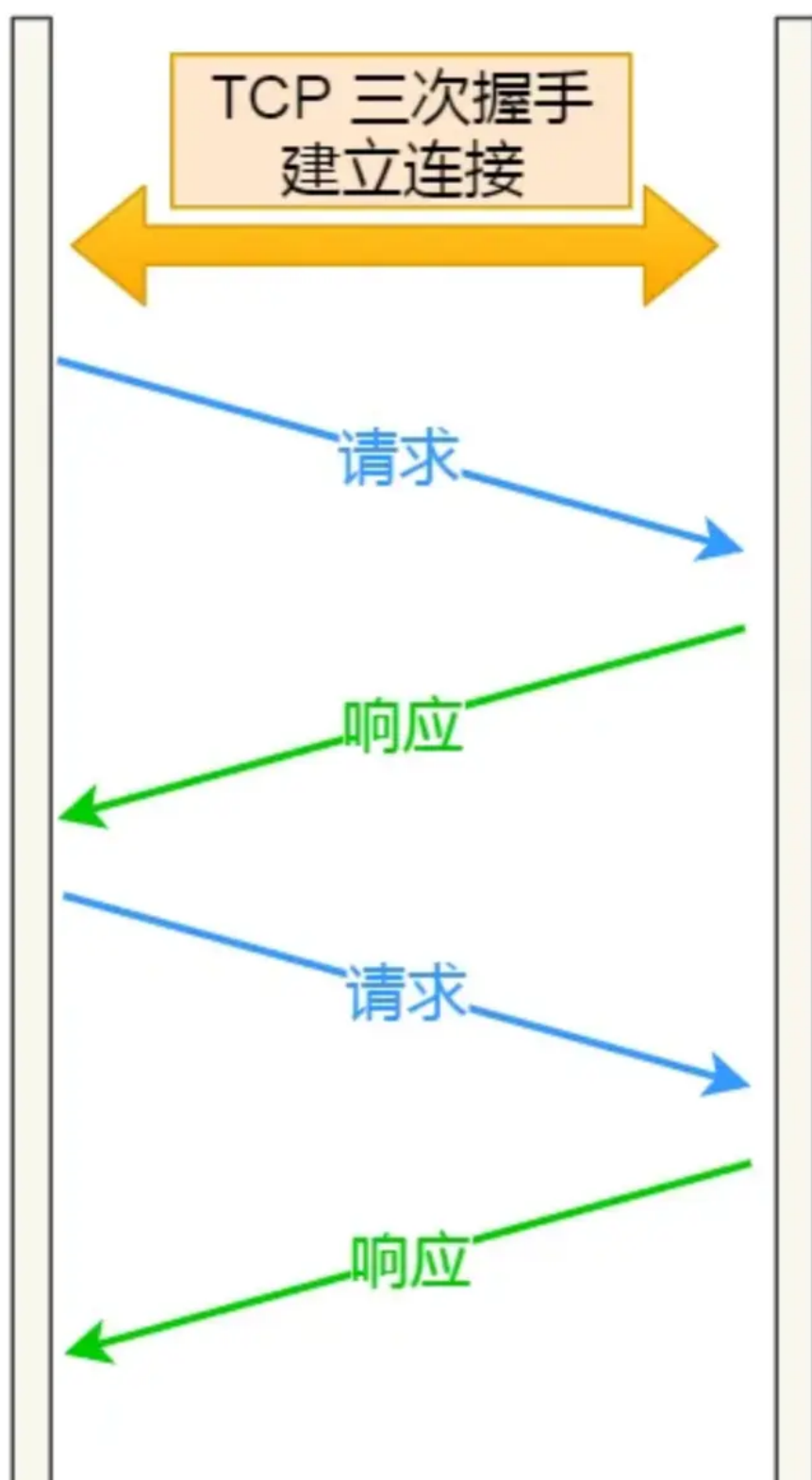
当然可以，HTTP 的 Keep-Alive 就是实现了这个功能，可以使用同一个 TCP 连接来发送和接收多个 HTTP 请求/应答，避免了连接建立和释放的开销，这个方法称为 **HTTP 长连接**。

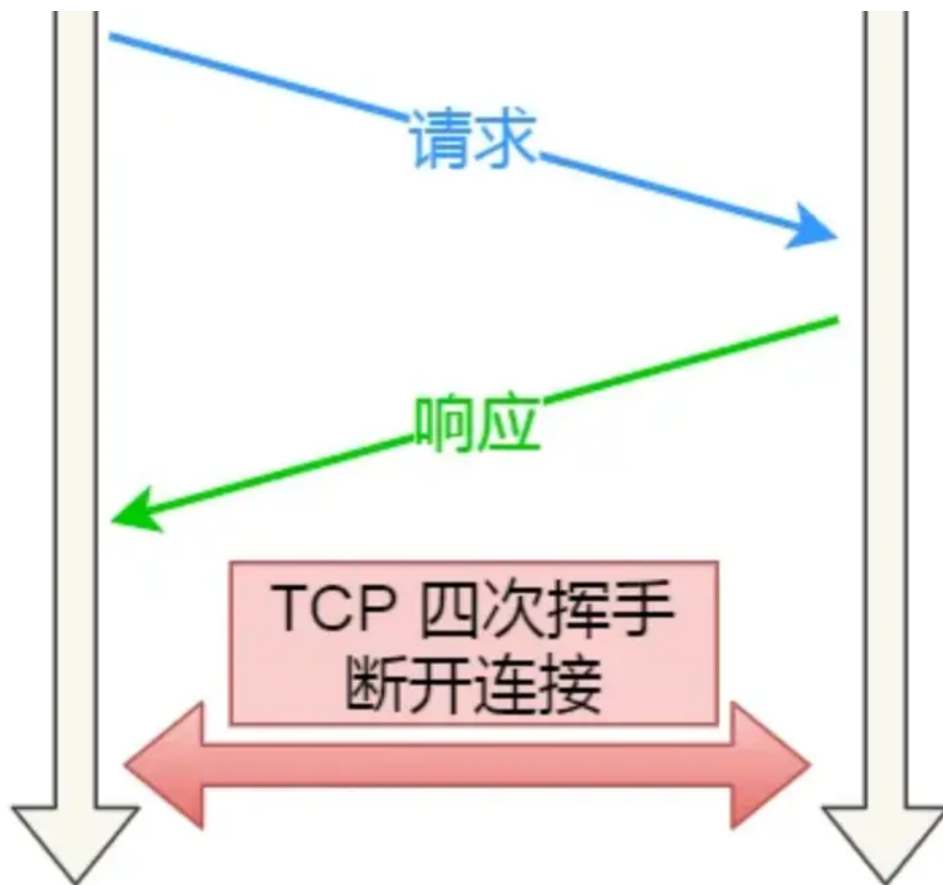


浏览器



服务器





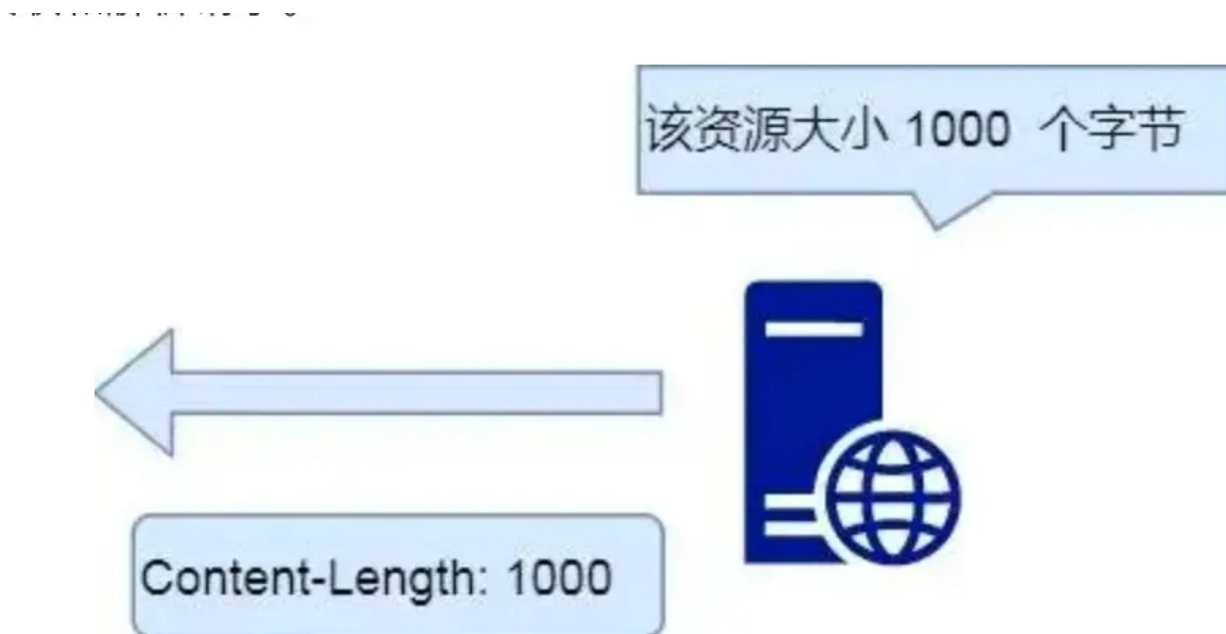
HTTP 长连接的特点是，只要任意一端没有明确提出断开连接，则保持 TCP 连接状态。

HTTP默认的端口是什么？

http 是 80，https 默认是 443。

HTTP1.1怎么对请求做拆包，具体来说怎么拆的？

在HTTP/1.1中，请求的拆包是通过"Content-Length"头字段来进行的。该字段指示了请求正文的长度，服务器可以根据该长度来正确接收和解析请求。



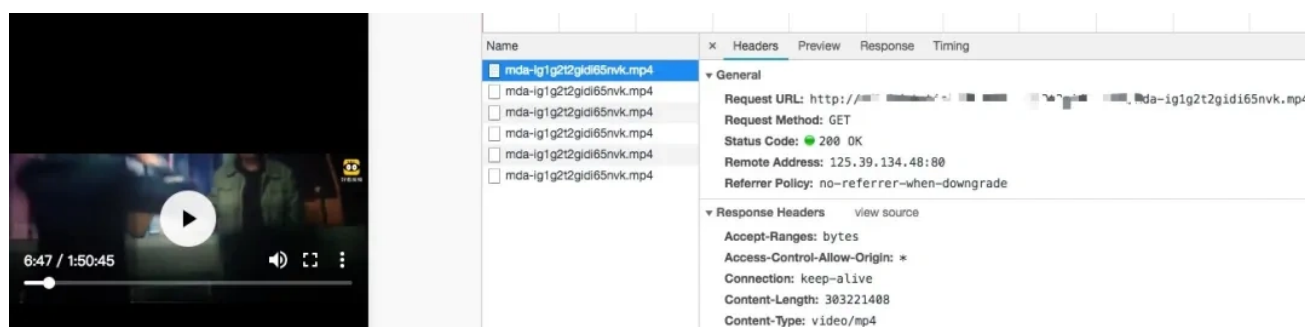
具体来说，当客户端发送一个HTTP请求时，会在请求头中添加"Content-Length"字段，该字段的值表示请求正文的字节数。

服务器在接收到请求后，会根据"Content-Length"字段的值来确定请求的长度，并从请求中读取相应数量的字节，直到读取完整个请求内容。

这种基于"Content-Length"字段的拆包机制可以确保服务器正确接收到完整的请求，避免了请求的丢失或截断问题。

http 断点重传是什么？

断点续传是HTTP/1.1协议支持的特性。实现断点续传的功能，需要客户端记录下当前的下载进度，并在需要续传的时候通知服务端本次需要下载的内容片段。



一个最简单的断点续传流程如下：

1. 客户端开始下载一个1024K的文件，服务端发送Accept-Ranges: bytes来告诉客户端，其支持带Range的请求
2. 假如客户端下载了其中512K时候网络突然断开了，过了一会网络可以了，客户端再下载时候，需要在HTTP头中申明本次需要续传的片段：Range:bytes=512000-这个头通知服务端从文件的512K位置开始传输文件，直到文件内容结束
3. 服务端收到断点续传请求，从文件的512K位置开始传输，并且在HTTP头中增加：Content-Range:bytes 512000-/1024000,Content-Length: 512000。并且此时服务端返回的HTTP状态码应该是206 Partial Content。如果客户端传递过来的Range超过资源的大小,则响应416 Requested Range Not Satisfiable

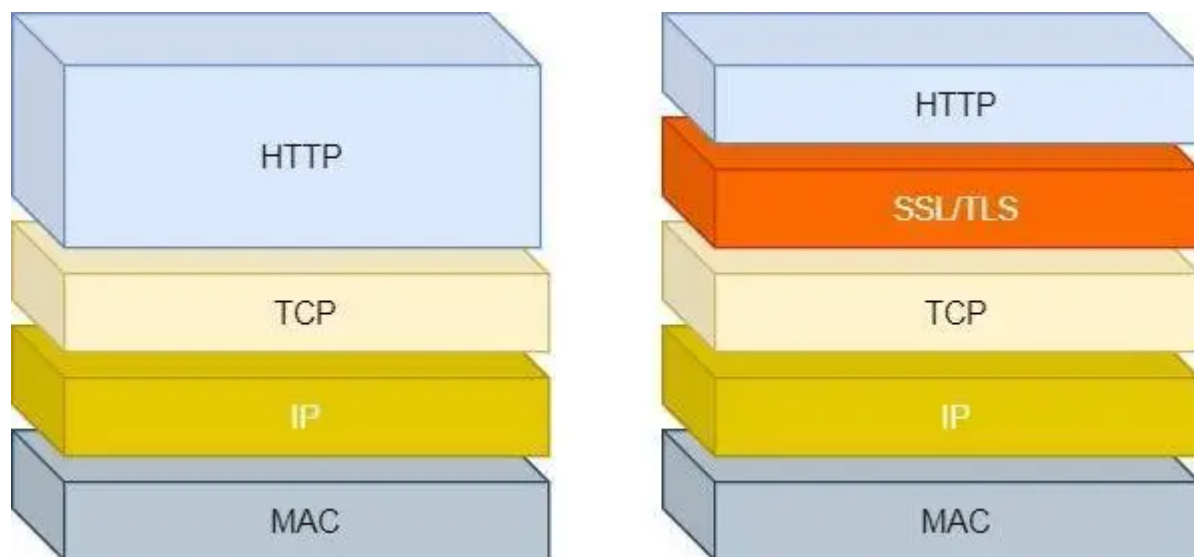
通过上面流程可以看出：断点续传中4个HTTP头不可少的，分别是**Range头**、**Content-Range头**、**Accept-Ranges头**、**Content-Length头**。其中第一个Range头是客户端发过来的，后面3个头需要服务端发送给客户端。下面是它们的说明：

- **Accept-Ranges: bytes**：这个值声明了可被接受的每一个范围请求, 大多数情况下是字节数 bytes
- **Range: bytes=开始位置-结束位置**：Range是浏览器告知服务器所需分部分内容范围的消息头。

HTTP为什么不安全？

HTTP 由于是明文传输，所以安全上存在以下三个风险：

- **窃听风险**，比如通信链路上可以获取通信内容，用户号容易没。
- **篡改风险**，比如强制植入垃圾广告，视觉污染，用户眼容易瞎。
- **冒充风险**，比如冒充淘宝网站，用户钱容易没。



HTTPS 在 HTTP 与 TCP 层之间加入了 SSL/TLS 协议，可以很好的解决了上述的风险：

- **信息加密**：交互信息无法被窃取，但你的号会因为「自身忘记」账号而没。
- **校验机制**：无法篡改通信内容，篡改了就不能正常显示，但百度「竞价排名」依然可以搜索垃圾广告。
- **身份证书**：证明淘宝是真的淘宝网，但你的钱还是会因为「剁手」而没。

HTTP和HTTPS 的区别？

区别主要有以下四点：

- HTTP 是超文本传输协议，信息是明文传输，存在安全风险的问题。HTTPS 则解决 HTTP 不安全的缺陷，在 TCP 和 HTTP 网络层之间加入了 SSL/TLS 安全协议，使得报文能够加密传输。
- HTTP 连接建立相对简单，TCP 三次握手之后便可进行 HTTP 的报文传输。而 HTTPS 在 TCP 三次握手之后，还需进行 SSL/TLS 的握手过程，才可进入加密报文传输。
- 两者的默认端口不一样，HTTP 默认端口号是 80，HTTPS 默认端口号是 443。
- HTTPS 协议需要向 CA（证书权威机构）申请数字证书，来保证服务器的身份是可信的。

HTTPS握手过程说一下

传统的 TLS 握手基本都是使用 RSA 算法来实现密钥交换的，在将 TLS 证书部署服务端时，证书文件其实就是服务端的公钥，会在 TLS 握手阶段传递给客户端，而服务端的私钥则一直留在服务端，一定要确保私钥不能被窃取。

在 RSA 密钥协商算法中，客户端会生成随机密钥，并使用服务端的公钥加密后再传给服务端。根据非对称加密算法，公钥加密的消息仅能通过私钥解密，这样服务端解密后，双方就得到了相同的密钥，再用它加密应用消息。

我用 Wireshark 工具抓了用 RSA 密钥交换的 TLS 握手过程，你可以从下面看到，一共经历了四次握手：

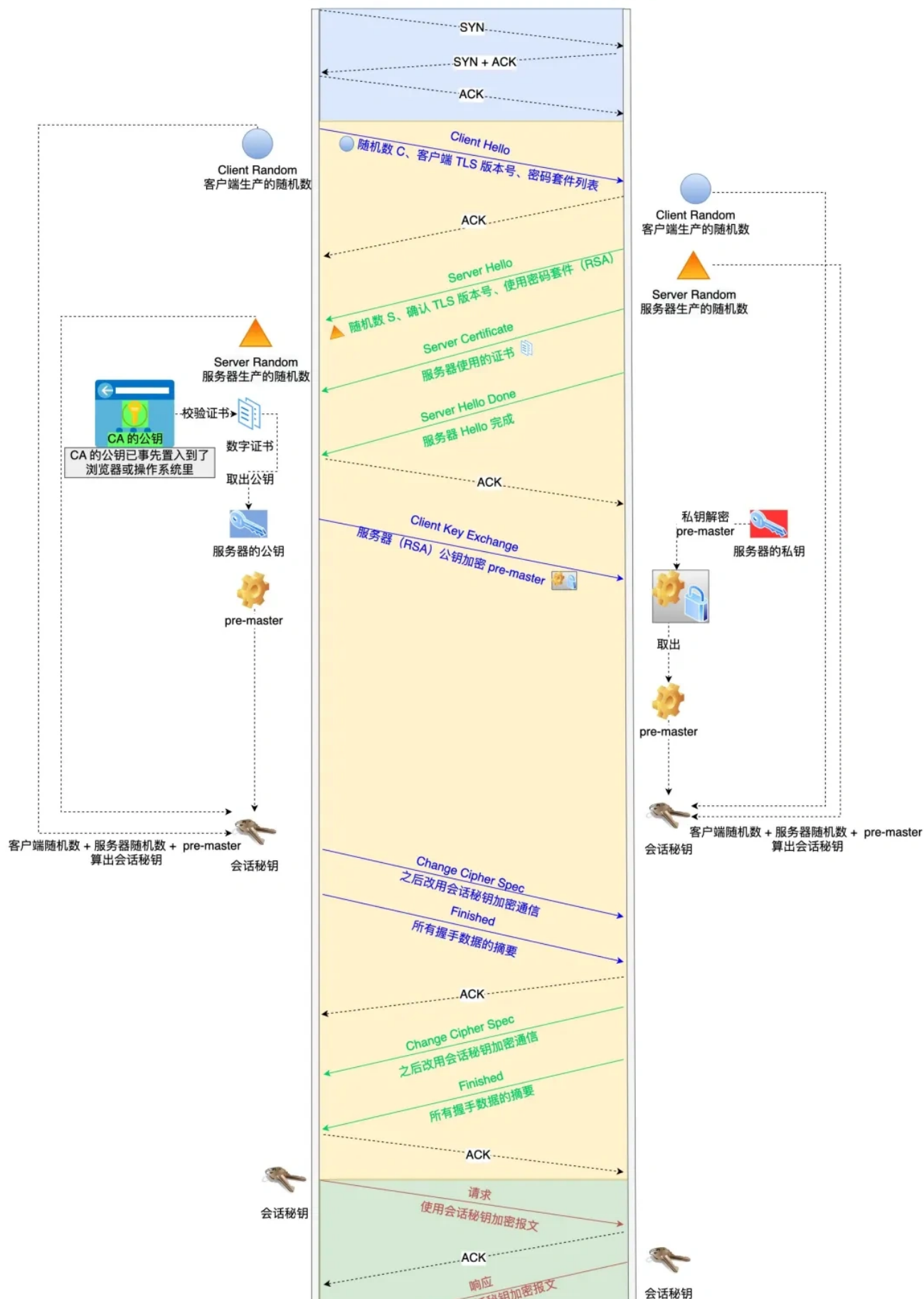
Protocol	Length	Info
TCP	108	63043 → 440 [SYN] Seq=0 Win=64240 Len=0 MSS=65495 WS=256 SACK_PERM=1
TCP	108	440 → 63043 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1
TCP	84	63043 → 440 [ACK] Seq=1 Ack=1 Win=525568 Len=0
TLSv1.2	1118	<u>Client Hello</u> • TLS 第一次握手
TCP	84	440 → 63043 [ACK] Seq=1 Ack=518 Win=525568 Len=0
TLSv1.2	1862	<u>Server Hello, Certificate, Server Hello Done</u> • TLS 第二次握手
TCP	84	63043 → 440 [ACK] Seq=518 Ack=890 Win=524544 Len=0
TLSv1.2	720	<u>Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message</u> • TLS 第三次握手
TCP	84	440 → 63043 [ACK] Seq=890 Ack=836 Win=525056 Len=0
TLSv1.2	186	<u>Change Cipher Spec, Encrypted Handshake Message</u> • TLS 第四次握手
TCP	84	63043 → 440 [ACK] Seq=836 Ack=941 Win=524544 Len=0
TLSv1.2	996	Application Data
TCP	84	440 → 63043 [ACK] Seq=941 Ack=1292 Win=524800 Len=0
TLSv1.2	506	Application Data
TCP	84	63043 → 440 [ACK] Seq=1292 Ack=1152 Win=524288 Len=0

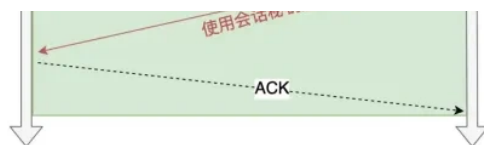


客户端



服务器





TLS 第一次握手

首先，由客户端向服务器发起加密通信请求，也就是 ClientHello 请求。在这一步，客户端主要向服务器发送以下信息：

- (1) 客户端支持的 TLS 协议版本，如 TLS 1.2 版本。
- (2) 客户端生产的随机数 (Client Random)，后面用于生成「会话密钥」条件之一。
- (3) 客户端支持的密码套件列表，如 RSA 加密算法。

TLS 第二次握手

服务器收到客户端请求后，向客户端发出响应，也就是 ServerHello。服务器回应的内容如下内容：

- (1) 确认 TLS 协议版本，如果浏览器不支持，则关闭加密通信。
- (2) 服务器生产的随机数 (Server Random)，也是后面用于生产「会话密钥」条件之一。
- (3) 确认的密码套件列表，如 RSA 加密算法。(4) 服务器的数字证书。

TLS 第三次握手

客户端收到服务器的回应之后，首先通过浏览器或者操作系统中的 CA 公钥，确认服务器的数字证书的真实性。

如果证书没有问题，客户端会**从数字证书中取出服务器的公钥**，然后使用它加密报文，向服务器发送如下信息：

- (1) 一个随机数 (pre-master key)。该随机数会被服务器公钥加密。
- (2) 加密通信算法改变通知，表示随后的信息都将用「会话密钥」加密通信。
- (3) 客户端握手结束通知，表示客户端的握手阶段已经结束。这一项同时把之前所有内容的发生的数据做个摘要，用来供服务端校验。

上面第一项的随机数是整个握手阶段的第三个随机数，会发给服务端，所以这个随机数客户端和服务端都是一样的。

服务器和客户端有了这三个随机数 (Client Random、Server Random、pre-master key)，接着就用双方协商的加密算法，各自生成本次通信的「会话密钥」。

TLS 第四次握手

服务器收到客户端的第三个随机数 (pre-master key) 之后，通过协商的加密算法，计算出本次通信的「会话密钥」。

然后，向客户端发送最后的信息：

- (1) 加密通信算法改变通知，表示随后的信息都将用「会话密钥」加密通信。
- (2) 服务器握手结束通知，表示服务器的握手阶段已经结束。这一项同时把之前所有内容的发生的数据做个摘要，用来供客户端校验。

至此，整个 TLS 的握手阶段全部结束。接下来，客户端与服务器进入加密通信，就完全是使用普通的 HTTP 协议，只不过用「会话密钥」加密内容。

HTTPS是如何防范中间人的攻击？

主要通过加密和身份校验机制来防范中间人攻击的：

- 加密：https 握手期间会通过非对称加密的方式来协商出对称加密密钥。

- 身份校验：服务器会向证书颁发机构申请数字证书，证书中包含了服务器的公钥和其他相关信息。当客户端与服务器建立连接时，服务器会将证书发送给客户端。客户端会验证证书的合法性，包括检查证书的有效期、颁发机构的信任等。如果验证通过，客户端会使用证书中的公钥来加密通信数据，并将加密后的数据发送给服务器，然后由服务端用私钥解密。

中间人攻击的关键在于攻击者冒充服务器与客户端建立连接，并同时与服务器建立连接。

但由于攻击者无法获得服务器的私钥，因此无法正确解密客户端发送的加密数据。同时，客户端会在建立连接时验证服务器的证书，如果证书验证失败或存在问题，客户端会发出警告或中止连接。

Http1.1和2.0的区别是什么？

HTTP/2 相比 HTTP/1.1 性能上的改进：

- **头部压缩**：HTTP/2 会**压缩头**（Header）如果你同时发出多个请求，他们的头是一样的或是相似的，那么，协议会帮你**消除重复的部分**。这就是所谓的 HPACK 算法：在客户端和服务器同时维护一张头信息表，所有字段都会存入这个表，生成一个索引号，以后就不发送同样字段了，只发送索引号，这样就**提高速度**了。
- **二进制格式**：HTTP/2 不再像 HTTP/1.1 里的纯文本形式的报文，而是全面采用了**二进制格式**，头信息和数据体都是二进制，并且统称为帧（frame）：**头信息帧（Headers Frame）和数据帧（Data Frame）**。这样虽然对人不友好，但是对计算机非常友好，因为计算机只懂二进制，那么收到报文后，无需再将明文的报文转成二进制，而是直接解析二进制报文，这**增加了数据传输的效率**。
- **并发传输**：引出了 Stream 概念，多个 Stream 复用在一条 TCP 连接。解决了 HTTP/1.1 队头阻塞的问题：
- **服务器主动推送资源**：HTTP/2 还在一定程度上改善了传统的「请求 - 应答」工作模式，服务端不再是被动地响应，可以**主动**向客户端发送消息。

HTTP进行TCP连接之后，在什么情况下会中断

- 当服务端或者客户端执行 close 系统调用的时候，会发送FIN报文，就会进行四次挥手的过程
- 当发送方发送了数据之后，接收方超过一段时间没有响应ACK报文，发送方重传数据达到最大次数的时候，就会断开TCP连接
- 当HTTP长时间没有进行请求和响应的时候，超过一定的时间，就会释放连接

HTTP、SOCKET和TCP的区别

HTTP是应用层协议，定义了客户端和服务端之间交换的数据格式和规则；Socket是通信的一端，提供了网络通信的接口；TCP是传输层协议，负责在网络中建立可靠的数据传输连接。它们在网络通信中扮演不同的角色和层次。

- HTTP是一种用于传输超文本数据的应用层协议，用于在客户端和服务端之间传输和显示Web页面。
- Socket是计算机网络中的一种抽象，用于描述通信链路的一端，提供了底层的通信接口，可实现不同计算机之间的数据交换。
- TCP是一种面向连接的、可靠的传输层协议，负责在通信的两端之间建立可靠的数据传输连接。

DNS的全称了解么？

DNS的全称是Domain Name System（域名系统），它是互联网中用于将域名转换为对应IP地址的分布式数据库系统。DNS扮演着重要的角色，使得人们可以通过易记的域名访问互联网资源，而无需记住复杂的IP地址。

DNS 中的域名都是用**句点**来分隔的，比如 www.server.com，这里的句点代表了不同层次之间的**界限**。

在域名中，**越靠右**的位置表示其层级**越高**。

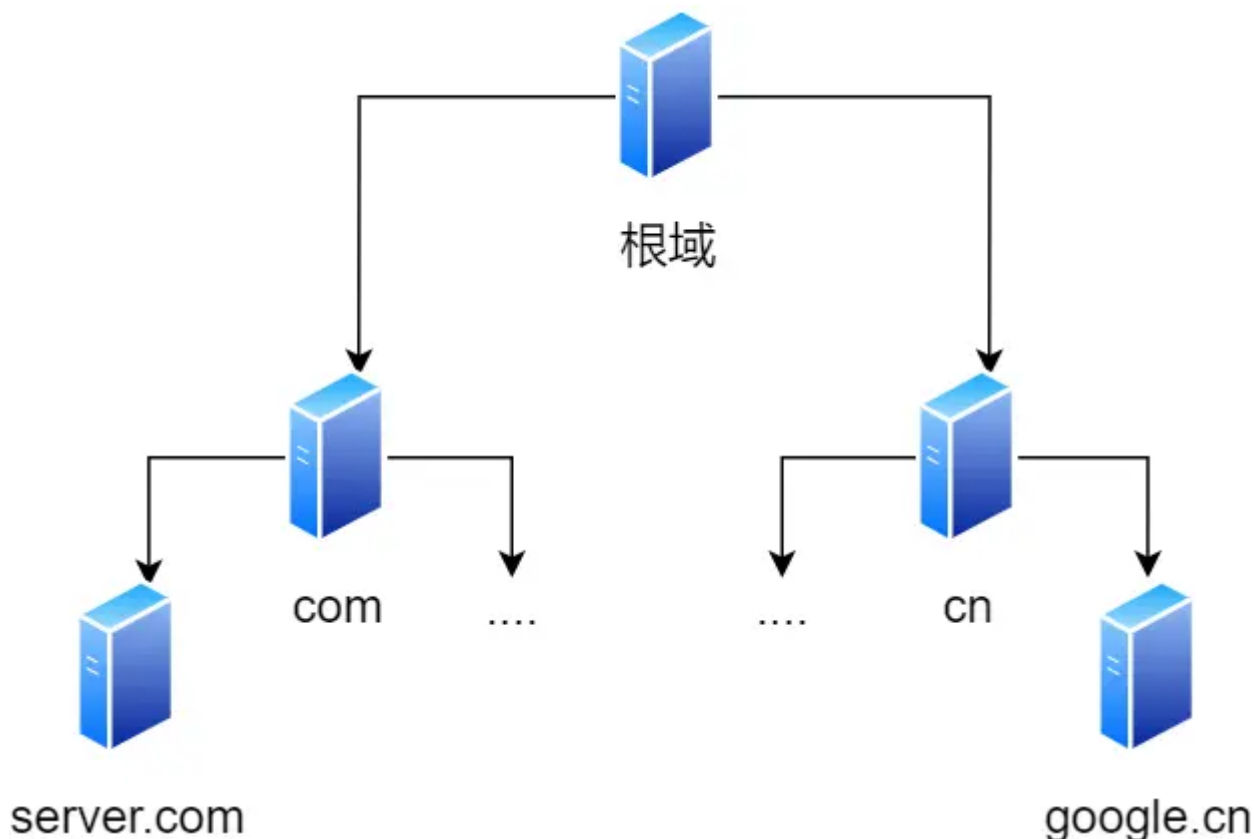
毕竟域名是外国人发明，所以思维和中国人相反，比如说一个城市地点的时候，外国喜欢从小到大的方式顺序说起（如 XX 街道 XX 区 XX 市 XX 省），而中国则喜欢从大到小的顺序（如 XX 省 XX 市 XX 区 XX 街道）。

实际上域名最后还有一个点，比如 www.server.com，这个最后的一个点代表根域名。

也就是，. 根域是在最顶层，它的下一层就是 .com 顶级域，再下面是 server.com。

所以域名的层级关系类似一个树状结构：

- 根 DNS 服务器 (.)
- 顶级域 DNS 服务器 (.com)
- 权威 DNS 服务器 (server.com)



根域的 DNS 服务器信息保存在互联网中所有的 DNS 服务器中。

这样一来，任何 DNS 服务器就都可以找到并访问根域 DNS 服务器了。

因此，客户端只要能够找到任意一台 DNS 服务器，就可以通过它找到根域 DNS 服务器，然后再一路顺藤摸瓜找到位于下层的某台目标 DNS 服务器。

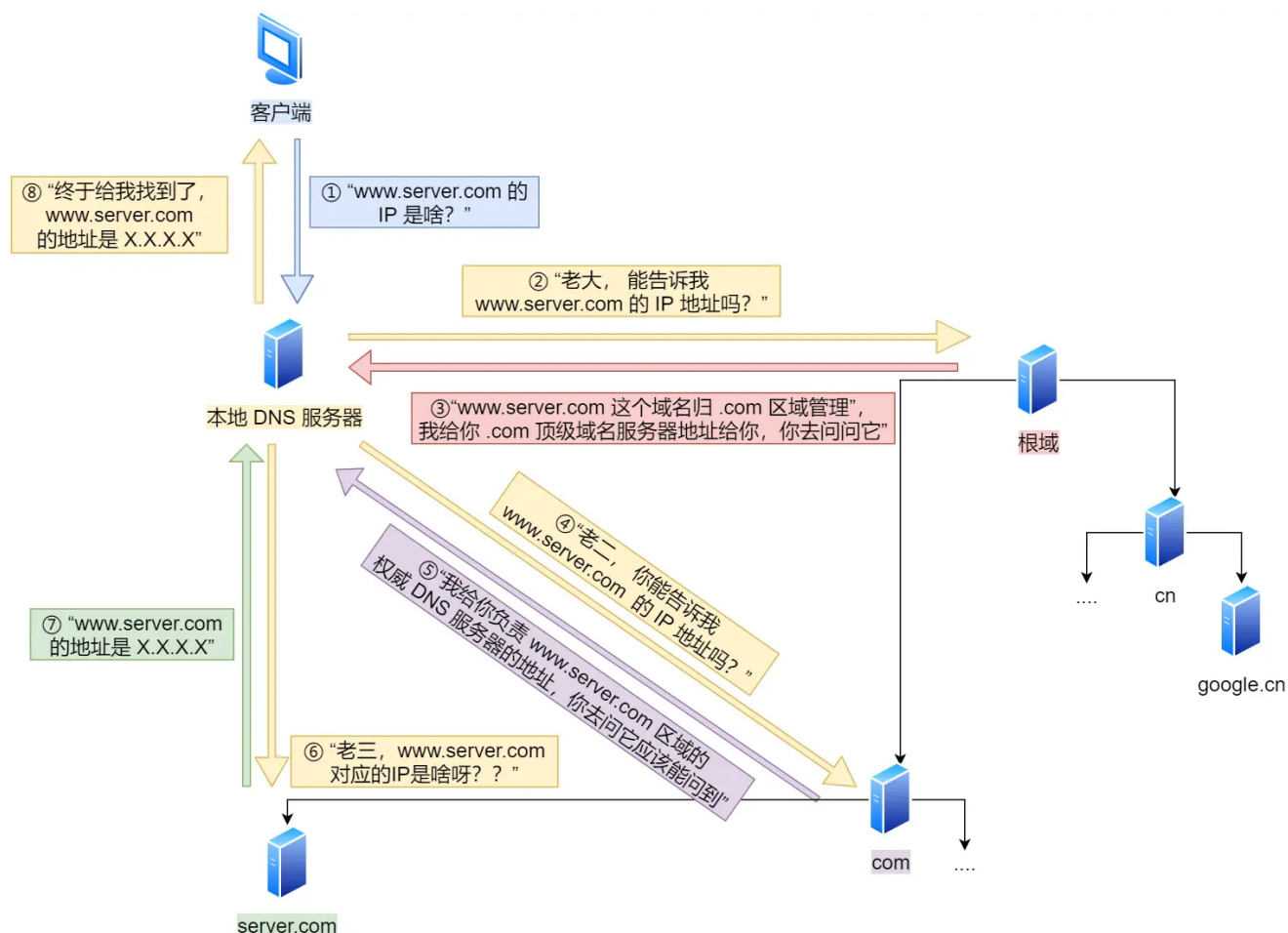
DNS 域名解析的工作流程？

1. 客户端首先会发出一个 DNS 请求，问 www.server.com 的 IP 是啥，并发给本地 DNS 服务器（也就是客户端的 TCP/IP 设置中填写的 DNS 服务器地址）。
2. 本地域名服务器收到客户端的请求后，如果缓存里的表格能找到 www.server.com，则它直接返回 IP 地址。如果没有，本地 DNS 会去问它的根域名服务器：“老大，能告诉我 www.server.com 的 IP 地址吗？”根域名服务器是最高层次的，它不直接用于域名解析，但能指明一条道路。
3. 根 DNS 收到来自本地 DNS 的请求后，发现后置是 .com，说：“www.server.com 这个域名归 .com 区域管理”，我给你 .com 顶级域名服务器地址给你，你去问问它吧。”
4. 本地 DNS 收到顶级域名服务器的地址后，发起请求问“老二，你能告诉我 www.server.com 的 IP 地址吗？”
5. 顶级域名服务器说：“我给你负责 www.server.com 区域的权威 DNS 服务器的地址，你去问它应该能问到”。
6. 本地 DNS 于是转向问权威 DNS 服务器：“老三，www.server.com 对应的 IP 是啥呀？”server.com 的权威 DNS 服务器，它是域名解析结果的原出处。为啥叫权威呢？就是我的域名我做主。

7. 权威 DNS 服务器查询后将对应的 IP 地址 X.X.X.X 告诉本地 DNS。

8. 本地 DNS 再将 IP 地址返回客户端，客户端和目标建立连接。

至此，我们完成了 DNS 的解析过程。现在总结一下，整个过程我画成了一个图。



DNS的端口是多少？

默认端口号是53。

DNS的底层使用TCP还是UDP？

DNS 基于UDP协议实现，DNS使用UDP协议进行域名解析和数据传输。因为基于UDP实现DNS能够提供低延迟、简单快速、轻量级的特性，更适合DNS这种需要快速响应的域名解析服务。

- **低延迟**：UDP是一种无连接的协议，不需要在数据传输前建立连接，因此可以减少传输时延，适合DNS这种需要快速响应的应用场景。
- **简单快速**：UDP相比于TCP更简单，没有TCP的连接管理和流量控制机制，传输效率更高，适合DNS这种需要快速传输数据的场景。
- **轻量级**：UDP头部较小，占用较少的网络资源，对于小型请求和响应来说更加轻量级，适合DNS这种频繁且短小的数据交换。

尽管 UDP 存在丢包和数据包损坏的风险，但在 DNS 的设计中，这些风险是可以被容忍的。DNS 使用了一些机制来提高可靠性，例如查询超时重传、请求重试、缓存等，以确保数据传输的可靠性和正确性。

HTTP到底是不是无状态的？

HTTP是无状态的，这意味着每个请求都是独立的，服务器不会在多个请求之间保留关于客户端状态的信息。在每个HTTP请求中，服务器不会记住之前的请求或会话状态，因此每个请求都是相互独立的。

虽然HTTP本身是无状态的，但可以通过一些机制来实现状态保持，其中最常见的方式是使用Cookie和Session来跟踪用户状态。通过在客户端存储会话信息或状态信息，服务器可以识别和跟踪特定用户的状态，以提供一定程度的状态保持功能。

携带Cookie的HTTP请求是有状态还是无状态的？Cookie是HTTP协议簇的一部分，那为什么还说HTTP是无状态的？

携带Cookie的HTTP请求实际上是可以一定程度上实现状态保持的，因为Cookie是用来在客户端存储会话信息和状态信息的一种机制。当浏览器发送包含Cookie的HTTP请求时，服务器可以通过读取这些Cookie来识别用户、管理会话状态以及保持特定的用户状态。因此，可以说即使HTTP本身是无状态的协议，但通过Cookie的使用可以实现一定程度的状态保持功能。

HTTP被描述为“无状态”的主要原因是每个HTTP请求都是独立的，服务器并不保存关于客户端的状态信息，每个请求都需要提供足够的信息来理解请求的意图。这样的设计使得Web系统更具有规模化和简单性，但也导致了一些挑战，比如需要额外的机制来处理用户状态和会话管理。

虽然Cookie是HTTP协议簇的一部分，但是HTTP协议在设计初衷上仍然保持无状态特性，即每个请求都是相互独立的。使用Cookie只是在无状态协议下的一种补充机制，用于在客户端存储状态信息以实现状态保持。

cookie和session有什么区别？

Cookie和Session都是Web开发中用于跟踪用户状态的技术，但它们在存储位置、数据容量、安全性以及生命周期等方面存在显著差异：

- **存储位置**：Cookie的数据存储在客户端（通常是浏览器）。当浏览器向服务器发送请求时，会自动附带Cookie中的数据。Session的数据存储在服务器端。服务器为每个用户分配一个唯一的Session ID，这个ID通常通过Cookie或URL重写的方式发送给客户端，客户端后续的请求会带上这个Session ID，服务器根据ID查找对应的Session数据。
- **数据容量**：单个Cookie的大小限制通常在4KB左右，而且大多数浏览器对每个域名的总Cookie数量也有限制。由于Session存储在服务器上，理论上不受数据大小的限制，主要受限于服务器的内存大小。
- **安全性**：Cookie相对不安全，因为数据存储在客户端，容易受到XSS（跨站脚本攻击）的威胁。不过，可以通过设置HttpOnly属性来防止JavaScript访问，减少XSS攻击的风险，但仍然可能受到CSRF（跨站请求伪造）的攻击。Session通常认为比Cookie更安全，因为敏感数据存储在服务器端。但仍然需要防范Session劫持（通过获取他人的Session ID）和会话固定攻击。
- **生命周期**：Cookie可以设置过期时间，过期后自动删除。也可以设置为会话Cookie，即浏览器关闭时自动删除。Session在默认情况下，当用户关闭浏览器时，Session结束。但服务器也可以设置Session的超时时间，超过这个时间未活动，Session也会失效。
- **性能**：使用Cookie时，因为数据随每个请求发送到服务器，可能会影响网络传输效率，尤其是在Cookie数据较大时。使用Session时，因为数据存储在服务器端，每次请求都需要查询服务器上的Session数据，这可能会增加服务器的负载，特别是在高并发场景下。

token, session, cookie的区别？

- session存储于服务器，可以理解为一个状态列表，拥有一个唯一识别符号sessionId，通常存放于cookie中。服务器收到cookie后解析出sessionId，再去session列表中查找，才能找到相应session，依赖cookie。
- cookie类似一个令牌，装有sessionId，存储在客户端，浏览器通常会自动添加。
- token也类似一个令牌，无状态，用户信息都被加密到token中，服务器收到token后解密就可知道是哪个用户，需要开发者手动添加。

如果客户端禁用了cookie，session还能用吗？

默认情况下禁用 Cookie 后，Session 是无法正常使用的，因为大多数 Web 服务器都是依赖于 Cookie 来传递 Session 的会话 ID 的。

客户端浏览器禁用 Cookie 时，服务器将无法把会话 ID 发送给客户端，客户端也无法在后续请求中携带会话 ID 返回给服务器，从而导致服务器无法识别用户会话。

但是，有几种方法可以绕过这个问题，尽管它们可能会引入额外的复杂性和/或降低用户体验：

1. **URL重写**：每当服务器响应需要保持状态请求时，将Session ID附加到URL中作为参数。例如，原本的链接 <http://example.com/page> 变为 <http://example.com/page;jsessionid=XXXXXX>，服务器端需要相应地解析URL来获取Session ID，并维护用户的会话状态。这种方式的缺点是URL变得不那么整洁，且如果用户通过电子邮件或其他方式分享了这样的链接，可能导致Session ID的意外泄露。
2. **隐藏表单字段**：在每个需要Session信息的HTML表单中包含一个隐藏字段，用来存储Session ID。当表单提交时，Session ID随表单数据一起发送回服务器，服务器通过解析表单数据中的Session ID来获取用户的会话状态。这种方法仅适用于通过表单提交的交互模式，不适合链接点击或Ajax请求。

如果我把数据存储在 localStorage，和Cookie有什么区别？

- 存储容量: Cookie 的存储容量通常较小,每个 Cookie 的大小限制在几 KB 左右。而 localStorage 的存储容量通常较大,一般限制在几 MB 左右。因此,如果需要存储大量数据, localStorage 通常更适合;
- 数据发送: Cookie 在每次 HTTP 请求中都会自动发送到服务器,这使得 Cookie 适合用于在客户端和服务器之间传递数据。而 localStorage 的数据不会自动发送到服务器,它仅在浏览器端存储数据,因此 localStorage 适合用于在同一域名下的不同页面之间共享数据;
- 生命周期: Cookie 可以设置一个过期时间,使得数据在指定时间后自动过期。而 localStorage 的数据将永久存储在浏览器中,除非通过 JavaScript 代码手动删除;
- 安全性: Cookie 的安全性较低,因为 Cookie 在每次 HTTP 请求中都会自动发送到服务器,存在被窃取或篡改的风险。而 localStorage 的数据仅在浏览器端存储,不会自动发送到服务器,相对而言更安全一些;

什么数据应该存在到cookie，什么数据存放到 Localstorage

Cookie 适合用于在客户端和服务器之间传递数据、跨域访问和设置过期时间，而 localStorage 适合用于在同一域名下的不同页面之间共享数据、存储大量数据和永久存储数据。

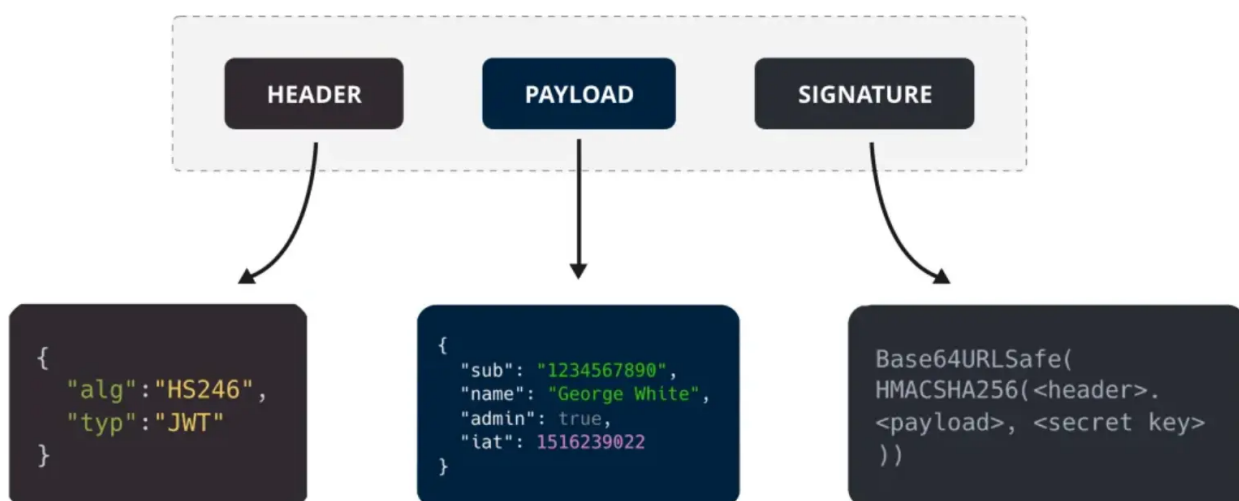
JWT 令牌和传统方式有什么区别？

- 无状态性: JWT是无状态的令牌，不需要在服务器端存储会话信息。相反，JWT令牌中包含了所有必要的信息，如用户身份、权限等。这使得JWT在分布式系统中更加适用，可以方便地进行扩展和跨域访问。
- 安全性: JWT使用密钥对令牌进行签名，确保令牌的完整性和真实性。只有持有正确密钥的服务器才能对令牌进行验证和解析。这种方式比传统的基于会话和Cookie的验证更加安全，有效防止了CSRF（跨站请求伪造）等攻击。
- 跨域支持: JWT令牌可以在不同域之间传递，适用于跨域访问的场景。通过在请求的头部或参数中携带JWT令牌，可以实现无需Cookie的跨域身份验证。

JWT 令牌都有哪些字段？（没答上来，忘了有哪些，没想到会问）

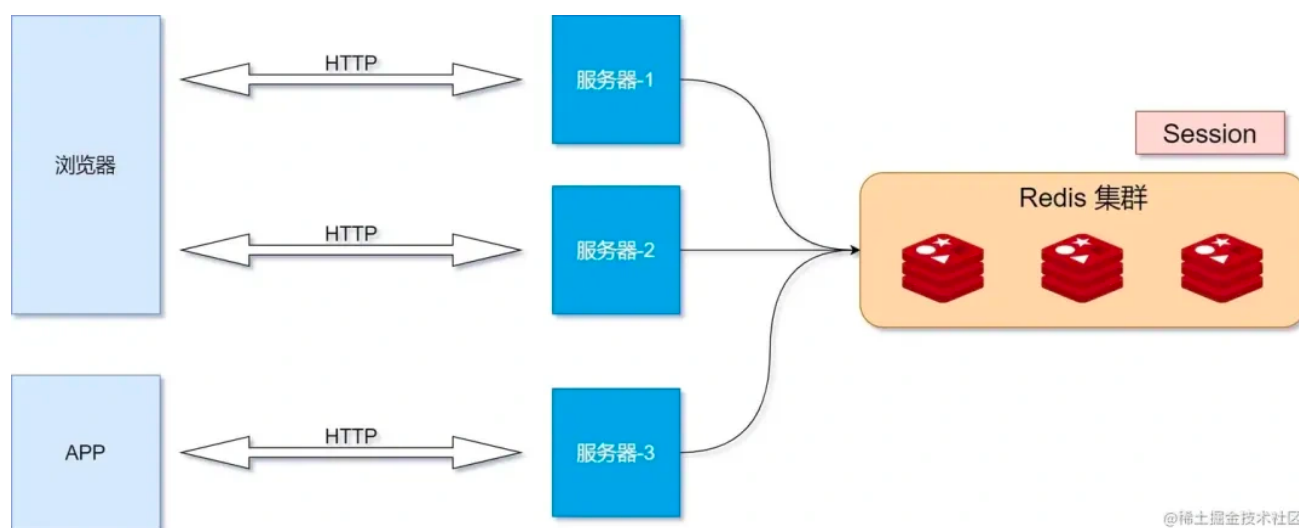
JWT令牌由三个部分组成：头部（Header）、载荷（Payload）和签名（Signature）。其中，头部和载荷均为JSON格式，使用Base64编码进行序列化，而签名部分是对头部、载荷和密钥进行签名后的结果。

Structure of a JSON Web Token (JWT)



JWT 令牌为什么能解决集群部署，什么是集群部署？（答上来了）

在传统的基于会话和Cookie的身份验证方式中，会话信息通常存储在服务器的内存或数据库中。但在集群部署中，不同服务器之间没有共享的会话信息，这会导致用户在不同服务器之间切换时需要重新登录，或者需要引入额外的共享机制（如Redis），增加了复杂性和性能开销。



而JWT令牌通过在令牌中包含所有必要的身份验证和会话信息，使得服务器无需存储会话信息，从而解决了集群部署中的身份验证和会话管理问题。当用户进行登录认证后，服务器将生成一个JWT令牌并返回给客户端。客户端在后续的请求中携带该令牌，服务器可以通过对令牌进行验证和解析来获取用户身份和权限信息，而无需访问共享的会话存储。

由于JWT令牌是自包含的，服务器可以独立地对令牌进行验证，而不需要依赖其他服务器或共享存储。这使得集群中的每个服务器都可以独立处理请求，提高了系统的可伸缩性和容错性。

jwt的缺点是什么？

JWT 一旦派发出去，在失效之前都是有效的，没办法即使撤销JWT。

要解决这个问题，得在业务层增加判断逻辑，比如增加**黑名单机制**。使用内存数据库比如 Redis 维护一个黑名单，如果想让某个 JWT 失效的话就直接将这个 JWT 加入到 **黑名单** 即可。然后，每次使用 JWT 进行请求的话都会先判断这个 JWT 是否存在于黑名单中。

JWT 令牌如果泄露了，怎么解决，JWT是怎么做的？

- 及时失效令牌：当检测到JWT令牌泄露或存在风险时，可以立即将令牌标记为失效状态。服务器在接收到带有失效标记的令牌时，会拒绝对其进行任何操作，从而保护用户的身份和数据安全。
- 刷新令牌：JWT令牌通常具有一定的有效期，过期后需要重新获取新的令牌。当检测到令牌泄露时，可以主动刷新令牌，即重新生成一个新的令牌，并将旧令牌标记为失效状态。这样，即使泄露的令牌被恶意使用，也会很快失效，减少了被攻击者滥用的风险。
- 使用黑名单：服务器可以维护一个令牌的黑名单，将泄露的令牌添加到黑名单中。在接收到令牌时，先检查令牌是否在黑名单中，如果在则拒绝操作。这种方法需要服务器维护黑名单的状态，对性能有一定的影响，但可以有效地保护泄露的令牌不被滥用。

前端是如何存储JWT的？

JSON Web Token（缩写 JWT）是目前最流行的跨域认证解决方案。互联网服务离不开用户认证。

一般流程如下：

1. 用户向服务器发送用户名和密码。
2. 服务器验证通过后，在当前对话（session）里面保存相关数据，比如用户角色、登录时间等等。
3. 服务器向用户返回一个 session_id，写入用户的 Cookie。
4. 用户随后的每一次请求，都会通过 Cookie，将 session_id 传回服务器。
5. 服务器收到 session_id，找到前期保存的数据，由此得知用户的身份。

这种模式的问题在于，扩展性（scaling）不好。单机当然没有问题，如果是服务器集群，或者是跨域的服务导向架构，就要求 session 数据共享，每台服务器都能够读取 session。

举例来说，A 网站和 B 网站是同一家公司的关联服务。现在要求，用户只要在其中一个网站登录，再访问另一个网站就会自动登录，请问怎么实现？

一种解决方案是 session 数据持久化，写入数据库或别的持久层。各种服务收到请求后，都向持久层请求数据。这种方案的优点是架构清晰，缺点是工程量比较大。另外，持久层万一挂了，就会单点失败。

另一种方案是服务器索性不保存 session 数据了，**所有数据都保存在客户端，每次请求都发回服务器。JWT 就是这种方案的一个代表。**

客户端收到服务器返回的 JWT，**可以储存在 Local Storage 里面，也可以储存在Cookie里面，还可以存储在 Session Storage里面。下面将说明存在上述各个地方的优劣势：**

Local Storage（本地存储）

- **优点：**Local Storage 提供了较大的存储空间（一般为5MB），且不会随着HTTP请求一起发送到服务器，因此不会出现在HTTP缓存或日志中。
- **缺点：**存在XSS（跨站脚本攻击）的风险，恶意脚本可以通过JavaScript访问到存储在Local Storage中的 JWT，从而盗取用户凭证。

Session Storage（会话存储）

- **优点：**与Local Storage类似，但仅限于当前浏览器窗口或标签页，当窗口关闭后数据会被清除，这在一定程度上减少了数据泄露的风险。
- **缺点：**用户体验可能受影响，因为刷新页面或在新标签页打开相同应用时需要重新认证。

- **优点**：可以设置HttpOnly标志来防止通过JavaScript访问，减少XSS攻击的风险；可以利用Secure标志确保仅通过HTTPS发送，增加安全性。
- **缺点**：大小限制较小（通常4KB），并且每次HTTP请求都会携带Cookie，可能影响性能；设置不当可能会受到CSRF（跨站请求伪造）攻击。

为什么有HTTP协议了?还要用RPC?

- RPC 本质上不算是协议，而是一种调用方式，而像 gRPC 和 Thrift 这样的具体实现，才是协议，它们是实现了 RPC 调用的协议。目的是希望程序员能像调用本地方法那样去调用远端的服务方法。同时 RPC 有很多种实现方式，不一定非得基于 TCP 协议。
- 从发展历史来说，HTTP 主要用于 B/S 架构，而 RPC 更多用于 C/S 架构。但现在其实已经没分那么清了，B/S 和 C/S 在慢慢融合。很多软件同时支持多端，所以对外一般用 HTTP 协议，而内部集群的微服务之间则采用 RPC 协议进行通讯。
- RPC 其实比 HTTP 出现的要早，且比目前主流的 HTTP/1.1 性能要更好，所以大部分公司内部都还在使用 RPC。
- HTTP/2.0在 HTTP/1.1的基础上做了优化，性能可能比很多 RPC 协议都要好，但由于是这几年才出来的，所以也不太可能取代掉 RPC。

HTTP长连接与WebSocket有什么区别?

- **全双工和半双工**：TCP 协议本身是**全双工**的，但我们最常用的 HTTP/1.1，虽然是基于 TCP 的协议，但它是**半双工**的，对于大部分需要服务器主动推送数据到客户端的场景，都不太友好，因此我们需要使用支持全双工的 WebSocket 协议。
- **应用场景区别**：在 HTTP/1.1 里，只要客户端不问，服务端就不答。基于这样的特点，对于登录页面这样的简单场景，可以使用**定时轮询或者长轮询**的方式实现**服务器推送**(comet)的效果。对于客户端和服务端之间需要频繁交互的复杂场景，比如网页游戏，都可以考虑使用 WebSocket 协议。

Nginx有哪些负载均衡算法?

Nginx支持的负载均衡算法包括：

- **轮询**：按照顺序依次将请求分配给后端服务器。这种算法最简单，但是也无法处理某个节点变慢或者客户端操作有连续性的情况。
- **IP哈希**：根据客户端IP地址的哈希值来确定分配请求的后端服务器。适用于需要保持同一客户端的请求始终发送到同一台后端服务器的场景，如会话保持。
- **URL哈希**：按访问的URL的哈希结果来分配请求，使每个URL定向到一台后端服务器，可以进一步提高后端缓存服务器的效率。
- **最短响应时间**：按照后端服务器的响应时间来分配请求，响应时间短的优先分配。适用于后端服务器性能不均的场景，能够将请求发送到响应时间快的服务器，实现负载均衡。
- **加权轮询**：按照权重分配请求给后端服务器，权重越高的服务器获得更多的请求。适用于后端服务器性能不同的场景，可以根据服务器权重分配请求，提高高性能服务器的利用率。

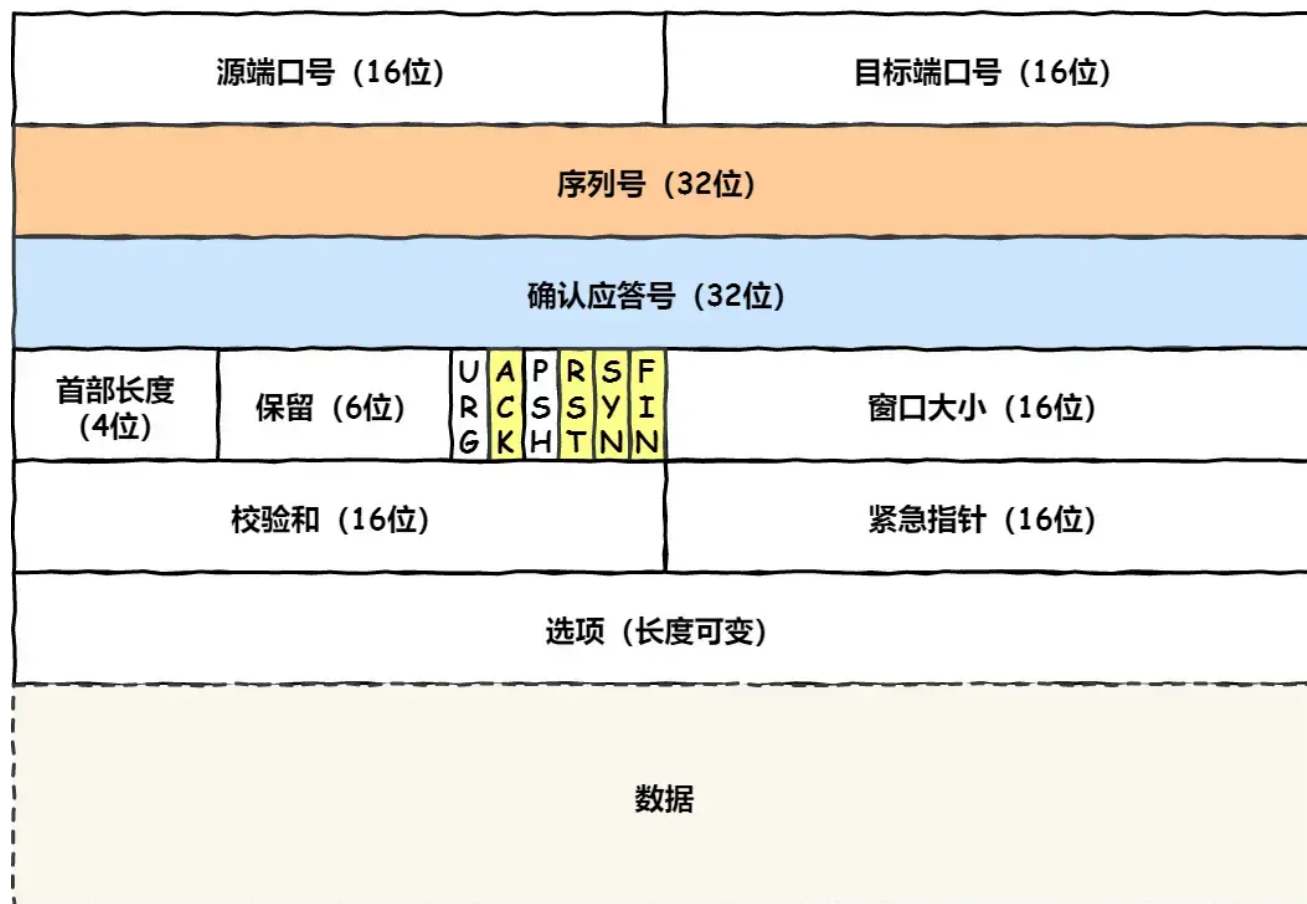
Nginx位于七层网络结构中的哪一层?

应用层，nginx 是七层负载均衡。

传输层

说一下tcp的头部

TCP 头部格式



标注颜色的表示与本文关联比较大的字段，其他字段不做详细阐述。

序列号：在建立连接时由计算机生成的随机数作为其初始值，通过 SYN 包传给接收端主机，每发送一次数据，就「累加」一次该「数据字节数」的大小。**用来解决网络包乱序问题。**

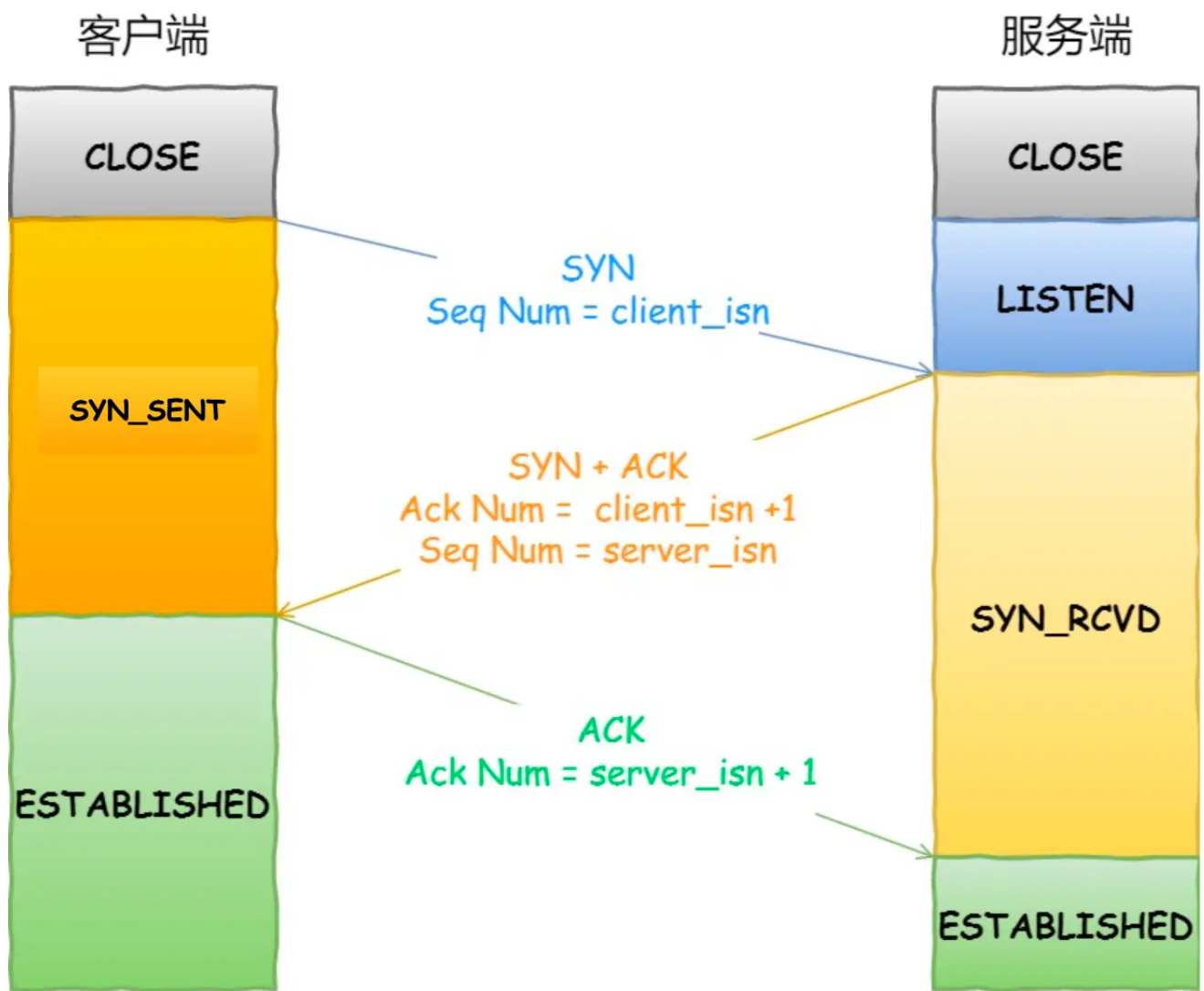
确认应答号：指下一次「期望」收到的数据的序列号，发送端收到这个确认应答以后可以认为在这个序号以前的数据都已经被正常接收。**用来解决丢包的问题。**

控制位：

- **ACK**：该位为 1 时，「确认应答」的字段变为有效，TCP 规定除了最初建立连接时的 SYN 包之外该位必须设置为 1。
- **RST**：该位为 1 时，表示 TCP 连接中出现异常必须强制断开连接。
- **SYN**：该位为 1 时，表示希望建立连接，并在其「序列号」的字段进行序列号初始值的设定。
- **FIN**：该位为 1 时，表示今后不会再有数据发送，希望断开连接。当通信结束希望断开连接时，通信双方的主机之间就可以相互交换 FIN 位为 1 的 TCP 段。

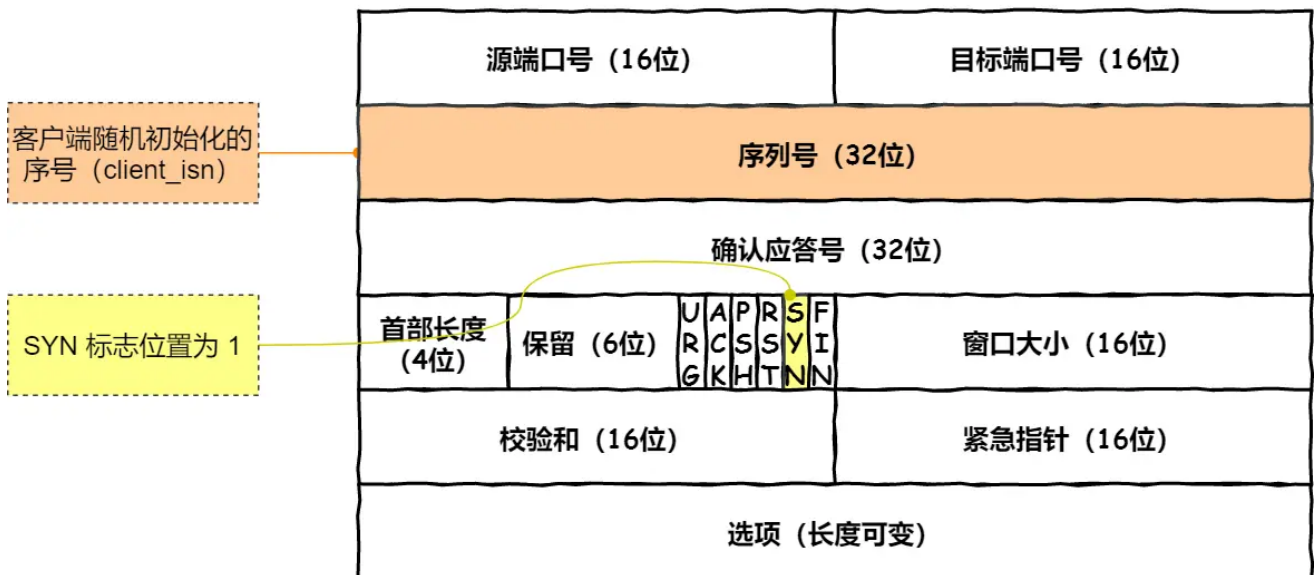
TCP三次握手过程说一下？

TCP 是面向连接的协议，所以使用 TCP 前必须先建立连接，而**建立连接是通过三次握手来进行的**。三次握手的过程如下图：



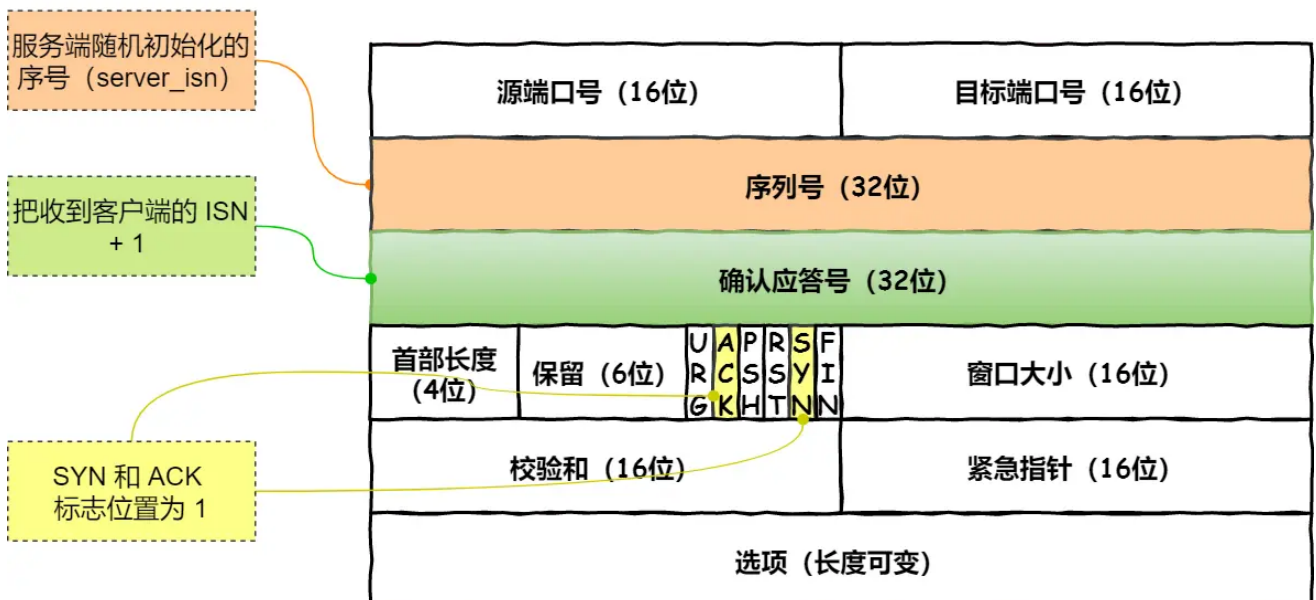
- 一开始，客户端和服务端都处于 **CLOSE** 状态。先是服务端主动监听某个端口，处于 **LISTEN** 状态

三次握手的第一个报文： SYN 报文



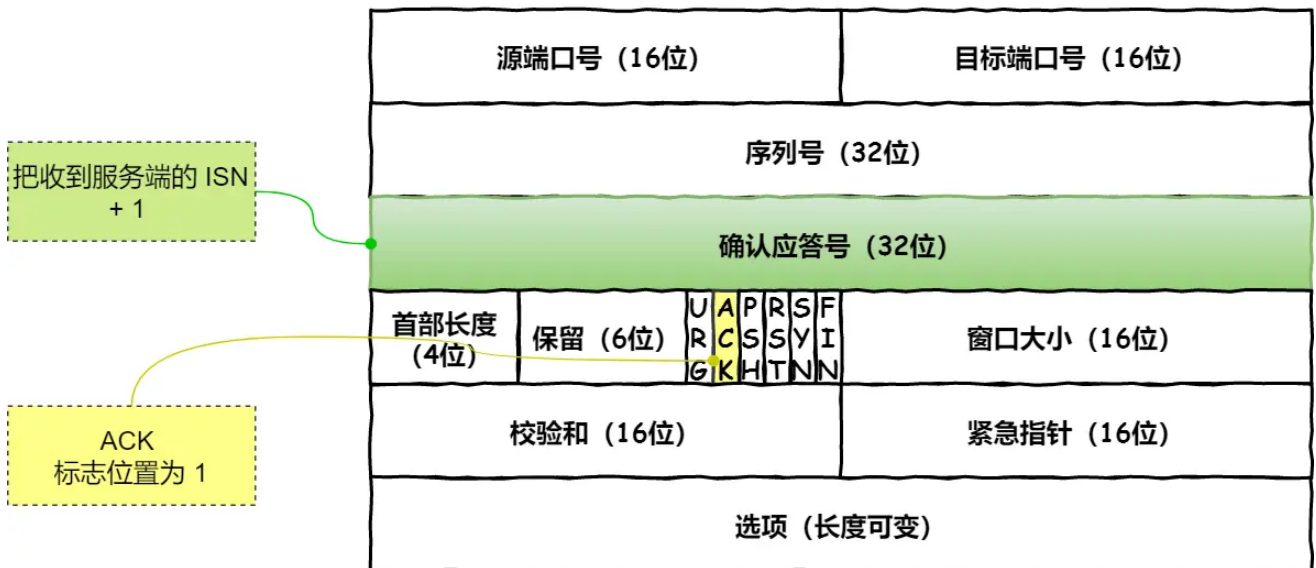
- 客户端会随机初始化序号 (client_isn)，将此序号置于 TCP 首部的「序号」字段中，同时把 SYN 标志位置为 1，表示 SYN 报文。接着把第一个 SYN 报文发送给服务端，表示向服务端发起连接，该报文不包含应用层数据，之后客户端处于 SYN-SENT 状态。

三次握手的第二个报文： SYN + ACK 报文



- 服务端收到客户端的 SYN 报文后，首先服务端也随机初始化自己的序号 (server_isn)，将此序号填入 TCP 首部的「序号」字段中，其次把 TCP 首部的「确认应答号」字段填入 client_isn + 1，接着把 SYN 和 ACK 标志位置为 1。最后把该报文发给客户端，该报文也不包含应用层数据，之后服务端处于 SYN-RCVD 状态。

三次握手的第三个报文： ACK 报文



- 客户端收到服务端报文后，还要向服务端回应最后一个应答报文，首先该应答报文 TCP 首部 ACK 标志位置为 1，其次「确认应答号」字段填入 $\text{server_isn} + 1$ ，最后把报文发送给服务端，这次报文可以携带客户到服务端的数据，之后客户端处于 ESTABLISHED 状态。
- 服务端收到客户端的应答报文后，也进入 ESTABLISHED 状态。

从上面的过程可以发现**第三次握手是可以携带数据的**，**前两次握手是不可以携带数据的**，这也是面试常问的题。

一旦完成三次握手，双方都处于 ESTABLISHED 状态，此时连接就已建立完成，客户端和服务端就可以相互发送数据了。

tcp为什么需要三次握手建立连接？

三次握手的原因：

- 三次握手才可以阻止重复历史连接的初始化（主要原因）
- 三次握手才可以同步双方的初始序列号
- 三次握手才可以避免资源浪费

原因一：避免历史连接

我们来看看 RFC 793 指出的 TCP 连接使用三次握手的首要原因：

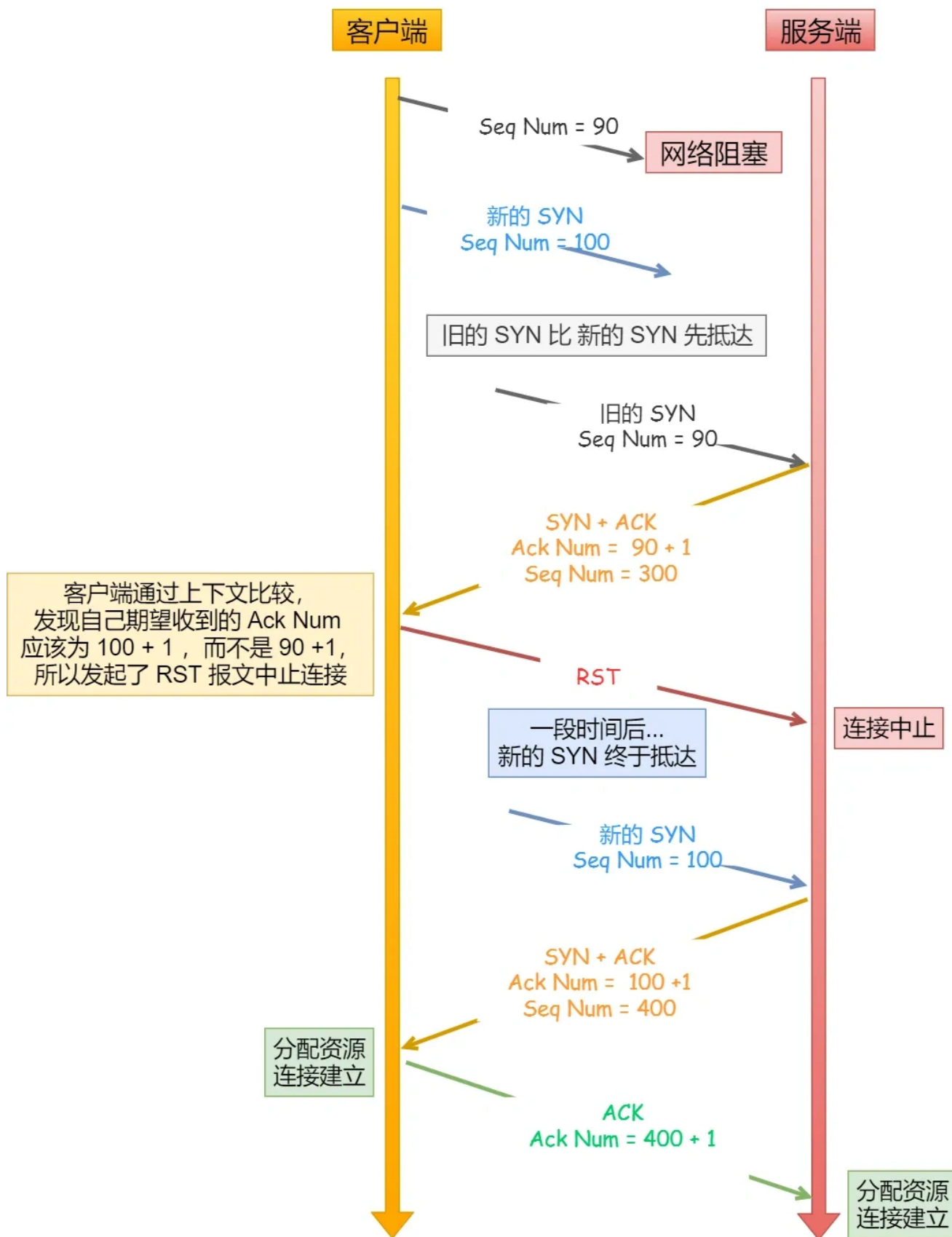
The principle reason for the three-way handshake is to prevent old duplicate connection initiations from causing confusion.

简单来说，三次握手的首要原因是为了防止旧的重复连接初始化造成混乱。

我们考虑一个场景，客户端先发送了 SYN ($\text{seq} = 90$) 报文，然后客户端宕机了，而且这个 SYN 报文还被网络阻塞了，服务端并没有收到，接着客户端重启后，又重新向服务端建立连接，发送了 SYN ($\text{seq} = 100$) 报文（注意！不是重传 SYN，重传的 SYN 的序列号是一样的）。

看看三次握手是如何阻止历史连接的：

三次握手 避免历史连接



客户端连续发送多次 SYN（都是同一个四元组）建立连接的报文，在**网络拥堵**情况下：

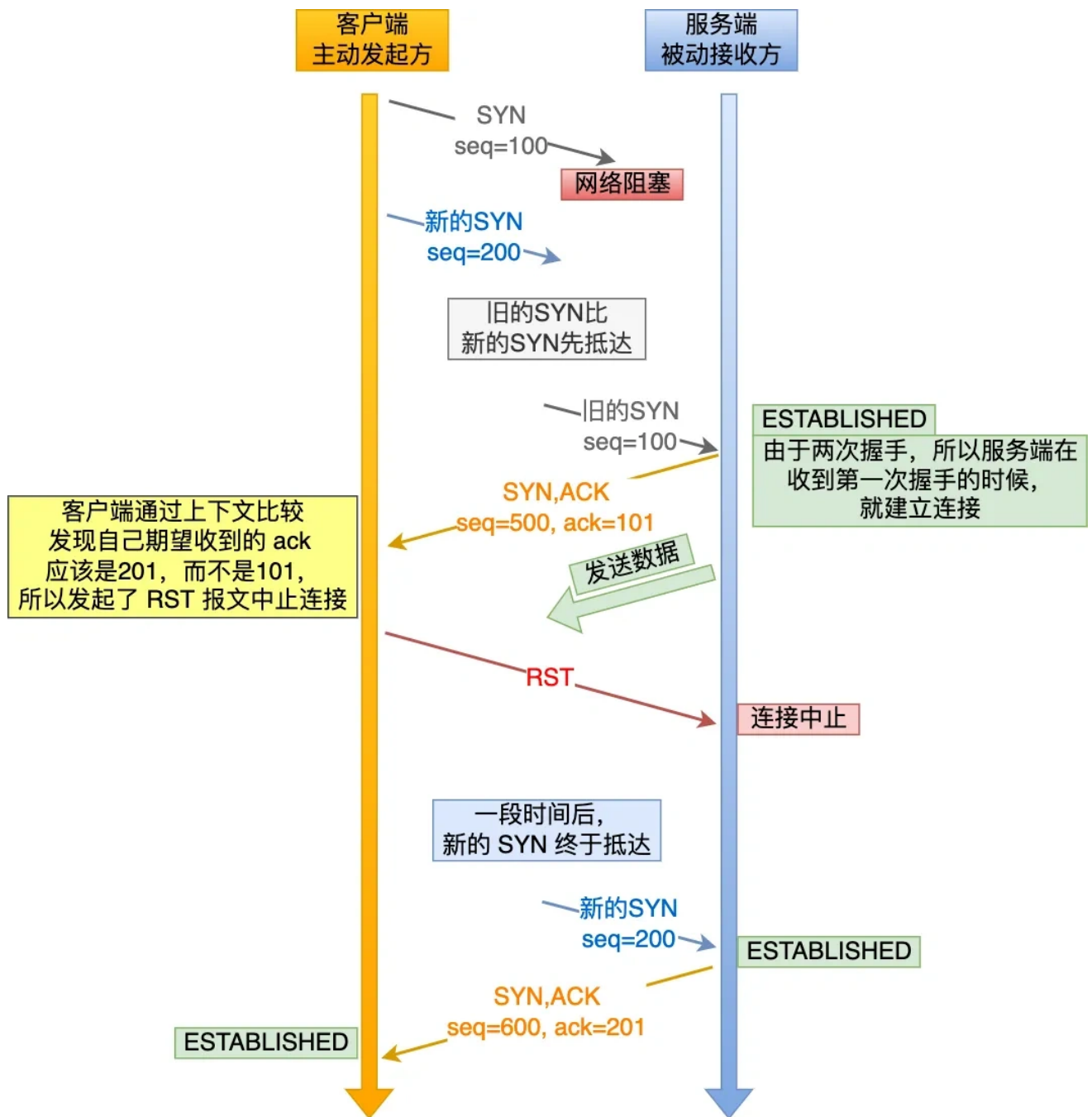
- 一个「旧 SYN 报文」比「最新的 SYN」报文早到达了服务端，那么此时服务端就会回一个 SYN + ACK 报文给客户端，此报文中的确认号是 91 (90+1)。
- 客户端收到后，发现自己期望收到的确认号应该是 100 + 1，而不是 90 + 1，于是就会回 RST 报文。
- 服务端收到 RST 报文后，就会释放连接。
- 后续最新的 SYN 抵达了服务端后，客户端与服务端就可以正常的完成三次握手了。

上述中的「旧 SYN 报文」称为历史连接，TCP 使用三次握手建立连接的**最主要原因就是防止「历史连接」初始化了连接。**

如果是两次握手连接，就无法阻止历史连接，那为什么 TCP 两次握手为什么无法阻止历史连接呢？

我先直接说结论，主要是因为**在两次握手的情况下，服务端没有中间状态给客户端来阻止历史连接，导致服务端可能建立一个历史连接，造成资源浪费。**

你想想，在两次握手的情况下，服务端在收到 SYN 报文后，就进入 ESTABLISHED 状态，意味着这时可以给对方发送数据，但是客户端此时还没有进入 ESTABLISHED 状态，假设这次是历史连接，客户端判断到此次连接为历史连接，那么就会回 RST 报文来断开连接，而服务端在第一次握手的时候就进入 ESTABLISHED 状态，所以它可以发送数据的，但是它并不知道这个是历史连接，它只有在收到 RST 报文后，才会断开连接。



可以看到, 如果采用两次握手建立 TCP 连接的场景下, 服务端在向客户端发送数据前, 并没有阻止掉历史连接, 导致服务端建立了一个历史连接, 又白白发送了数据, 妥妥地浪费了服务端的资源。

因此, 要解决这种现象, 最好就是在服务端发送数据前, 也就是建立连接之前, 要阻止掉历史连接, 这样就不会造成资源浪费, 而要实现这个功能, 就需要三次握手。

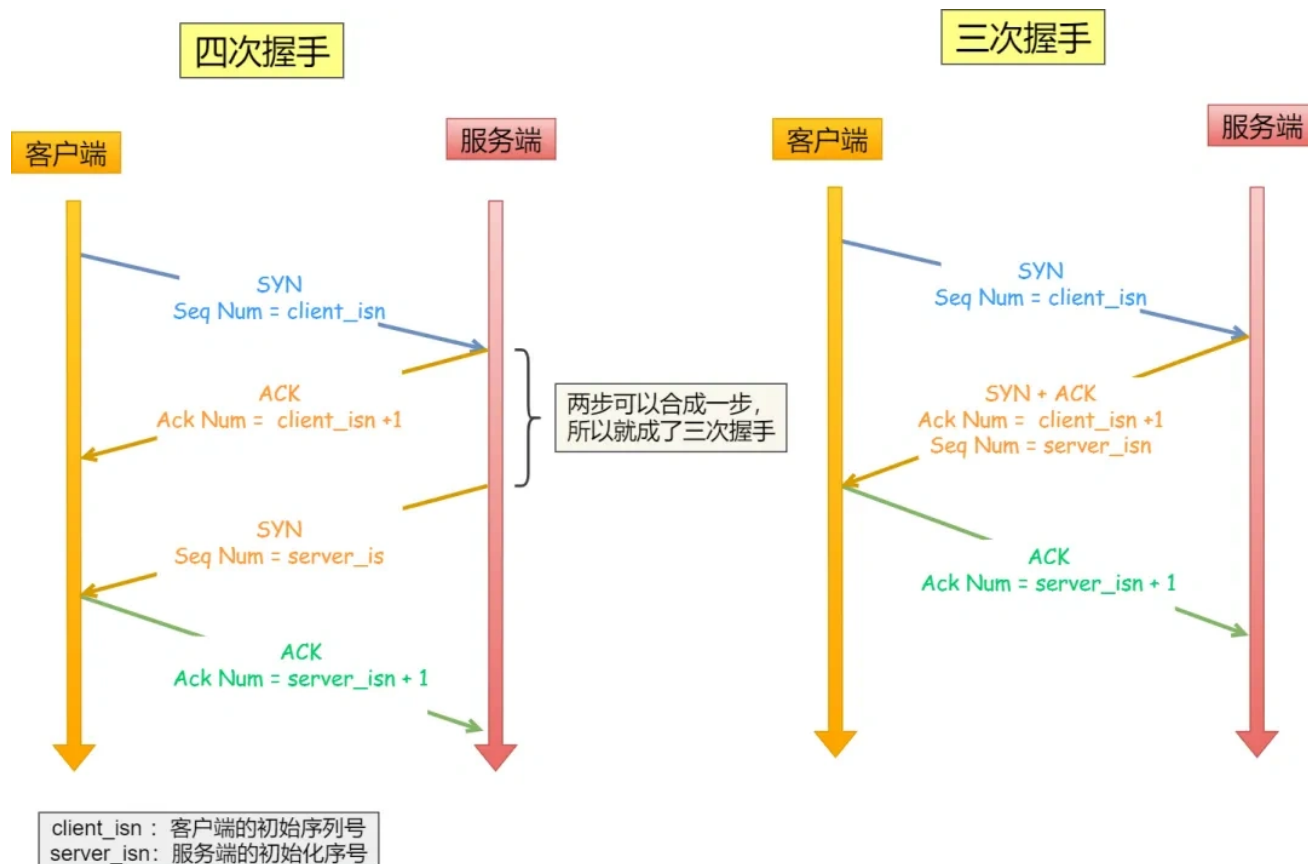
所以, TCP 使用三次握手建立连接的最主要原因是防止「历史连接」初始化了连接。

原因二: 同步双方初始序列号

TCP 协议的通信双方, 都必须维护一个「序列号」, 序列号是可靠传输的一个关键因素, 它的作用:

- 接收方可以去除重复的数据;
- 接收方可以根据数据包的序列号按序接收;
- 可以标识发送出去的数据包中, 哪些是已经被对方收到的 (通过 ACK 报文中的序列号知道);

可见，序列号在 TCP 连接中占据着非常重要的作用，所以当客户端发送携带「初始序列号」的 SYN 报文的时候，需要服务端回一个 ACK 应答报文，表示客户端的 SYN 报文已被服务端成功接收，那当服务端发送「初始序列号」给客户端的时候，依然也要得到客户端的应答回应，**这样一来一回，才能确保双方的初始序列号能被可靠的同步。**



四次握手与三次握手

四次握手其实也能够可靠的同步双方的初始化序号，但由于**第二步和第三步可以优化成一步**，所以就变成了「三次握手」。

而两次握手只保证了一方的初始序列号能被对方成功接收，没办法保证双方的初始序列号都能被确认接收。

原因三：避免资源浪费

如果只有「两次握手」，当客户端发生的 SYN 报文在网络中阻塞，客户端没有接收到 ACK 报文，就会重新发送 SYN，**由于没有第三次握手，服务端不清楚客户端是否收到了自己回复的 ACK 报文，所以服务端每收到一个 SYN 就只能先主动建立一个连接**，这会造成什么情况呢？

如果客户端发送的 SYN 报文在网络中阻塞了，重复发送多次 SYN 报文，那么服务端在收到请求后就会**建立多个冗余的无效链接，造成不必要的资源浪费。**

两次握手

客户端

服务端

超时时间

① SYN
Seq Num = client_isn

网络拥堵阻塞

超时重发

② SYN
Seq Num = client_isn

分配资源
连接建立

分配资源
连接建立

SYN + ACK
Ack Num = client_isn + 1
Seq Num = server_isn

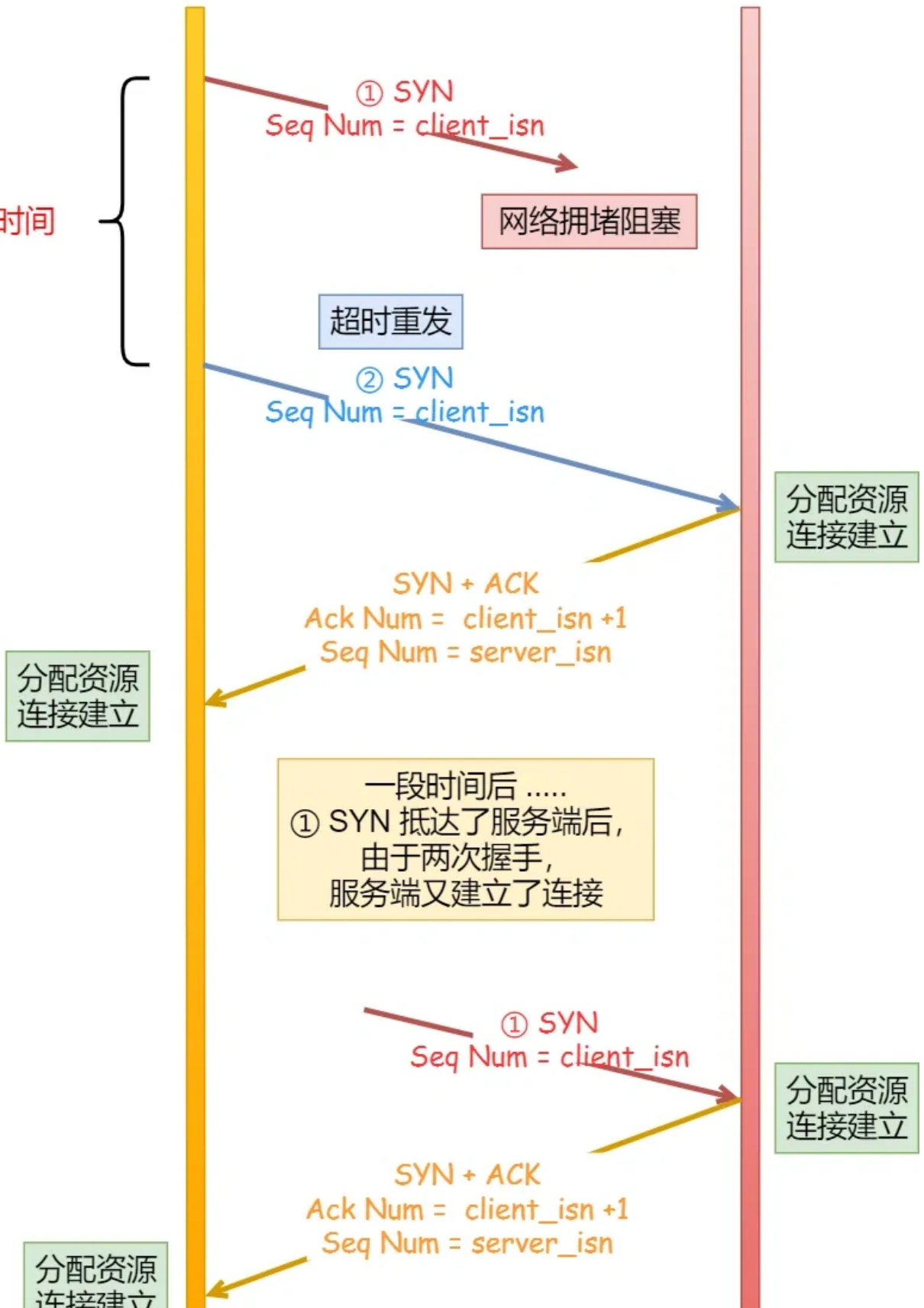
一段时间后
① SYN 抵达了服务端后,
由于两次握手,
服务端又建立了连接

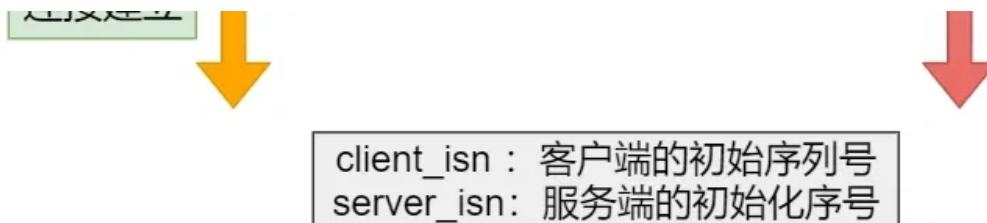
① SYN
Seq Num = client_isn

分配资源
连接建立

SYN + ACK
Ack Num = client_isn + 1
Seq Num = server_isn

分配资源
连接建立





即两次握手会造成消息滞留情况下，服务端重复接受无用的连接请求 SYN 报文，而造成重复分配资源。

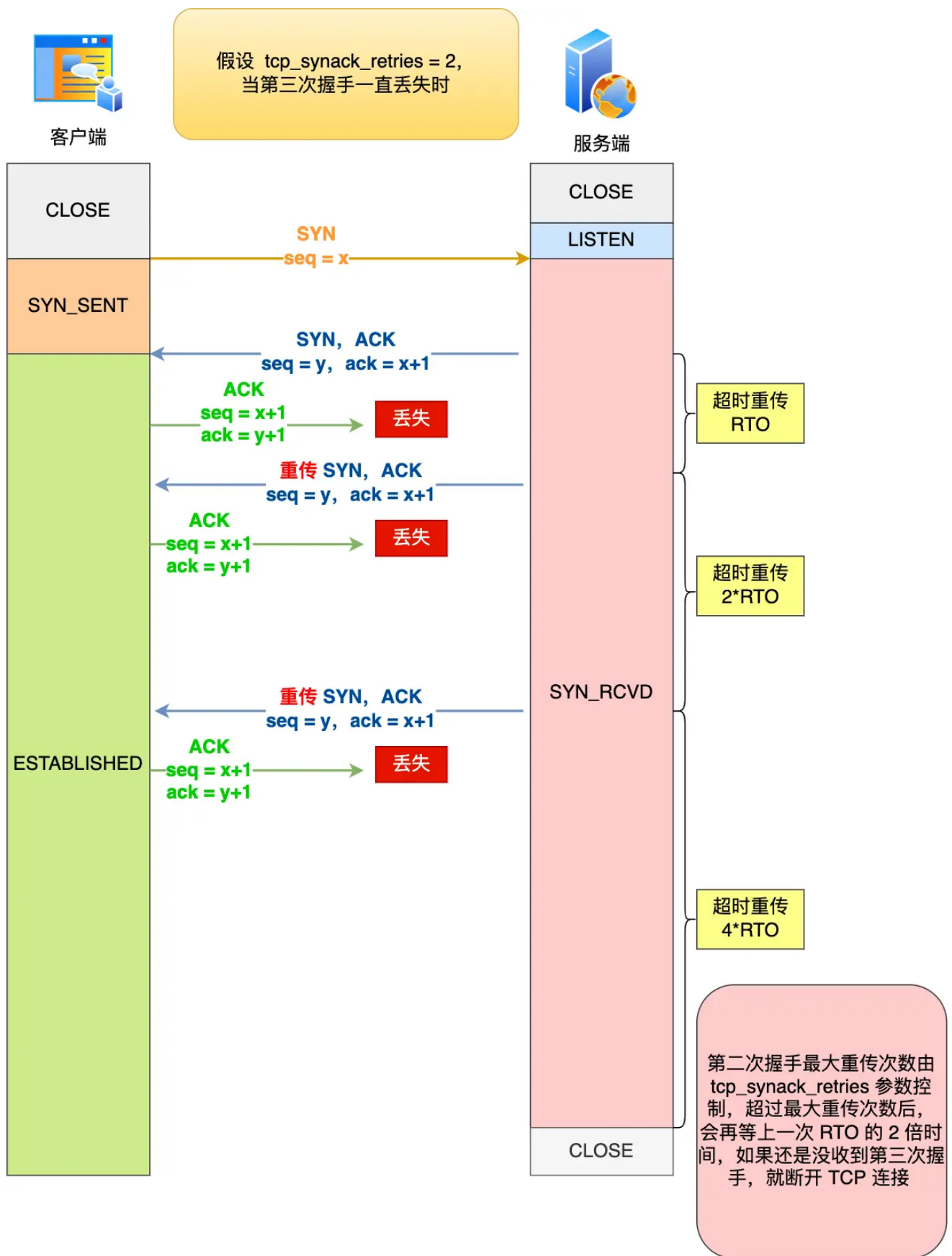
TCP 三次握手，客户端第三次发送的确认包丢失了发生什么？

客户端收到服务端的 SYN-ACK 报文后，就会给服务端回一个 ACK 报文，也就是第三次握手，此时客户端状态进入到 ESTABLISH 状态。

因为这个第三次握手的 ACK 是对第二次握手的 SYN 的确认报文，所以当第三次握手丢失了，如果服务端那一方迟迟收不到这个确认报文，就会触发超时重传机制，重传 SYN-ACK 报文，直到收到第三次握手，或者达到最大重传次数。

注意，ACK 报文是不会有重传的，当 ACK 丢失了，就由对方重传对应的报文。

举个例子，假设 tcp_synack_retries 参数值为 2，那么当第三次握手一直丢失时，发生的过程如下图：



具体过程：

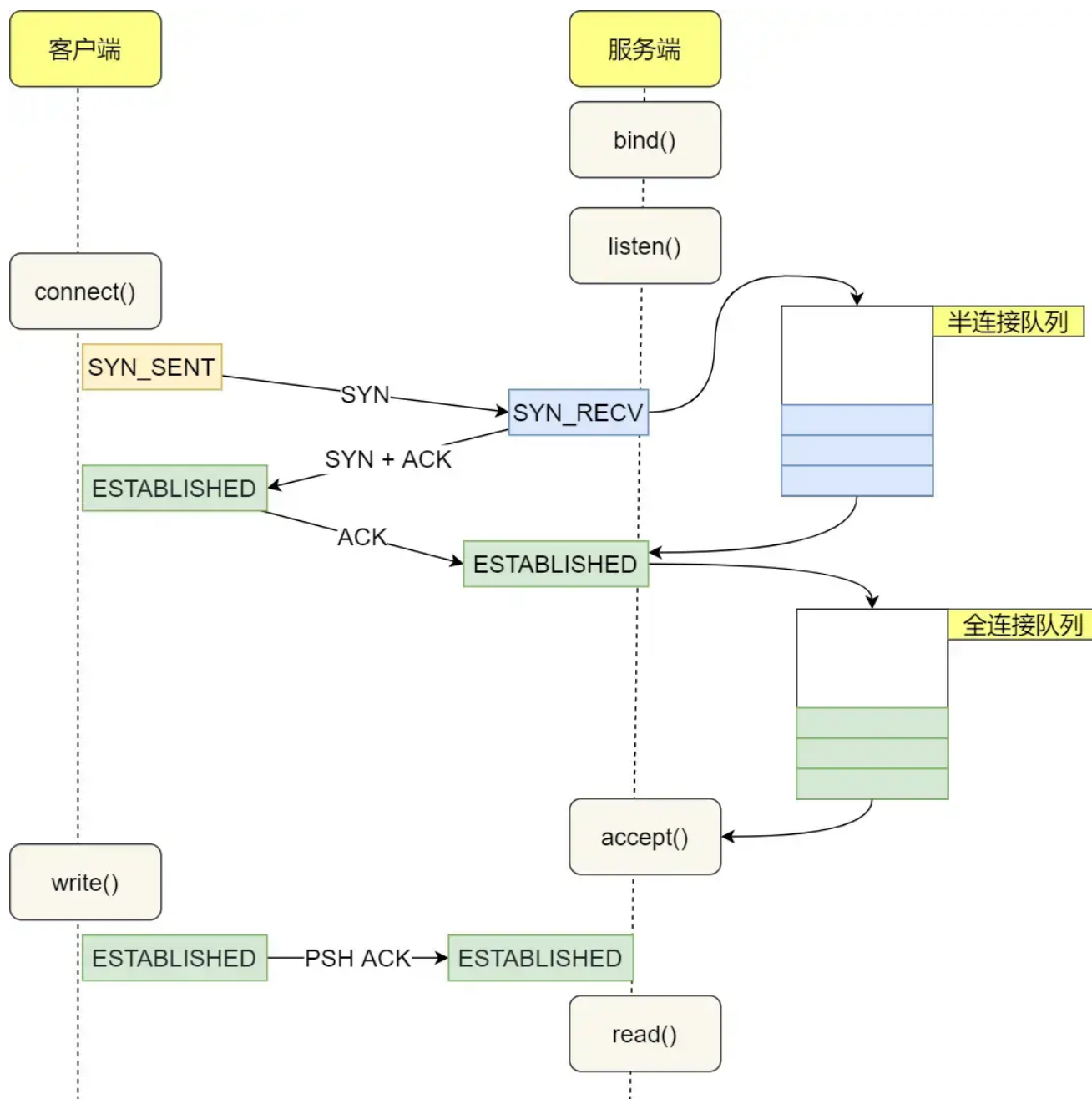
- 当服务端超时重传 2 次 SYN-ACK 报文后，由于 tcp_synack_retries 为 2，已达到最大重传次数，于是再等待一段时间（时间为上一次超时时间的 2 倍），如果还是没能收到客户端的第三次握手（ACK 报文），那么服务端就会断开连接。

服务端发送第二个报文后连接的状态进入什么状态

syn_rcvd 状态

三次握手和 accept 是什么关系？ accept 做了哪些事情？

tcp 完成三次握手后，连接会被保存到内核的全连接队列，调用 accept 就是从把连接取出来给用户程序使用。



客户端发送的第一个 SYN 报文，服务器没有收到怎么办？

当客户端想和服务端建立 TCP 连接的时候，首先第一个发的就是 SYN 报文，然后进入到 SYN_SENT 状态。

在这之后，如果客户端迟迟收不到服务端的 SYN-ACK 报文（第二次握手），就会触发「超时重传」机制，重传 SYN 报文，而且重传的 SYN 报文的序列号都是一样的。

不同版本的操作系统可能超时时间不同，有的 1 秒的，也有 3 秒的，这个超时时间是写死在内核里的，如果想要更改则需要重新编译内核，比较麻烦。

当客户端在 1 秒后没收到服务端的 SYN-ACK 报文后，客户端就会重发 SYN 报文，那到底重发几次呢？

在 Linux 里，客户端的 SYN 报文最大重传次数由 `tcp_syn_retries` 内核参数控制，这个参数是可以自定义的，默认值一般是 5。

```
# cat /proc/sys/net/ipv4/tcp_syn_retries
5
```

通常，第一次超时重传是在 1 秒后，第二次超时重传是在 2 秒，第三次超时重传是在 4 秒后，第四次超时重传是在 8 秒后，第五次是在超时重传 16 秒后。没错，**每次超时的时间是上一次的 2 倍**。

当第五次超时重传后，会继续等待 32 秒，如果服务端仍然没有回应 ACK，客户端就不再发送 SYN 包，然后断开 TCP 连接。

所以，总耗时是 $1+2+4+8+16+32=63$ 秒，大约 1 分钟左右。

举个例子，假设 `tcp_syn_retries` 参数值为 3，那么当客户端的 SYN 报文一直在网络中丢失时，会发生下图的过程：



具体过程:

- 当客户端超时重传 3 次 SYN 报文后, 由于 `tcp_syn_retries` 为 3, 已达到最大重传次数, 于是再等待一段时间 (时间为上一次超时时间的 2 倍), 如果还是没能收到服务端的第二次握手 (SYN-ACK 报文), 那么客户端就会断开连接。

服务器收到第一个 SYN 报文，回复的 SYN + ACK 报文丢失了怎么办？

当服务端收到客户端的第一次握手后，就会回 SYN-ACK 报文给客户端，这个就是第二次握手，此时服务端会进入 SYN_RCVD 状态。

第二次握手的 SYN-ACK 报文其实有两个目的：

- 第二次握手里的 ACK，是对第一次握手的确认报文；
- 第二次握手里的 SYN，是服务端发起建立 TCP 连接的报文；

所以，如果第二次握手丢了，就会发生比较有意思的事情，具体会怎么样呢？

因为第二次握手报文里是包含对客户端的第一次握手的 ACK 确认报文，所以，如果客户端迟迟没有收到第二次握手，那么客户端就觉得可能自己的 SYN 报文（第一次握手）丢失了，于是**客户端就会触发超时重传机制，重传 SYN 报文。**

然后，因为第二次握手中包含服务端的 SYN 报文，所以当客户端收到后，需要给服务端发送 ACK 确认报文（第三次握手），服务端才会认为该 SYN 报文被客户端收到了。

那么，如果第二次握手丢失了，服务端就收不到第三次握手，于是**服务端这边会触发超时重传机制，重传 SYN-ACK 报文。**

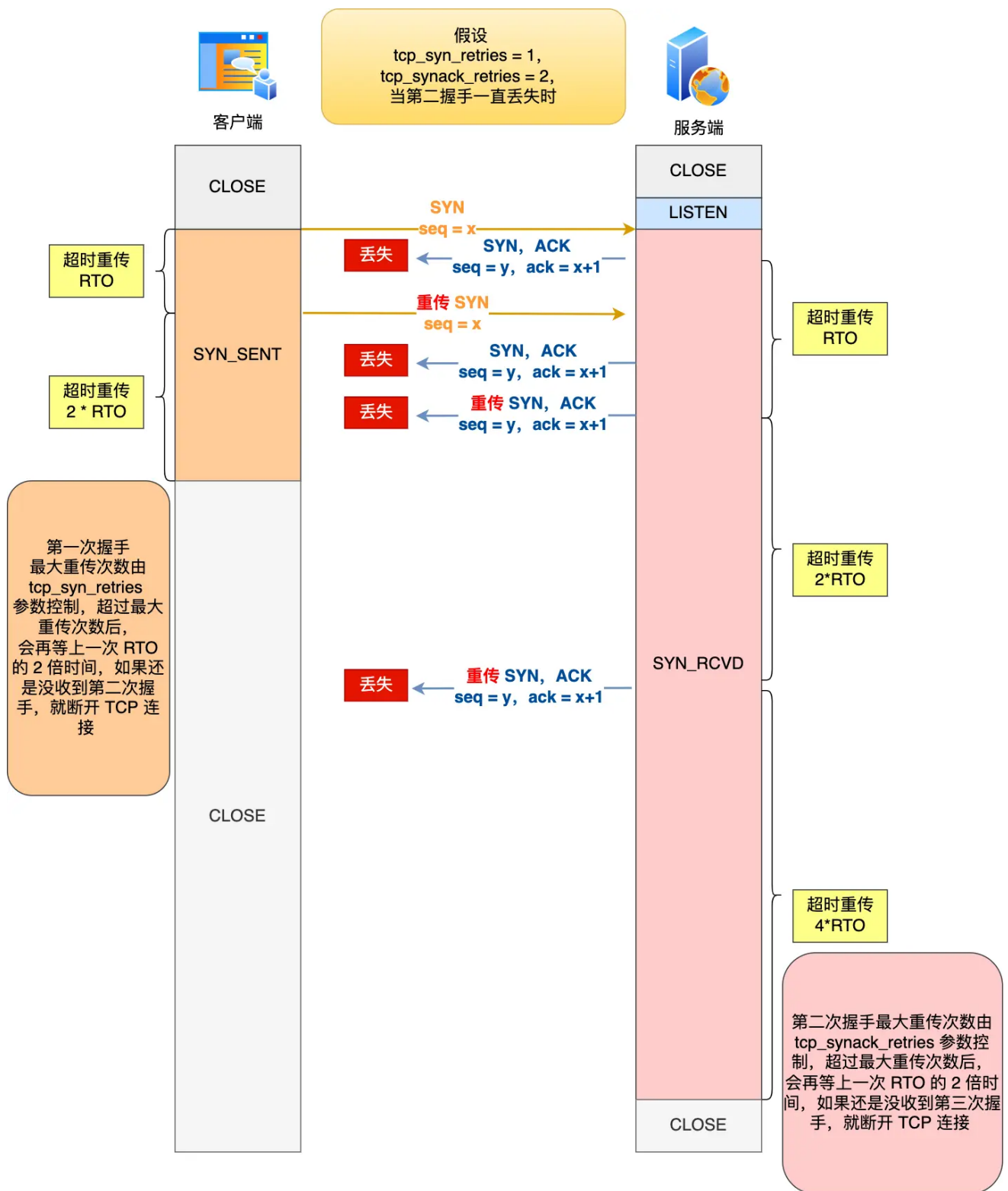
在 Linux 下，SYN-ACK 报文的最大重传次数由 tcp_synack_retries 内核参数决定，默认值是 5。

```
# cat /proc/sys/net/ipv4/tcp_synack_retries
5
```

因此，当第二次握手丢失了，客户端和服务端都会重传：

- 客户端会重传 SYN 报文，也就是第一次握手，最大重传次数由 tcp_syn_retries 内核参数决定；
- 服务端会重传 SYN-ACK 报文，也就是第二次握手，最大重传次数由 tcp_synack_retries 内核参数决定。

举个例子，假设 tcp_syn_retries 参数值为 1，tcp_synack_retries 参数值为 2，那么当第二次握手一直丢失时，发生的过程如下图：



具体过程:

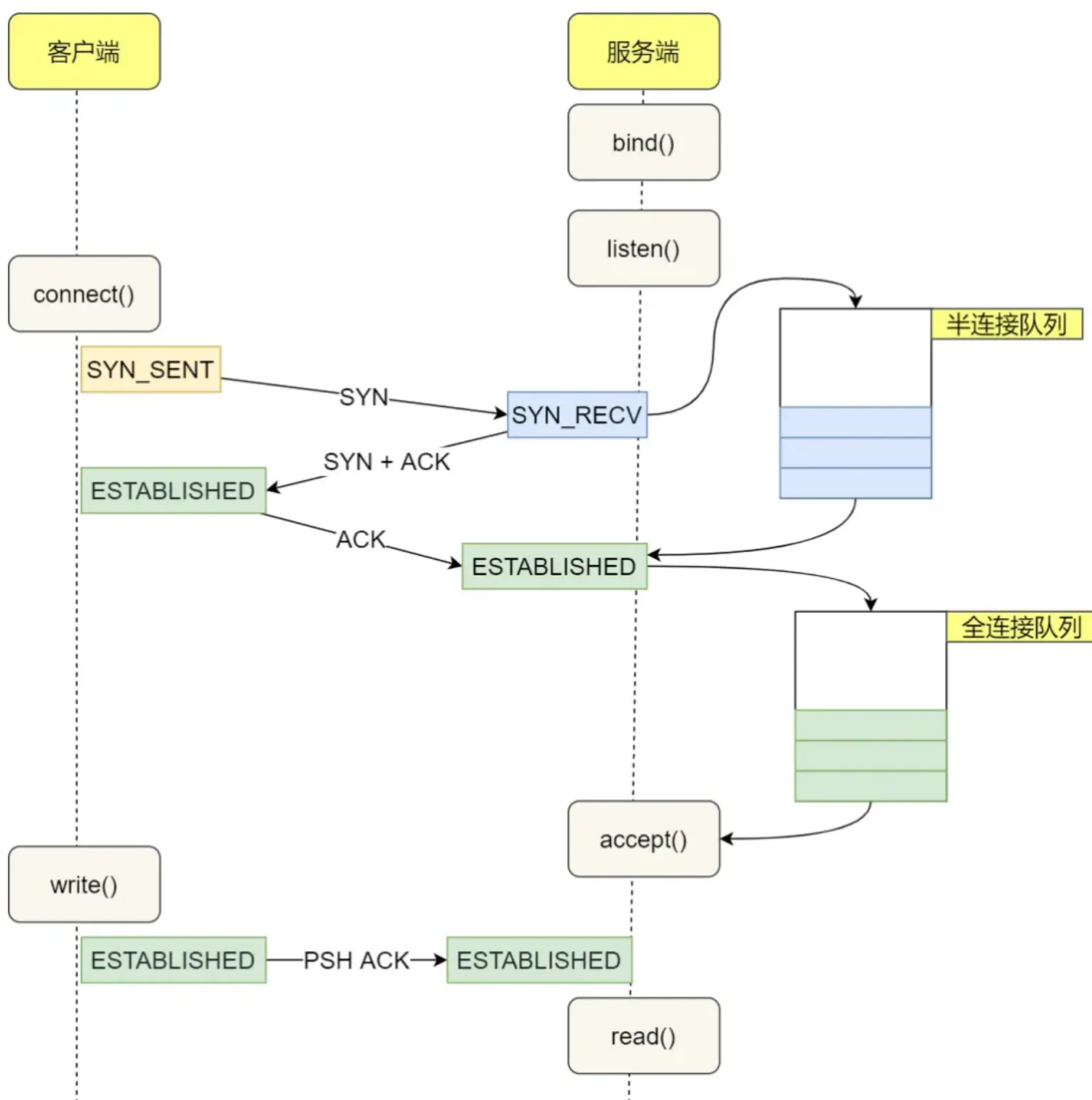
- 当客户端超时重传 1 次 SYN 报文后, 由于 `tcp_syn_retries` 为 1, 已达到最大重传次数, 于是再等待一段时间 (时间为上一次超时时间的 2 倍), 如果还是没能收到服务端的第二次握手 (SYN-ACK 报文), 那么客户端就会断开连接。
- 当服务端超时重传 2 次 SYN-ACK 报文后, 由于 `tcp_synack_retries` 为 2, 已达到最大重传次数, 于是再等待一段时间 (时间为上一次超时时间的 2 倍), 如果还是没能收到客户端的第三次握手 (ACK 报文), 那么服务端就会断开连接。

假设客户端重传了 SYN 报文，服务端这边又收到重复的 SYN 报文怎么办？

会继续发送第二次握手报文。

第一次握手，客户端发送SYN报后，服务端回复ACK报，那这个过程中服务端内部做了哪些工作？

服务端收到客户端发起的 SYN 请求后，内核会把该连接存储到半连接队列，并向客户端响应 SYN+ACK，接着客户端会返回 ACK，服务端收到第三次握手的 ACK 后，内核会把连接从半连接队列移除，然后创建新的完全的连接，并将其添加到 accept 队列，等待进程调用 accept 函数时把连接取出来。



不管是半连接队列还是全连接队列，都有最大长度限制，超过限制时，内核会直接丢弃，或返回 RST 包。

大量SYN包发送给服务端服务端会发生什么事情？

有可能会导致TCP 半连接队列打满，这样当 **TCP 半连接队列满了**，后续再在收到 **SYN 报文就会丢弃**，导致客户端无法和服务端建立连接。

避免 SYN 攻击方式，可以有以下四种方法：

- 调大 netdev_max_backlog;
- 增大 TCP 半连接队列;
- 开启 tcp_syncookies;
- 减少 SYN+ACK 重传次数

方式一：调大 netdev_max_backlog

当网卡接收数据包的速度大于内核处理的速度时，会有一个队列保存这些数据包。控制该队列的最大值如下参数，默认值是 1000，我们要适当调大该参数的值，比如设置为 10000：

```
net.core.netdev_max_backlog = 10000
```

方式二：增大 TCP 半连接队列

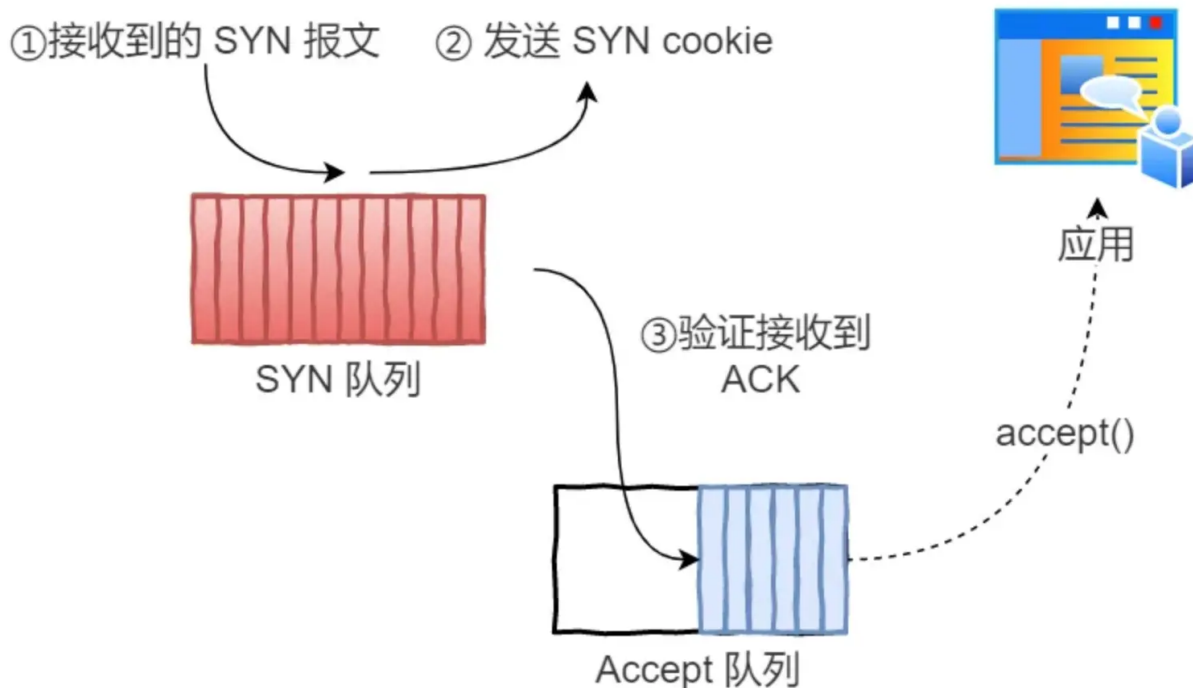
增大 TCP 半连接队列，要同时增大下面这三个参数：

- 增大 net.ipv4.tcp_max_syn_backlog
- 增大 listen() 函数中的 backlog
- 增大 net.core.somaxconn

方式三：开启 net.ipv4.tcp_syncookies

开启 syncookies 功能就可以在不使用 SYN 半连接队列的情况下成功建立连接，相当于绕过了 SYN 半连接来建立连接。

SYN 队列占满，启动 cookie



具体过程：

- 当「SYN 队列」满之后，后续服务端收到 SYN 包，不会丢弃，而是根据算法，计算出一个 `cookie` 值；
- 将 `cookie` 值放到第二次握手报文的「序列号」里，然后服务端回第二次握手给客户端；
- 服务端接收到客户端的应答报文时，服务端会检查这个 ACK 包的合法性。如果合法，将该连接对象放入到「Accept 队列」。
- 最后应用程序通过调用 `accept()` 接口，从「Accept 队列」取出的连接。

可以看到，当开启了 `tcp_syncookies` 了，即使受到 SYN 攻击而导致 SYN 队列满时，也能保证正常的连接成功建立。

`net.ipv4.tcp_syncookies` 参数主要有以下三个值：

- 0 值，表示关闭该功能；
- 1 值，表示仅当 SYN 半连接队列放不下时，再启用它；
- 2 值，表示无条件开启功能；

那么在应对 SYN 攻击时，只需要设置为 1 即可。

```
$ echo 1 > /proc/sys/net/ipv4/tcp_syncookies
```

方式四：减少 SYN+ACK 重传次数

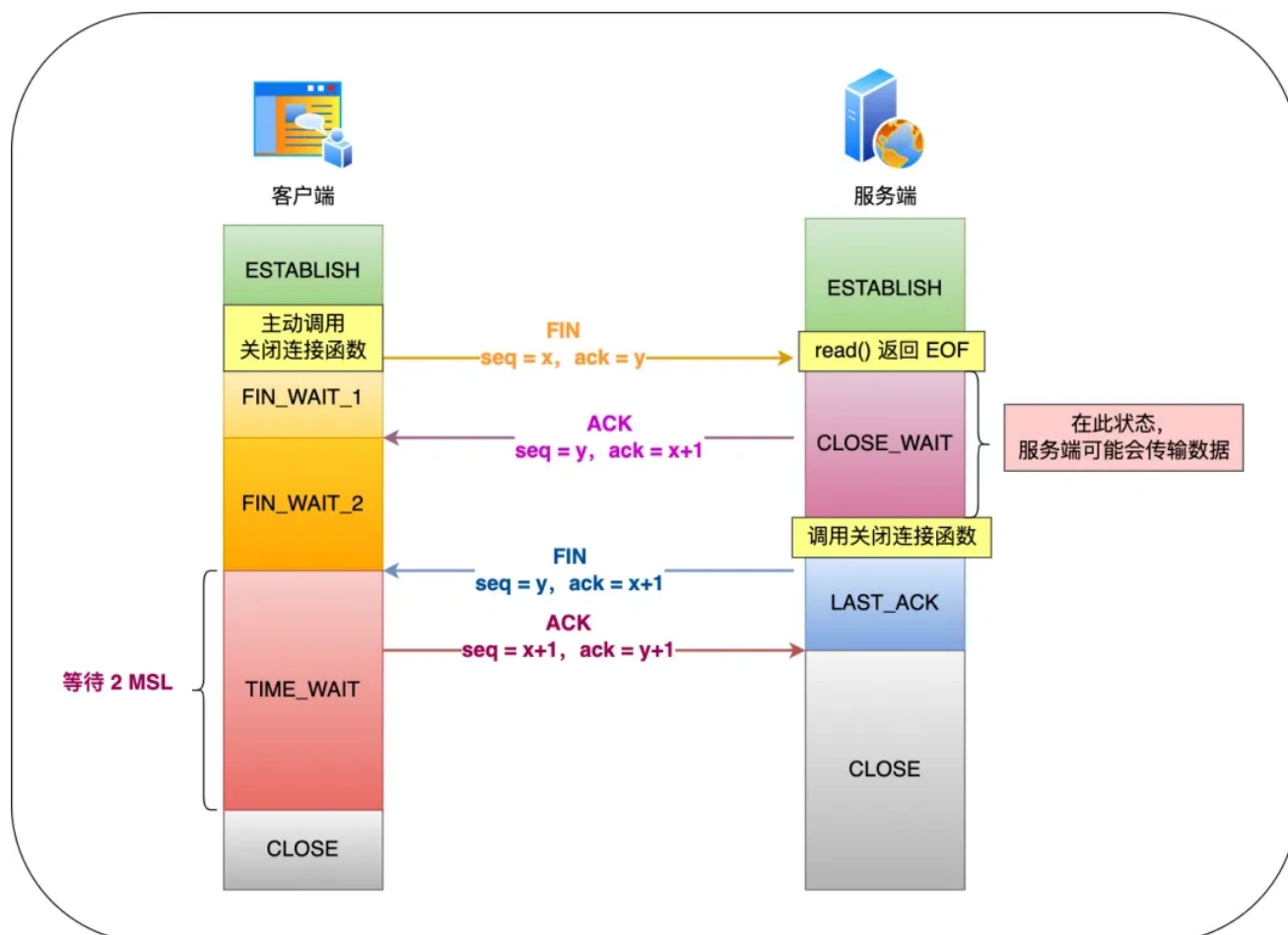
当服务端受到 SYN 攻击时，就会有大量处于 `SYN_RECV` 状态的 TCP 连接，处于这个状态的 TCP 会重传 SYN+ACK，当重传超过次数达到上限后，就会断开连接。

那么针对 SYN 攻击的场景，我们可以减少 SYN-ACK 的重传次数，以加快处于 `SYN_RECV` 状态的 TCP 连接断开。

SYN-ACK 报文的最大重传次数由 `tcp_synack_retries` 内核参数决定（默认值是 5 次），比如将 `tcp_synack_retries` 减少到 2 次：

```
$ echo 2 > /proc/sys/net/ipv4/tcp_synack_retries
```

TCP 四次挥手过程说一下？



具体过程：

- 客户端主动调用关闭连接的函数，于是就会发送 FIN 报文，这个 FIN 报文代表客户端不会再发送数据了，进入 FIN_WAIT_1 状态；
- 服务端收到了 FIN 报文，然后马上回复一个 ACK 确认报文，此时服务端进入 CLOSE_WAIT 状态。在收到 FIN 报文的时候，TCP 协议栈会为 FIN 包插入一个文件结束符 EOF 到接收缓冲区中，服务端应用程序可以通过 read 调用来感知这个 FIN 包，这个 EOF 会被放在已排队等候的其他已接收的数据之后，所以必须要得继续 read 接收缓冲区已接收的数据；
- 接着，当服务端在 read 数据的时候，最后自然会读到 EOF，接着 read() 就会返回 0，这时服务端应用程序如果有数据要发送的话，就发完数据后才调用关闭连接的函数，如果服务端应用程序没有数据要发送的话，可以直接调用关闭连接的函数，这时服务端就会发一个 FIN 包，这个 FIN 报文代表服务端不会再发送数据了，之后处于 LAST_ACK 状态；
- 客户端接收到服务端的 FIN 包，并发送 ACK 确认包给服务端，此时客户端将进入 TIME_WAIT 状态；
- 服务端收到 ACK 确认包后，就进入了最后的 CLOSE 状态；
- 客户端经过 2MSL 时间之后，也进入 CLOSE 状态；

为什么4次握手中间两次不能变成一次？

服务器收到客户端的 FIN 报文时，内核会马上回一个 ACK 应答报文，但是服务端应用程序可能还有数据要发送，所以并不能马上发送 FIN 报文，而是将发送 FIN 报文的控制权交给服务端应用程序：

- 如果服务端应用程序有数据要发送的话，就发完数据后，才调用关闭连接的函数；
- 如果服务端应用程序没有数据要发送的话，可以直接调用关闭连接的函数，

从上面过程可知，是否要发送第三次挥手的控制权不在内核，而是在被动关闭方（上图的服务端）的应用程序，因为应用程序可能还有数据要发送，由应用程序决定什么时候调用关闭连接的函数，当调用了关闭连接的函数，内核就会发送 FIN 报文了，所以服务端的 ACK 和 FIN 一般都会分开发送。

第二次和第三次挥手能合并嘛

当被动关闭方在 TCP 挥手过程中，「没有数据要发送」并且「开启了 TCP 延迟确认机制」，那么第二和第三次挥手就会合并传输，这样就出现了三次挥手。

Protocol	Length	Info
TCP	74	50468 → 8888 [SYN] Seq=1768572993 Win=43690 Len=0 MSS=
TCP	74	8888 → 50468 [SYN, ACK] Seq=3710633045 Ack=1768572994
TCP	66	50468 → 8888 [ACK] Seq=1768572994 Ack=3710633046 Win=4
TCP	85	50468 → 8888 [PSH, ACK] Seq=1768572994 Ack=3710633046
TCP	66	8888 → 50468 [ACK] Seq=3710633046 Ack=1768573013 Win=4
TCP	66	50468 → 8888 [FIN, ACK] Seq=1768573013 Ack=3710633047 Win=4
TCP	66	8888 → 50468 [FIN, ACK] Seq=3710633046 Ack=1768573014
TCP	66	50468 → 8888 [ACK] Seq=1768573014 Ack=3710633047 Win=4

第三次挥手一直没发，会发生什么？

当主动方收到 ACK 报文后，会处于 FIN_WAIT2 状态，就表示主动方的发送通道已经关闭，接下来将等待对方发送 FIN 报文，关闭对方的发送通道。

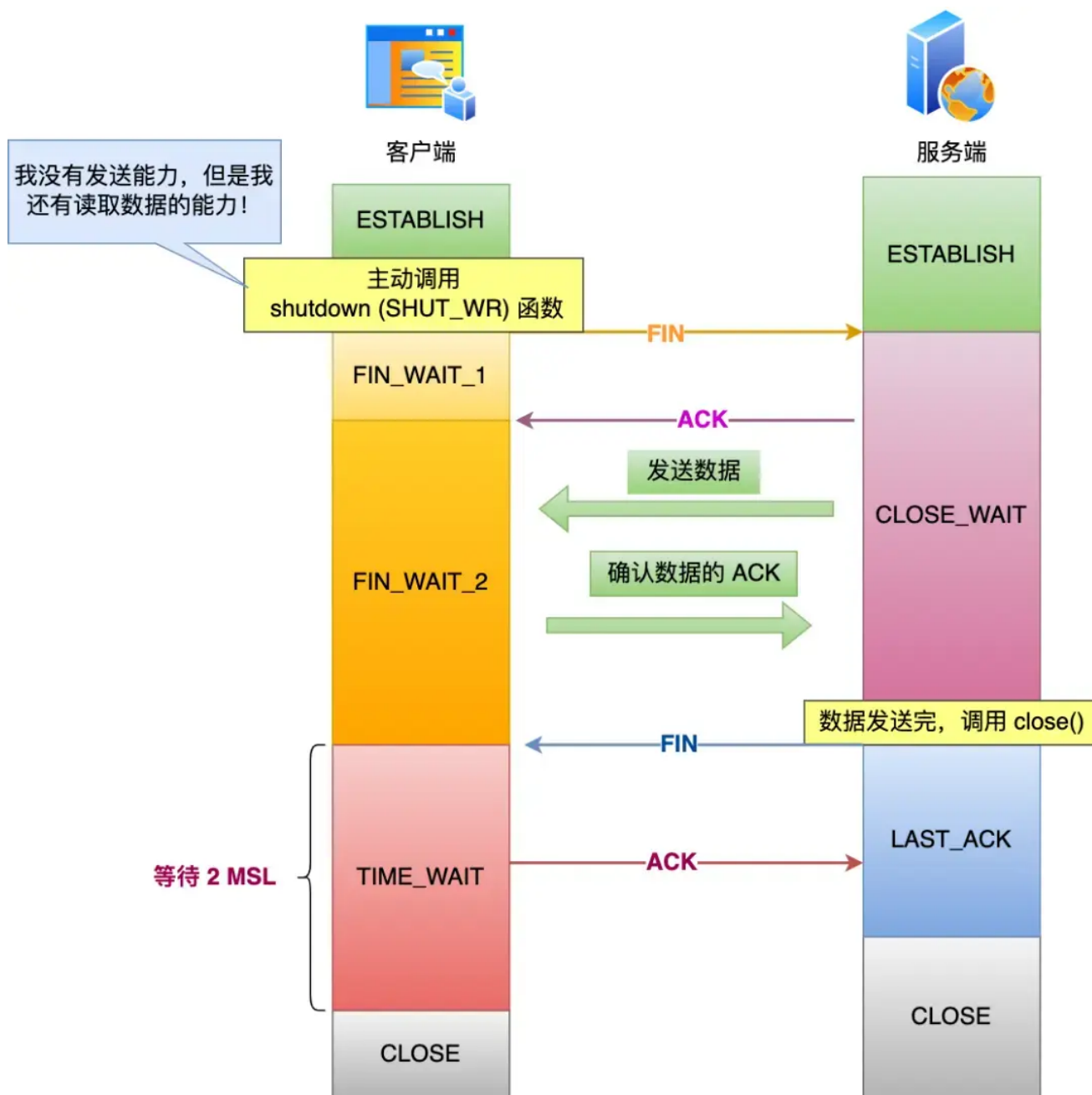
这时，如果连接是用 shutdown 函数关闭的，连接可以一直处于 FIN_WAIT2 状态，因为它可能还可以发送或接收数据。但对于 close 函数关闭的孤儿连接，由于无法再发送和接收数据，所以这个状态不可以持续太久，而 tcp_fin_timeout 控制了这个状态下连接的持续时长，默认值是 60 秒：

调整孤儿连接 FIN_WAIT2 状态的持续时间，默认值是 60
\$ echo 60 > /proc/sys/net/ipv4/tcp_fin_timeout

它意味着对于孤儿连接（调用 close 关闭的连接），如果在 60 秒后还没有收到 FIN 报文，连接就会直接关闭。

第二次和第三次挥手之间，主动断开的那端能干什么

如果主动断开的一方，是调用了 shutdown 函数来关闭连接，并且只选择了关闭发送能力且没有关闭接收能力的话，那么主动断开的一方在第二次和第三次挥手之间还可以接收数据。



断开连接时客户端 FIN 包丢失，服务端的状态是什么？

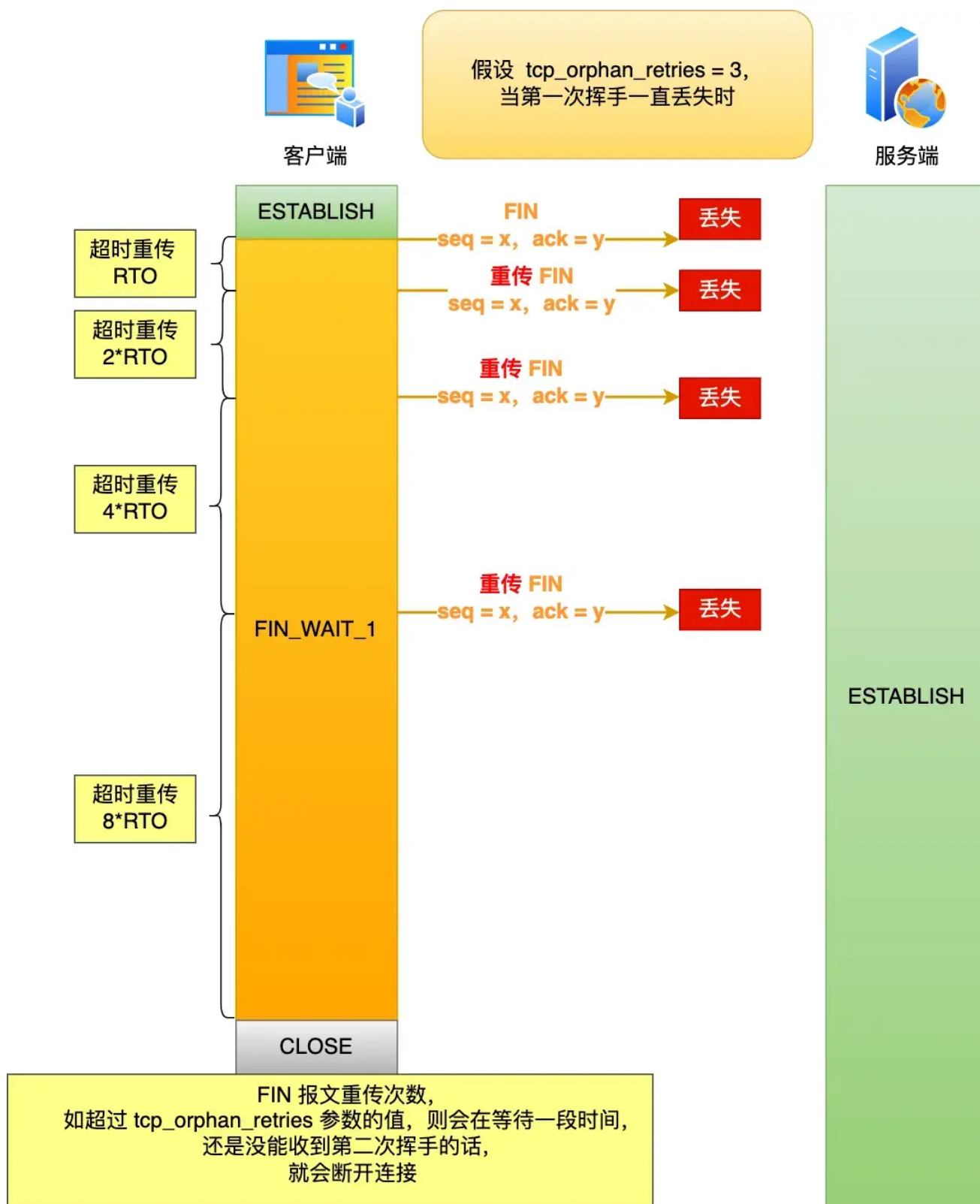
当客户端（主动关闭方）调用 close 函数后，就会向服务端发送 FIN 报文，试图与服务端断开连接，此时客户端的连接进入到 FIN_WAIT_1 状态。

正常情况下，如果能及时收到服务端（被动关闭方）的 ACK，则会很快变为 FIN_WAIT2 状态。

如果第一次挥手丢失了，那么客户端迟迟收不到被动方的 ACK 的话，也就会触发超时重传机制，重传 FIN 报文，重发次数由 tcp_orphan_retries 参数控制。

当客户端重传 FIN 报文的次数超过 tcp_orphan_retries 后，就不再发送 FIN 报文，则会在等待一段时间（时间为上一次超时时间的 2 倍），如果还是没能收到第二次挥手，那么客户端直接进入到 close 状态，而服务端还是 ESTABLISHED 状态

举个例子，假设 tcp_orphan_retries 参数值为 3，当第一次挥手一直丢失时，发生的过程如下图：



为什么四次挥手之后要等2MSL?

MSL 是 Maximum Segment Lifetime, **报文最大生存时间**, 它是任何报文在网络上存在的最长时间, 超过这个时间报文将被丢弃。因为 TCP 报文基于 IP 协议的, 而 IP 头中有一个 TTL 字段, 是 IP 数据报可以经过的最大路由数, 每经过一个处理他的路由器此值就减 1, 当此值为 0 则数据报将被丢弃, 同时发送 ICMP 报文通知源主机。

MSL 与 TTL 的区别: MSL 的单位是时间, 而 TTL 是经过路由跳数。所以 **MSL 应该要大于等于 TTL 消耗为 0 的时间**, 以确保报文已被自然消亡。

TTL 的值一般是 64，Linux 将 MSL 设置为 30 秒，意味着 Linux 认为数据报文经过 64 个路由器的时间不会超过 30 秒，如果超过了，就认为报文已经消失在网络中了。

TIME_WAIT 等待 2 倍的 MSL，比较合理的解释是：网络中可能存在来自发送方的数据包，当这些发送方的数据包被接收方处理后又向对方发送响应，所以一来一回需要等待 2 倍的时间。

比如，如果被动关闭方没有收到断开连接的最后的 ACK 报文，就会触发超时重发 FIN 报文，另一方接收到 FIN 后，会重发 ACK 给被动关闭方，一来一去正好 2 个 MSL。

可以看到 **2MSL 时长** 这其实是相当于 **至少允许报文丢失一次**。比如，若 ACK 在一个 MSL 内丢失，这样被动方重发的 FIN 会在第 2 个 MSL 内到达，TIME_WAIT 状态的连接可以应对。

服务端出现大量的timewait有哪些原因？

问题来了，什么场景下服务端会主动断开连接呢？

- 第一个场景：HTTP 没有使用长连接
- 第二个场景：HTTP 长连接超时
- 第三个场景：HTTP 长连接的请求数量达到上限

接下来，分别介绍下。

第一个场景：HTTP 没有使用长连接

我们先来看看 HTTP 长连接（Keep-Alive）机制是怎么开启的。

在 HTTP/1.0 中默认是关闭的，如果浏览器要开启 Keep-Alive，它必须在请求的 header 中添加：

```
Connection: Keep-Alive
```

然后当服务器收到请求，作出回应的时候，它也被添加到响应中 header 里：

```
Connection: Keep-Alive
```

这样做，TCP 连接就不会中断，而是保持连接。当客户端发送另一个请求时，它会使用同一个 TCP 连接。这一直继续到客户端或服务端提出断开连接。

从 HTTP/1.1 开始，就默认是开启了 Keep-Alive，现在大多数浏览器都默认是使用 HTTP/1.1，所以 Keep-Alive 都是默认打开的。一旦客户端和服务端达成协议，那么长连接就建立好了。

如果要关闭 HTTP Keep-Alive，需要在 HTTP 请求或者响应的 header 里添加 Connection:close 信息，也就是说，**只要客户端和服务端任意一方的 HTTP header 中有 Connection:close 信息，那么就无法使用 HTTP 长连接的机制。**

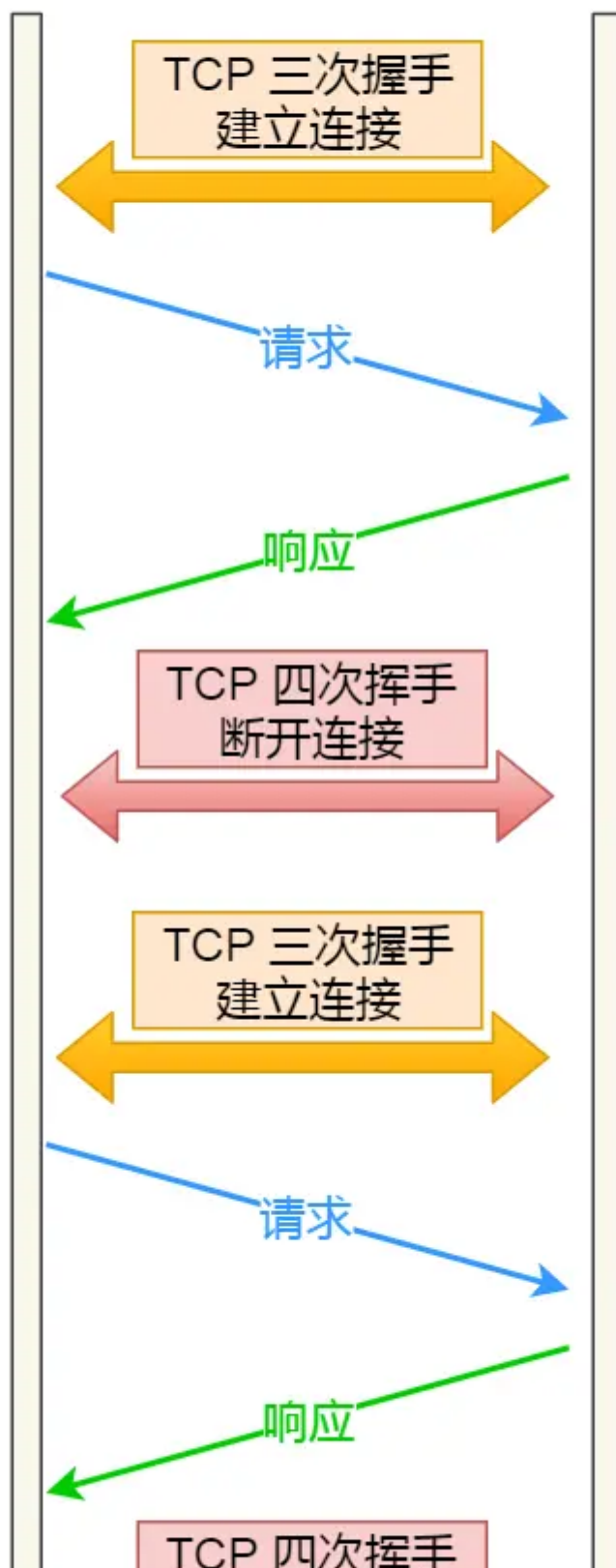
关闭 HTTP 长连接机制后，每次请求都要经历这样的过程：建立 TCP -> 请求资源 -> 响应资源 -> 释放连接，那么此方式就是 **HTTP 短连接**，如下图：

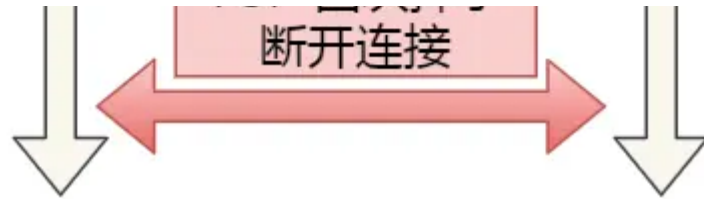


浏览器



服务器





在前面我们知道，只要任意一方的 HTTP header 中有 Connection:close 信息，就无法使用 HTTP 长连接机制，这样在完成一次 HTTP 请求/处理后，就会关闭连接。

问题来了，**这时候是客户端还是服务端主动关闭连接呢？**

在 RFC 文档中，并没有明确由谁来关闭连接，**请求和响应的双方都可以主动关闭 TCP 连接。**

不过，**根据大多数 Web 服务的实现，不管哪一方禁用了 HTTP Keep-Alive，都是由服务端主动关闭连接**，那么此时服务端上就会出现 TIME_WAIT 状态的连接。

第二个场景：HTTP 长连接超时

HTTP 长连接的特点是，只要任意一端没有明确提出断开连接，则保持 TCP 连接状态。

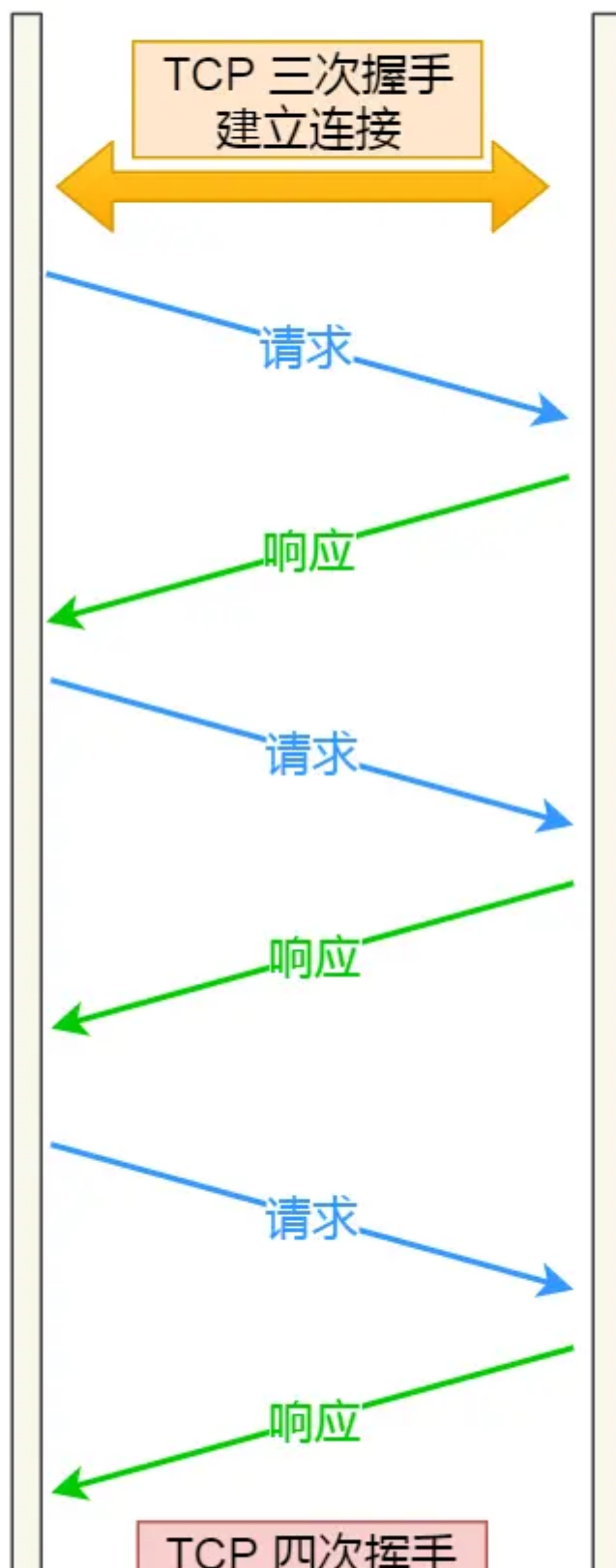
HTTP 长连接可以在同一个 TCP 连接上接收和发送多个 HTTP 请求/应答，避免了连接建立和释放的开销。

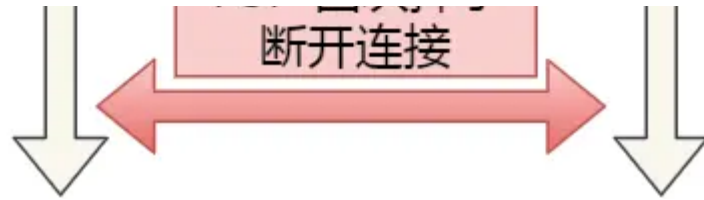


浏览器



服务器

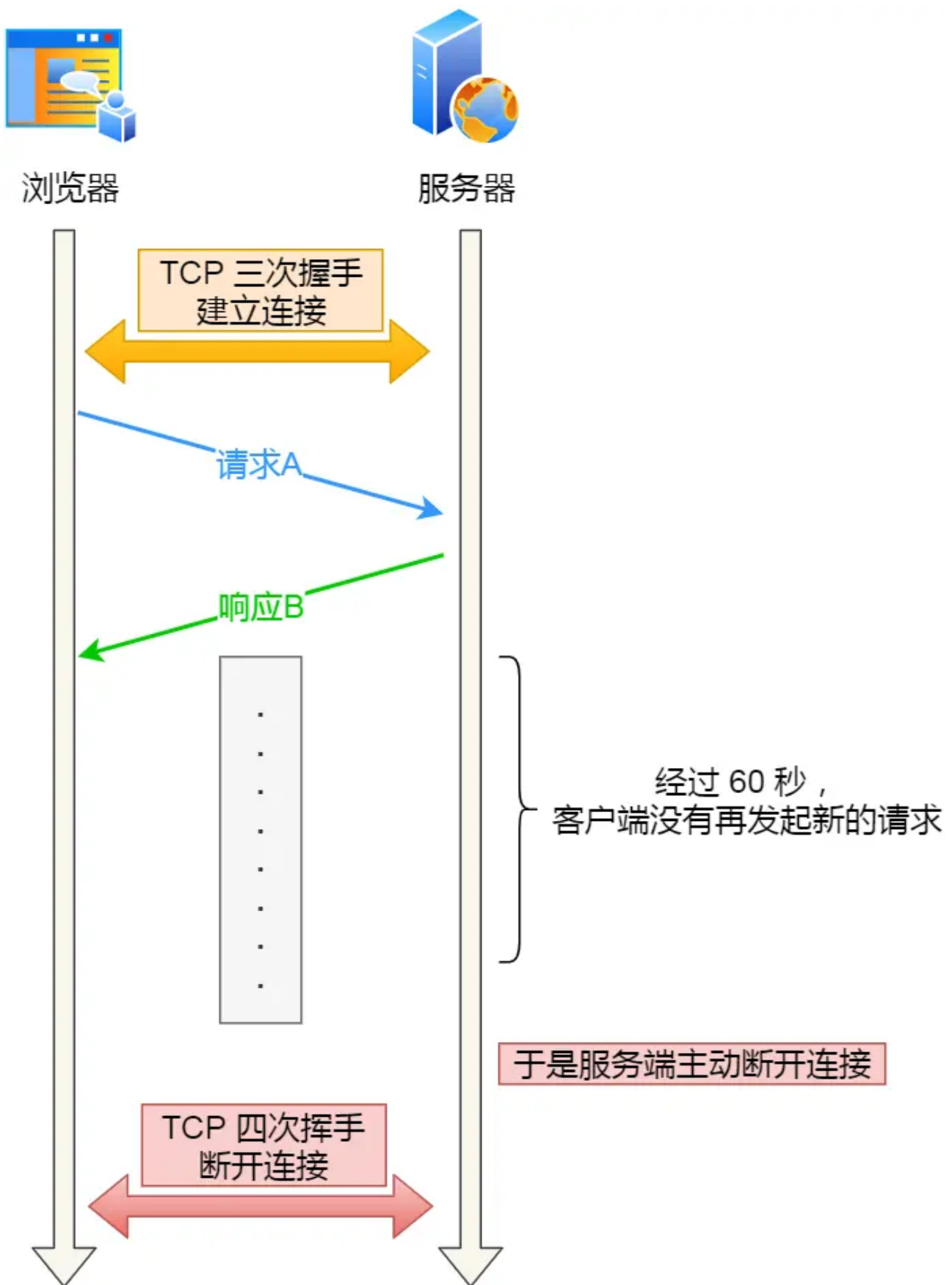




可能有的同学会问，如果使用了 HTTP 长连接，如果客户端完成一个 HTTP 请求后，就不再发起新的请求，此时这个 TCP 连接一直占用着不是挺浪费资源的吗？

对没错，所以为了避免资源浪费的情况，web 服务软件一般都会提供一个参数，用来指定 HTTP 长连接的超时时间，比如 nginx 提供的 `keepalive_timeout` 参数。

假设设置了 HTTP 长连接的超时时间是 60 秒，nginx 就会启动一个「定时器」，**如果客户端在完后一个 HTTP 请求后，在 60 秒内都没有再发起新的请求，定时器的时间一到，nginx 就会触发回调函数来关闭该连接，那么此时服务端上就会出现 `TIME_WAIT` 状态的连接。**



当服务端出现大量 TIME_WAIT 状态的连接时，如果现象是有大量的客户端建立完 TCP 连接后，很长一段时间没有发送数据，那么大概率就是因为 HTTP 长连接超时，导致服务端主动关闭连接，产生大量处于 TIME_WAIT 状态的连接。

可以往网络问题的方向排查，比如是否是因为网络问题，导致客户端发送的数据一直没有被服务端接收到，以至于 HTTP 长连接超时。

第三个场景：HTTP 长连接的请求数量达到上限

Web 服务端通常会有个参数，来定义一条 HTTP 长连接上最大能处理的请求数量，当超过最大限制时，就会主动关闭连接。

比如 nginx 的 `keepalive_requests` 这个参数，这个参数是指一个 HTTP 长连接建立之后，nginx 就会为这个连接设置一个计数器，记录这个 HTTP 长连接上已经接收并处理的客户端请求的数量。**如果达到这个参数设置的最大值时，则 nginx 会主动关闭这个长连接**，那么此时服务端上就会出现 `TIME_WAIT` 状态的连接。

`keepalive_requests` 参数的默认值是 100，意味着每个 HTTP 长连接最多只能跑 100 次请求，这个参数往往被大多数人忽略，因为当 QPS (每秒请求数) 不是很高时，默认值 100 凑合够用。

但是，**对于一些 QPS 比较高的场景，比如超过 10000 QPS，甚至达到 30000，50000 甚至更高，如果 `keepalive_requests` 参数值是 100，这时候就 nginx 就会很频繁地关闭连接，那么此时服务端上就会出大量的 `TIME_WAIT` 状态。**

针对这个场景下，解决的方式也很简单，调大 nginx 的 `keepalive_requests` 参数就行。

TCP和UDP区别是什么？

- 连接：TCP 是面向连接的传输层协议，传输数据前先要建立连接；UDP 是不需要连接，即刻传输数据。
- 服务对象：TCP 是一对一的两点服务，即一条连接只有两个端点。UDP 支持一对一、一对多、多对多的交互通信
- 可靠性：TCP 是可靠交付数据的，数据可以无差错、不丢失、不重复、按序到达。UDP 是尽最大努力交付，不保证可靠交付数据。但是我们可以基于 UDP 传输协议实现一个可靠的传输协议，比如 QUIC 协议
- 拥塞控制、流量控制：TCP 有拥塞控制和流量控制机制，保证数据传输的安全性。UDP 则没有，即使网络非常拥堵了，也不会影响 UDP 的发送速率。
- 首部开销：TCP 首部长度较长，会有一定的开销，首部在没有使用「选项」字段时是 20 个字节，如果使用了「选项」字段则会变长的。UDP 首部只有 8 个字节，并且是固定不变的，开销较小。
- 传输方式：TCP 是流式传输，没有边界，但保证顺序和可靠。UDP 是一个包一个包的发送，是有边界的，但可能会丢包和乱序。

TCP为什么可靠传输

TCP协议主要通过以下几点来保证传输可靠性：连接管理、序列号、确认应答、超时重传、流量控制、拥塞控制。

- **连接管理**：即三次握手和四次挥手。连接管理机制能够建立起可靠的连接，这是保证传输可靠性的前提。
- **序列号**：TCP将每个字节的数据都进行了编号，这就是序列号。序列号的具体作用如下：能够保证可靠性，既能防止数据丢失，又能避免数据重复。能够保证有序性，按照序列号顺序进行数据包还原。能够提高效率，基于序列号可实现多次发送，一次确认。
- **确认应答**：接收方接收数据之后，会回传ACK报文，报文中带有此次确认的序列号，用于告知发送方此次接收数据的情况。在指定时间后，若发送端仍未收到确认应答，就会启动超时重传。
- **超时重传**：超时重传主要有两种场景：数据包丢失：在指定时间后，若发送端仍未收到确认应答，就会启动超时重传，向接收端重新发送数据包。确认包丢失：当接收端收到重复数据(通过序列号进行识别)时将其丢弃，并重新回传ACK报文。
- **流量控制**：接收端处理数据的速度是有限的，如果发送方发送数据的速度过快，就会导致接收端的缓冲区溢出，进而导致丢包。为了避免上述情况的发生，TCP支持根据接收端的处理能力，来决定发送端的发送速度。这就是流量控制。流量控制是通过在TCP报文段首部维护一个滑动窗口来实现的。

- **拥塞控制**：拥塞控制就是当网络拥堵严重时，发送端减少数据发送。拥塞控制是通过发送端维护一个拥塞窗口来实现的。可以得出，发送端的发送速度，受限于滑动窗口和拥塞窗口中的最小值。拥塞控制方法分为：慢开始，拥塞避免、快重传和快恢复。

怎么用udp实现http?

UDP 是不可靠传输的，但基于 UDP 的 **QUIC 协议** 可以实现类似 TCP 的可靠性传输，在http3 就用了 quic 协议。

- 连接迁移：QUIC支持在网络变化时快速迁移连接，例如从WiFi切换到移动数据网络，以保持连接的可靠性。
- 重传机制：QUIC使用重传机制来确保丢失的数据包能够被重新发送，从而提高数据传输的可靠性。
- 前向纠错：QUIC可以使用前向纠错技术，在接收端修复部分丢失的数据，降低重传的需求，提高可靠性和传输效率。
- 拥塞控制：QUIC内置了拥塞控制机制，可以根据网络状况动态调整数据传输速率，以避免网络拥塞和丢包，提高可靠性。

tcp粘包怎么解决？

粘包的问题出现是因为不知道一个用户消息的边界在哪，如果知道了边界在哪，接收方就可以通过边界来划分出有效的用户消息。

一般有三种方式分包的方式：

- 固定长度的消息；
- 特殊字符作为边界；
- 自定义消息结构。

固定长度的消息

这种是最简单方法，即每个用户消息都是固定长度的，比如规定一个消息的长度是 64 个字节，当接收方接满 64 个字节，就认为这个内容是一个完整且有效的消息。

但是这种方式灵活性不高，实际中很少用。

特殊字符作为边界

我们可以在两个用户消息之间插入一个特殊的字符串，这样接收方在接收数据时，读到了这个特殊字符，就把认为已经读完一个完整的消息。

HTTP 是一个非常好的例子。

请求方法	空格	URL	空格	协议版本	回车	换行
头部字段名	:	值	回车	换行		
头部字段名	:	值	回车	换行		
回车	换行					
请求正文						

HTTP 通过设置回车符、换行符作为 HTTP 报文协议的边界。

有一点要注意，这个作为边界点的特殊字符，如果刚好消息内容里有这个特殊字符，我们要对这个字符转义，避免被接收方当作消息的边界点而解析到无效的数据。

自定义消息结构

我们可以自定义一个消息结构，由包头和数据组成，其中包头包是固定大小的，而且包头里有一个字段来说明紧随其后的数据有多大。

比如这个消息结构体，首先 4 个字节大小的变量来表示数据长度，真正的数据则在后面。

```
struct {
    u_int32_t message_length;
    char message_data[];
} message;
```

当接收方接收到包头的大小（比如 4 个字节）后，就解析包头的内容，于是就可以知道数据的长度，然后接下来就继续读取数据，直到读满数据的长度，就可以组装成一个完整到用户消息来处理了。

TCP的拥塞控制介绍一下？

一般来说，计算机网络都处在一个共享的环境。因此也有可能会因为其他主机之间的通信使得网络拥堵。

在网络出现拥堵时，如果继续发送大量数据包，可能会导致数据包时延、丢失等，这时 TCP 就会重传数据，但是一重传就会导致网络的负担更重，于是会导致更大的延迟以及更多的丢包，这个情况就会进入恶性循环被不断地放大....

所以，TCP 不能忽略网络上发生的事，它被设计成一个无私的协议，当网络发送拥塞时，TCP 会自我牺牲，降低发送的数据量。

于是，就有了**拥塞控制**，控制的目的是**避免「发送方」的数据填满整个网络**。

为了在「发送方」调节所要发送数据的量，定义了一个叫做「**拥塞窗口**」的概念。

拥塞窗口 cwnd是发送方维护的一个的状态变量，它会根据**网络的拥塞程度动态变化的**。发送窗口 swnd 和接收窗口 rwnd 是约等于的关系，那么由于加入了拥塞窗口的概念后，此时发送窗口的值是 $swnd = \min(cwnd, rwnd)$ ，也就是拥塞窗口和接收窗口中的最小值。

拥塞窗口 cwnd 变化的规则：

- 只要网络中没有出现拥塞，cwnd 就会增大；
- 但网络中出现了拥塞，cwnd 就减少；

那么怎么知道当前网络是否出现了拥塞呢？其实只要「发送方」没有在规定时间内接收到 ACK 应答报文，也就是**发生了超时重传，就会认为网络出现了拥塞**。

拥塞控制有哪些控制算法？

拥塞控制主要是四个算法：

- 慢启动
- 拥塞避免
- 拥塞发生
- 快速恢复

慢启动

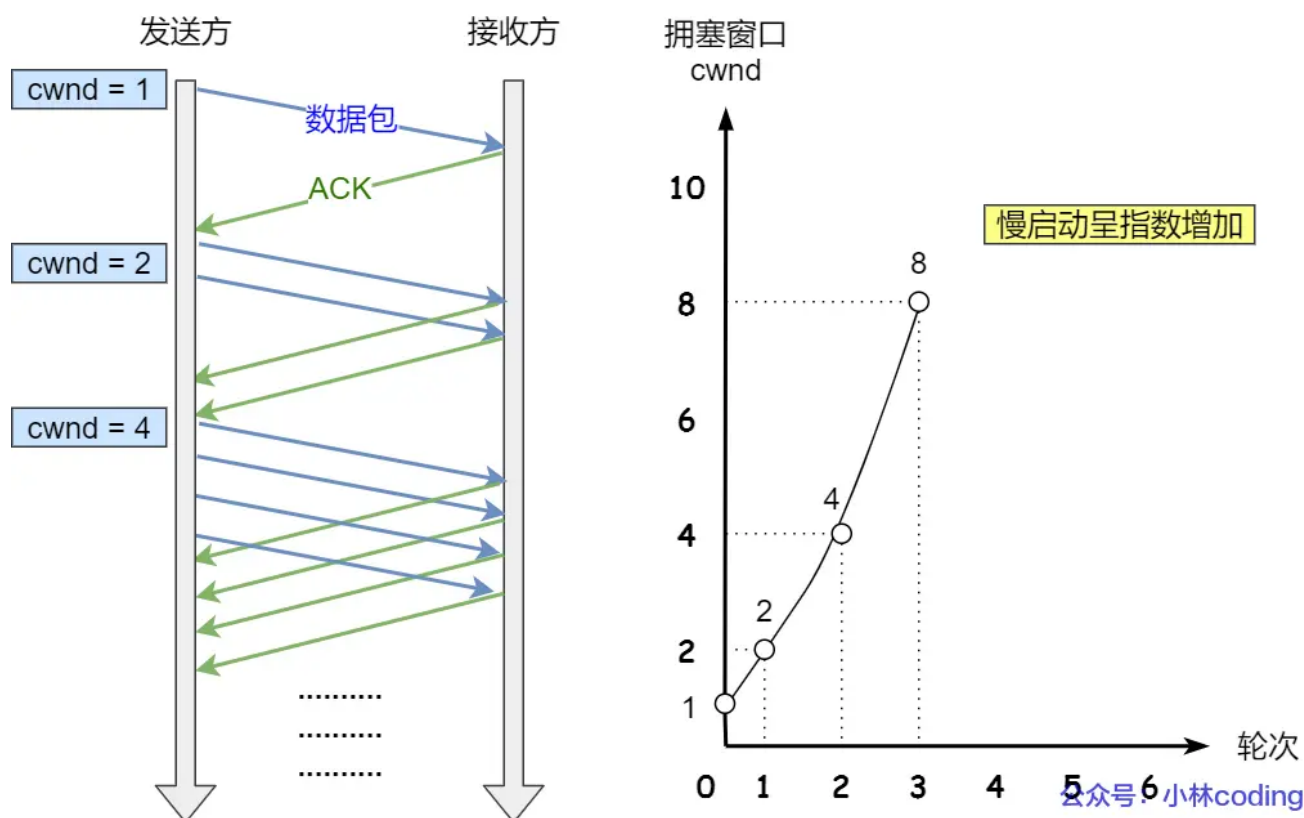
TCP 在刚建立连接完成后，首先是有个慢启动的过程，这个慢启动的意思就是一点一点的提高发送数据包的数量，如果一上来就发大量的数据，这不是给网络添堵吗？

慢启动的算法记住一个规则就行：**当发送方每收到一个 ACK，拥塞窗口 cwnd 的大小就会加 1。**

这里假定拥塞窗口 cwnd 和发送窗口 swnd 相等，下面举个栗子：

- 连接建立完成后，一开始初始化 $cwnd = 1$ ，表示可以传一个 MSS 大小的数据。
- 当收到一个 ACK 确认应答后， $cwnd$ 增加 1，于是一次能够发送 2 个
- 当收到 2 个的 ACK 确认应答后， $cwnd$ 增加 2，于是就可以比之前多发 2 个，所以这一次能够发送 4 个
- 当这 4 个的 ACK 确认到来的时候，每个确认 $cwnd$ 增加 1，4 个确认 $cwnd$ 增加 4，于是就可以比之前多发 4 个，所以这一次能够发送 8 个。

慢启动算法的变化过程如下图：



可以看出慢启动算法，发包的个数是**指数性的增长**。

那慢启动涨到什么时候是个头呢？

有一个叫慢启动门限 $ssthresh$ (slow start threshold) 状态变量。

- 当 $cwnd < ssthresh$ 时，使用慢启动算法。
- 当 $cwnd \geq ssthresh$ 时，就会使用「拥塞避免算法」。

拥塞避免算法

前面说道，当拥塞窗口 $cwnd$ 「超过」慢启动门限 $ssthresh$ 就会进入拥塞避免算法。

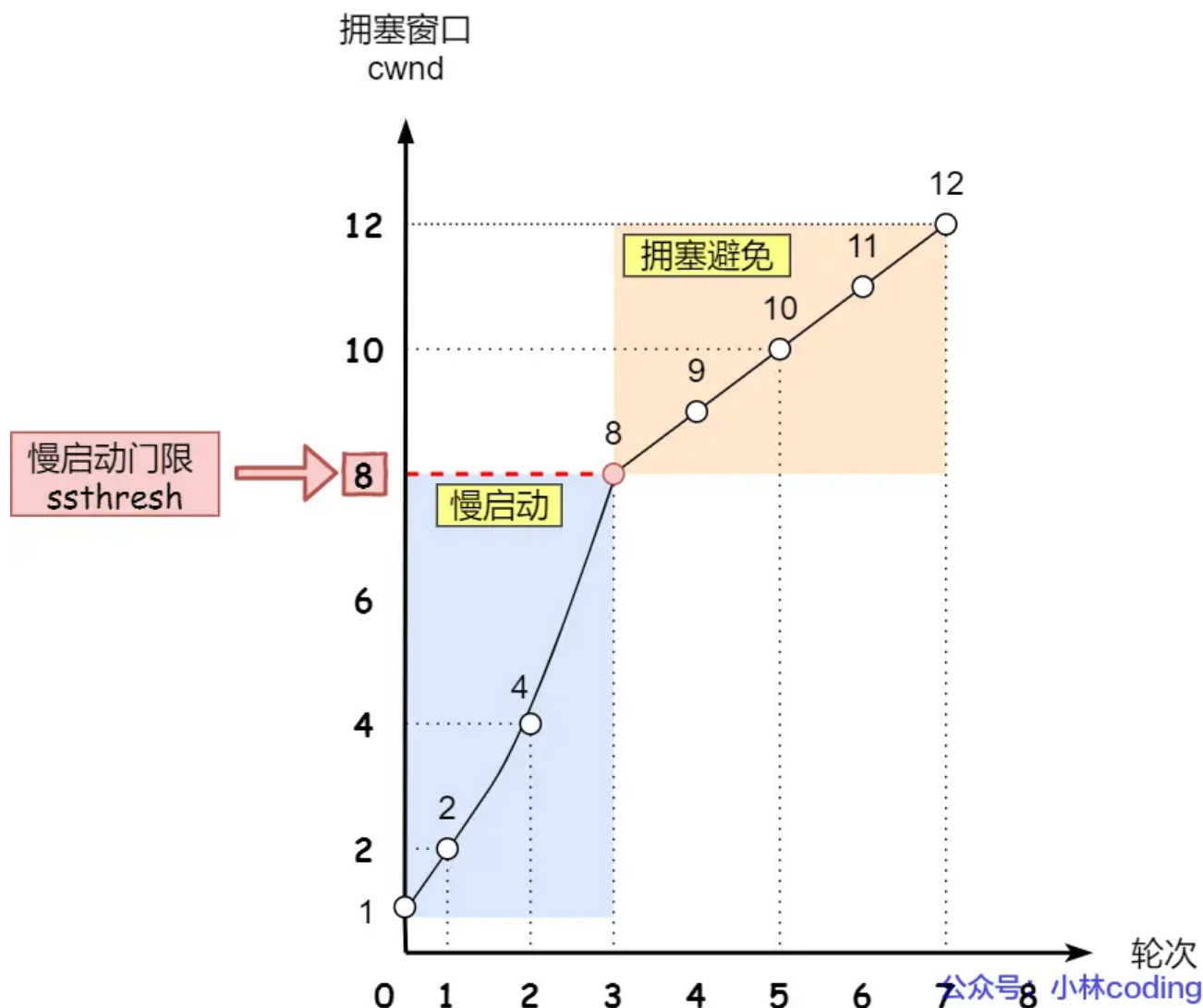
一般来说 $ssthresh$ 的大小是 65535 字节。

那么进入拥塞避免算法后，它的规则是：**每当收到一个 ACK 时， $cwnd$ 增加 $1/cwnd$ 。**

接上前面的慢启动的栗子，现假定 $ssthresh$ 为 8：

- 当 8 个 ACK 应答确认到来时，每个确认增加 1/8，8 个 ACK 确认 cwnd 一共增加 1，于是这一次能够发送 9 个 MSS 大小的数据，变成了**线性增长**。

拥塞避免算法的变化过程如下图：



所以，我们可以发现，拥塞避免算法就是将原本慢启动算法的指数增长变成了线性增长，还是增长阶段，但是增长速度缓慢了一些。

就这么一直增长着后，网络就会慢慢进入了拥塞的状况了，于是就会出现丢包现象，这时就需要对丢失的数据包进行重传。

当触发了重传机制，也就进入了「拥塞发生算法」。

拥塞发生

当网络出现拥塞，也就是会发生数据包重传，重传机制主要有两种：

- 超时重传
- 快速重传

这两种使用的拥塞发送算法是不同的，接下来分别来说说。

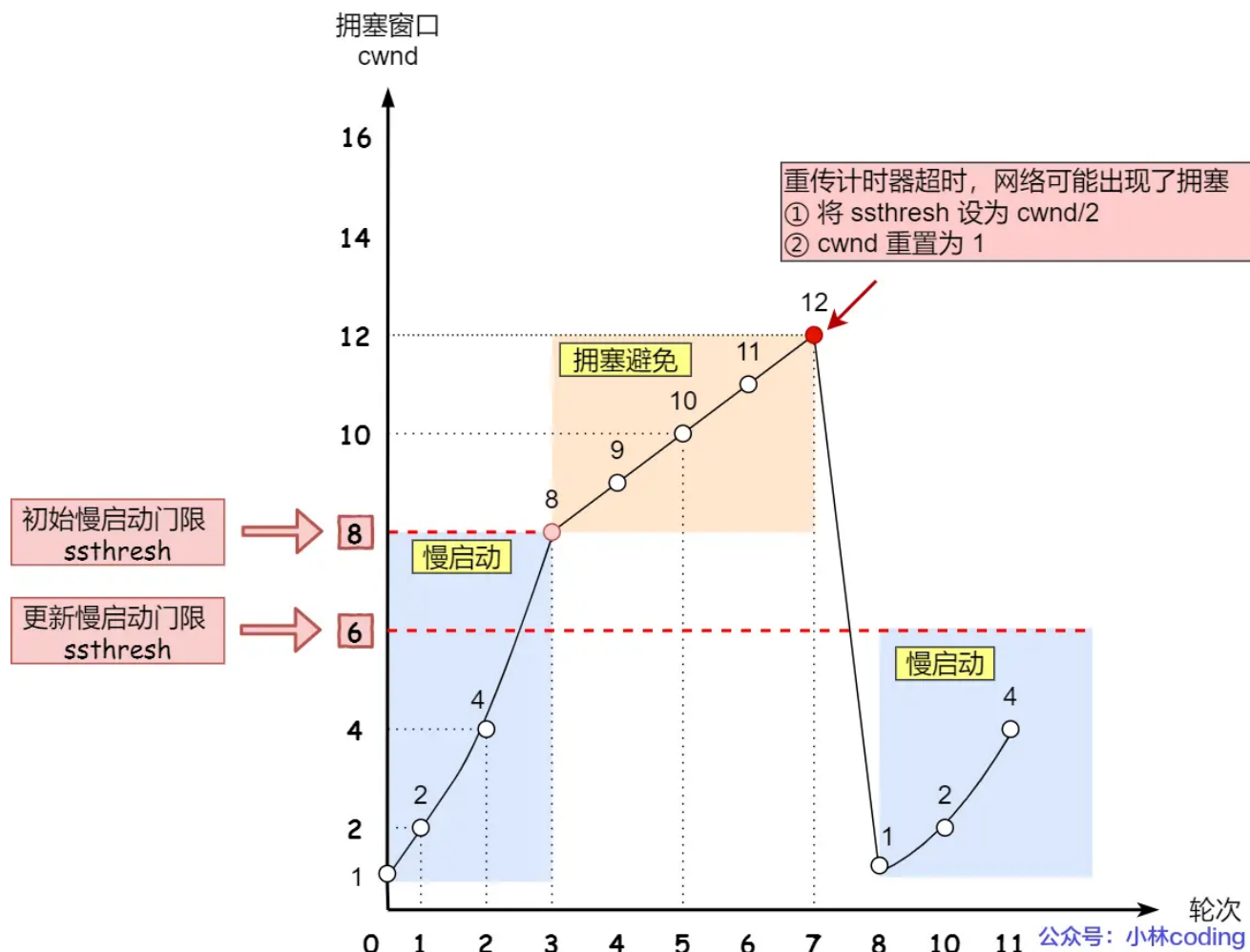
发生超时重传的拥塞发生算法

当发生了「超时重传」，则就会使用拥塞发生算法。

这个时候，ssthresh 和 cwnd 的值会发生变化：

- ssthresh 设为 $cwnd/2$ ，
- cwnd 重置为 1（是恢复为 cwnd 初始化值，我这里假定 cwnd 初始化值 1）

拥塞发生算法的变化如下图：



接着，就重新开始慢启动，慢启动是会突然减少数据流的。这真是一旦「超时重传」，马上回到解放前。但是这种方式太激进，反应也很强烈，会造成网络卡顿。

就好像本来在秋名山高速漂移着，突然来个紧急刹车，轮胎受得了吗。。。

发生快速重传的拥塞发生算法

还有更好的方式，前面我们讲过「快速重传算法」。当接收方发现丢了一个中间包的时候，发送三次前一个包的 ACK，于是发送端就会快速地重传，不必等待超时再重传。

TCP 认为这种情况不严重，因为大部分没丢，只丢了一小部分，则 ssthresh 和 cwnd 变化如下：

- $cwnd = cwnd/2$ ，也就是设置为原来的一半；
- $ssthresh = cwnd$ ；
- 进入快速恢复算法

快速恢复

快速重传和快速恢复算法一般同时使用，快速恢复算法是认为，你还能收到 3 个重复 ACK 说明网络也不那么糟糕，所以没有必要像 RTO 超时那么强烈。

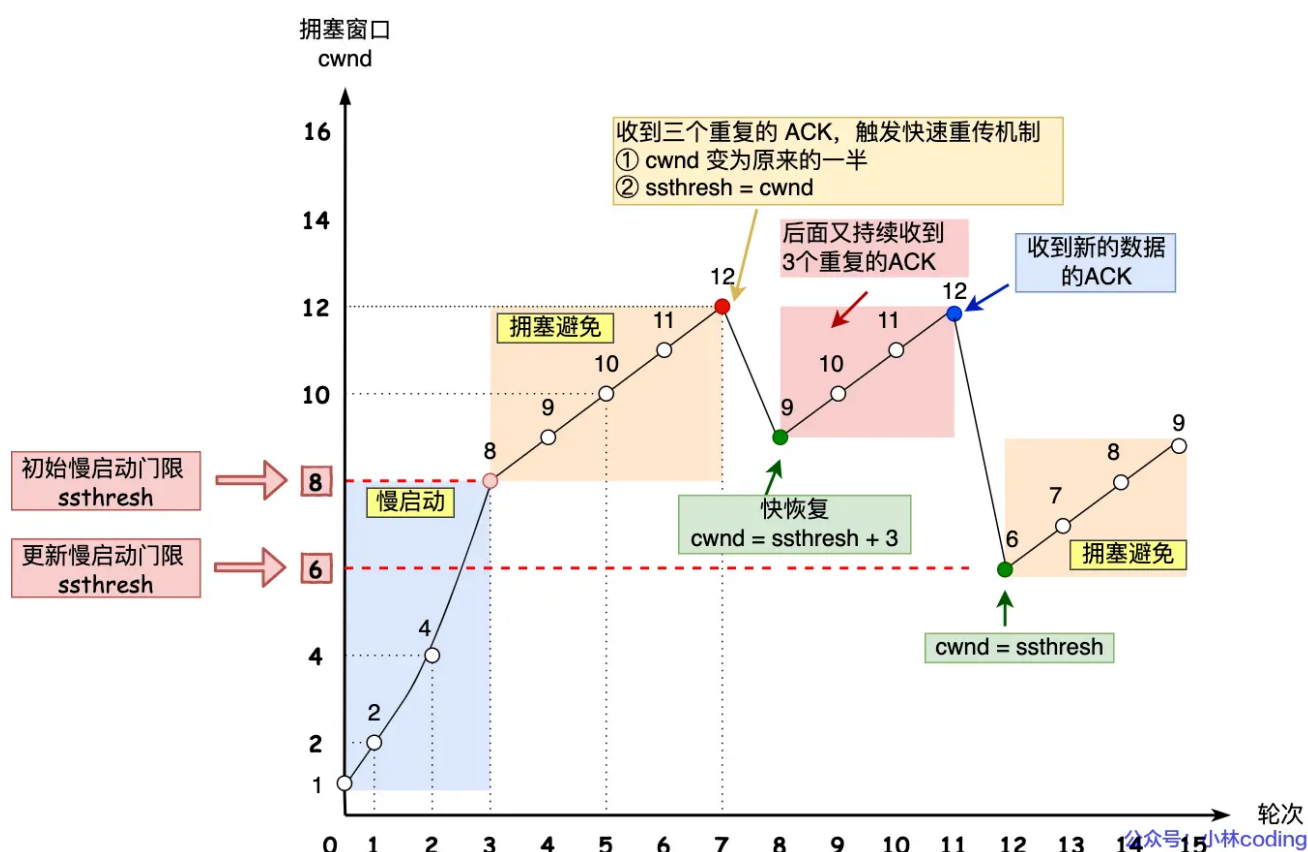
正如前面所说，进入快速恢复之前，cwnd 和 ssthresh 已被更新了：

- $cwnd = cwnd/2$ ，也就是设置为原来的一半；
- $ssthresh = cwnd$ ；

然后，进入快速恢复算法如下：

- 拥塞窗口 $cwnd = ssthresh + 3$ （3 的意思是确认有 3 个数据包被收到了）；
- 重传丢失的数据包；
- 如果再收到重复的 ACK，那么 cwnd 增加 1；
- 如果收到新数据的 ACK 后，把 cwnd 设置为第一步中的 ssthresh 的值，原因是该 ACK 确认了新的数据，说明从 duplicated ACK 时的数据都已收到，该恢复过程已经结束，可以回到恢复之前的状态了，也即再次进入拥塞避免状态；

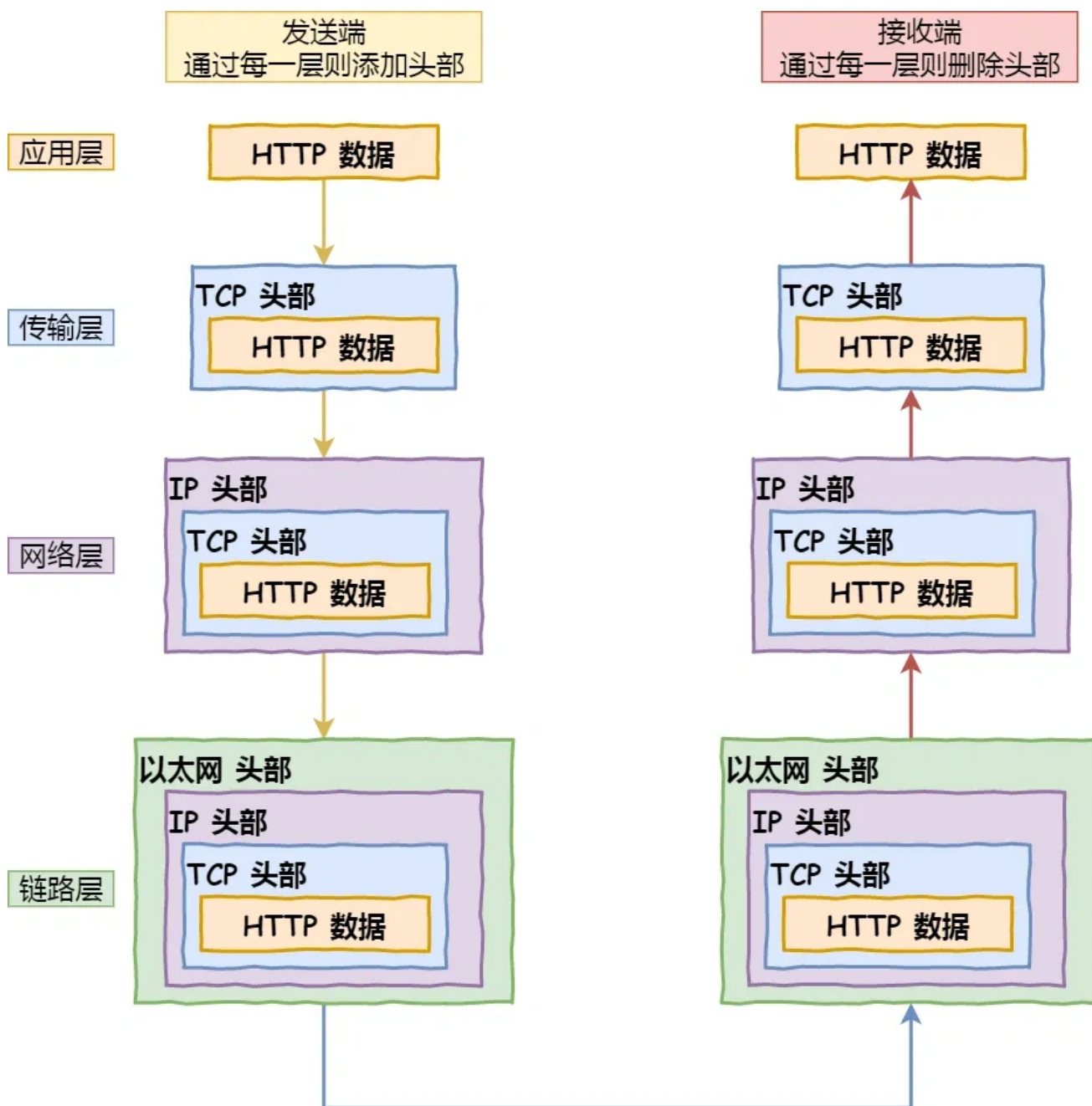
快速恢复算法的变化过程如下图：



也就是没有像「超时重传」一夜回到解放前，而是还在比较高的值，后续呈线性增长。

网络场景

描述一下打开百度首页后发生的网络过程



- 解析URL：分析 URL 所需要使用的传输协议和请求的资源路径。如果输入的 URL 中的协议或者主机名不合法，将会把地址栏中输入的内容传递给搜索引擎。如果没有问题，浏览器会检查 URL 中是否出现了非法字符，则对非法字符进行转义后在进行下一过程。
- 缓存判断：浏览器缓存 → 系统缓存 (hosts 文件) → 路由器缓存 → ISP 的 DNS 缓存，如果其中某个缓存存在，直接返回服务器的IP地址。
- DNS解析：如果缓存未命中，浏览器向本地 DNS 服务器发起请求，最终可能通过根域名服务器、顶级域名服务器 (.com)、权威域名服务器逐级查询，直到获取目标域名的 IP 地址。
- 获取MAC地址：当浏览器得到 IP 地址后，数据传输还需要知道目的主机 MAC 地址，因为应用层下发数据给传输层，TCP 协议会指定源端口号和目的端口号，然后下发给网络层。网络层会将本机地址作为源地址，获取的 IP 地址作为目的地址。然后将下发给数据链路层，数据链路层的发送需要加入通信双方的 MAC 地址，本机的 MAC 地址作为源 MAC 地址，目的 MAC 地址需要分情况处理。通过将 IP 地址与本机的子网掩码相结合，可以判断是否与请求主机在同一个子网里，如果在同一个子网里，可以使用 ARP 协议获取到目的主机的 MAC 地址，如果不在一个子网里，那么请求应该转发给网关，由它代为转发，此时同样可以通过 ARP 协议来获取网关的 MAC 地址，此时目的主机的 MAC 地址应该为网关的地址。

- 建立TCP连接：主机将使用目标 IP地址和目标MAC地址发送一个TCP SYN包，请求建立一个TCP连接，然后交给路由器转发，等路由器转到目标服务器后，服务器回复一个SYN-ACK包，确认连接请求。然后，主机发送一个ACK包，确认已收到服务器的确认，然后 TCP 连接建立完成。
- HTTPS 的 TLS 四次握手：如果使用的是 HTTPS 协议，在通信前还存在 TLS 的四次握手。
- 发送HTTP请求：连接建立后，浏览器会向服务器发送HTTP请求。请求中包含了用户需要获取的资源的信息，例如网页的URL、请求方法（GET、POST等）等。
- 服务器处理请求并返回响应：服务器收到请求后，会根据请求的内容进行相应的处理。例如，如果是请求网页，服务器会读取相应的网页文件，并生成HTTP响应。

网页非常慢转圈圈的时候，要定位问题需要从哪些角度？

最直接的办法就是抓包，排查的思路大概有：

1. 先确定是服务端的问题，还是客户端的问题。先确认浏览器是否可以访问其他网站，如果不可以，说明客户端网络自身的问题，然后检查客户端网络配置（连接wifi正不正常，有没有插网线）；如果可以正常访问其他网页，说明客户端网络是可以正常上网的。
2. 如果客户端网络没问题，就抓包确认 DNS 是否解析出了 IP 地址，如果没有解析出来，说明域名写错了，如果解析出了 IP 地址，抓包确认有没有和服务端建立三次握手，如果能成功建立三次握手，并且发出了 HTTP 请求，但是就是没有显示页面，可以查看服务端返回的响应码：
 - 如果是404错误码，检查输入的url是否正确；
 - 如果是500，说明服务器此时有问题；
 - 如果是200，F12看看前端代码有问题导致浏览器没有渲染出页面。
3. 如果客户端网络是正常的，但是访问速度很慢，导致很久才显示出来。这时候要看客户端的网口流量是否太大的了，导致tcp发生丢包之类的问题。

总之就是一层一层有没有插网线，网络配置是否正确、DNS有没有解析出 IP地址、TCP有没有三次握手、HTTP返回的响应码是什么。

server a和server b，如何判断两个服务器正常连接？出错怎么办？

直不会发送数据给客户端，那么服务端是永远无法感知到客户端宕机这个事件的，也就是服务端的 TCP 连接将一直处于 ESTABLISH 状态，占用着系统资源。

为了避免这种情况，TCP 搞了个**保活机制**。这个机制的原理是这样的：定义一个时间段，在这个时间段内，如果没有任何连接相关的活动，TCP 保活机制会开始作用，每隔一个时间间隔，发送一个探测报文，该探测报文包含的数据非常少，如果连续几个探测报文都没有得到响应，则认为当前的 TCP 连接已经死亡，系统内核将错误信息通知给上层应用程序。

在 Linux 内核可以有对应的参数可以设置保活时间、保活探测的次数、保活探测的时间间隔，以下都为默认值：

```
net.ipv4.tcp_keepalive_time=7200
net.ipv4.tcp_keepalive_intvl=75
net.ipv4.tcp_keepalive_probes=9
```

- tcp_keepalive_time=7200：表示保活时间是 7200 秒（2小时），也就 2 小时内如果没有任何连接相关的活动，则会启动保活机制
- tcp_keepalive_intvl=75：表示每次检测间隔 75 秒；
- tcp_keepalive_probes=9：表示检测 9 次无响应，认为对方是不可达的，从而中断本次的连接。

也就是说在 Linux 系统中，最少需要经过 2 小时 11 分 15 秒才可以发现一个「死亡」连接。

$$\text{tcp_keepalive_time} + (\text{tcp_keepalive_intvl} * \text{tcp_keepalive_probes})$$



$$7200 + (75 * 9) = 7875 \text{ 秒 (2 小时 11 分 15 秒)}$$

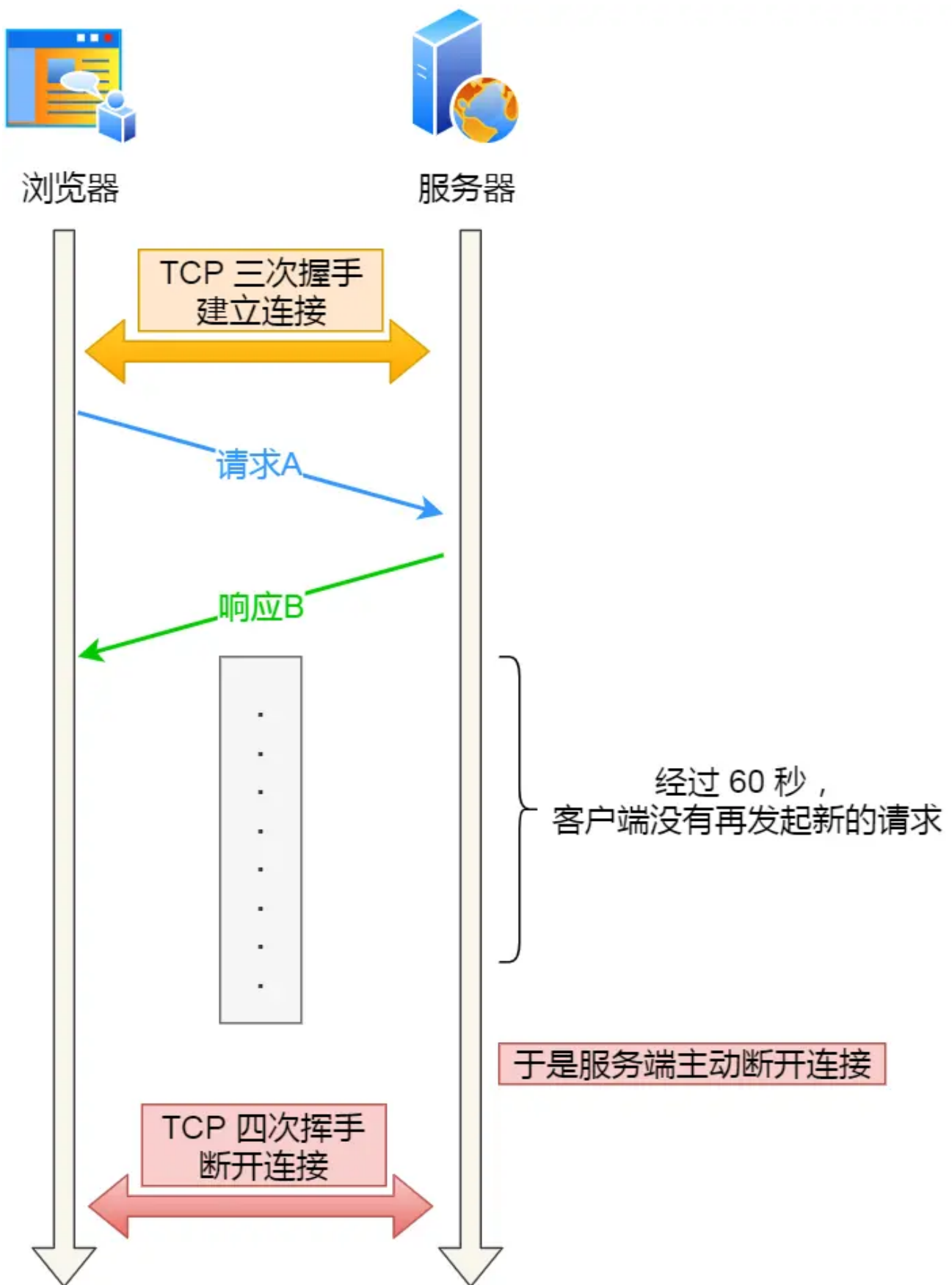
注意，应用程序若想使用 TCP 保活机制需要通过 socket 接口设置 SO_KEEPALIVE 选项才能够生效，如果没有设置，那么就无法使用 TCP 保活机制。

如果开启了 TCP 保活，需要考虑以下几种情况：

- 第一种，对端程序是正常工作的。当 TCP 保活的探测报文发送给对端，对端会正常响应，这样 **TCP 保活时间会被重置**，等待下一个 TCP 保活时间的到来。
- 第二种，对端主机宕机并重启。当 TCP 保活的探测报文发送给对端后，对端是可以响应的，但由于没有该连接的有效信息，**会产生一个 RST 报文**，这样很快就会发现 TCP 连接已经被重置。
- 第三种，是对端主机宕机（*注意不是进程崩溃，进程崩溃后操作系统在回收进程资源的时候，会发送 FIN 报文，而主机宕机则是无法感知的，所以需要 TCP 保活机制来探测对方是不是发生了主机宕机*），或对端由于其他原因导致报文不可达。当 TCP 保活的探测报文发送给对端后，石沉大海，没有响应，连续几次，达到保活探测次数后，**TCP 会报告该 TCP 连接已经死亡**。

TCP 保活的这个机制检测的时间是有点长，我们可以自己在应用层实现一个心跳机制。

比如，web 服务软件一般都会提供 keepalive_timeout 参数，用来指定 HTTP 长连接的超时时间。如果设置了 HTTP 长连接的超时时间是 60 秒，web 服务软件就会**启动一个定时器**，如果客户端在完成一个 HTTP 请求后，在 60 秒内都没有再发起新的请求，**定时器的时间一到，就会触发回调函数来释放该连接**。



服务端正常启动了，但是客户端请求不到有哪些原因？如何排查？

如果客户端请求的接口没有响应，排查的方式：

- 检查接口IP地址是否正确，ping一下接口地址。
- 检查被测接口端口号是否正确，可以在本机Telnet接口的IP和端口号，检查端口号能否连通
- 检查服务器的防火墙是否关闭，如果是以为安全或者权限问题不能关闭，需要找运维进行策略配置，开放对应的IP和端口。
- 检查你的客户端（浏览器、[测试工具 \(opens new window\)](#)），是否设置了网络代理，网络代理可以造成请求失败。

如果客户端的请求有响应，但是返回了错误状态码，那么根据错误码做对应的排查：

- 400：客户端请求错误，比如请求参数格式错误
- 401：未授权，比如请求header里，缺乏必要的信息头。（token，auth等）
- 403：禁止，常见原因是因为用户的账号没有对应的URL权限，还有就是项目中所用的中间件，不允许远程连接（Tomcat）
- 404：资源未找到，导致这种情况的原因很多，比如URL地址不正确
- 500：服务器内部错误，出现这种情况，说明服务器内部报错了，需要登录服务器，检查错误日志，根具体的提示信息在进行排查
- 502/503/504（错误的网关、服务器无法获得、网关超时）：如果单次调用接口就报该错误，说明后端服务器配置有问题或者服务不可用，挂掉了；如果是并发压测时出现的，说明后端压力太大，出现异常，此问题一般是后端出现了响应时间过长或者是无响应造成的

服务器ping不通但是http能请求成功，会出现这种情况吗？什么原因造成的？

ping 走的是 icmp 协议，http 走的是 tcp 协议。

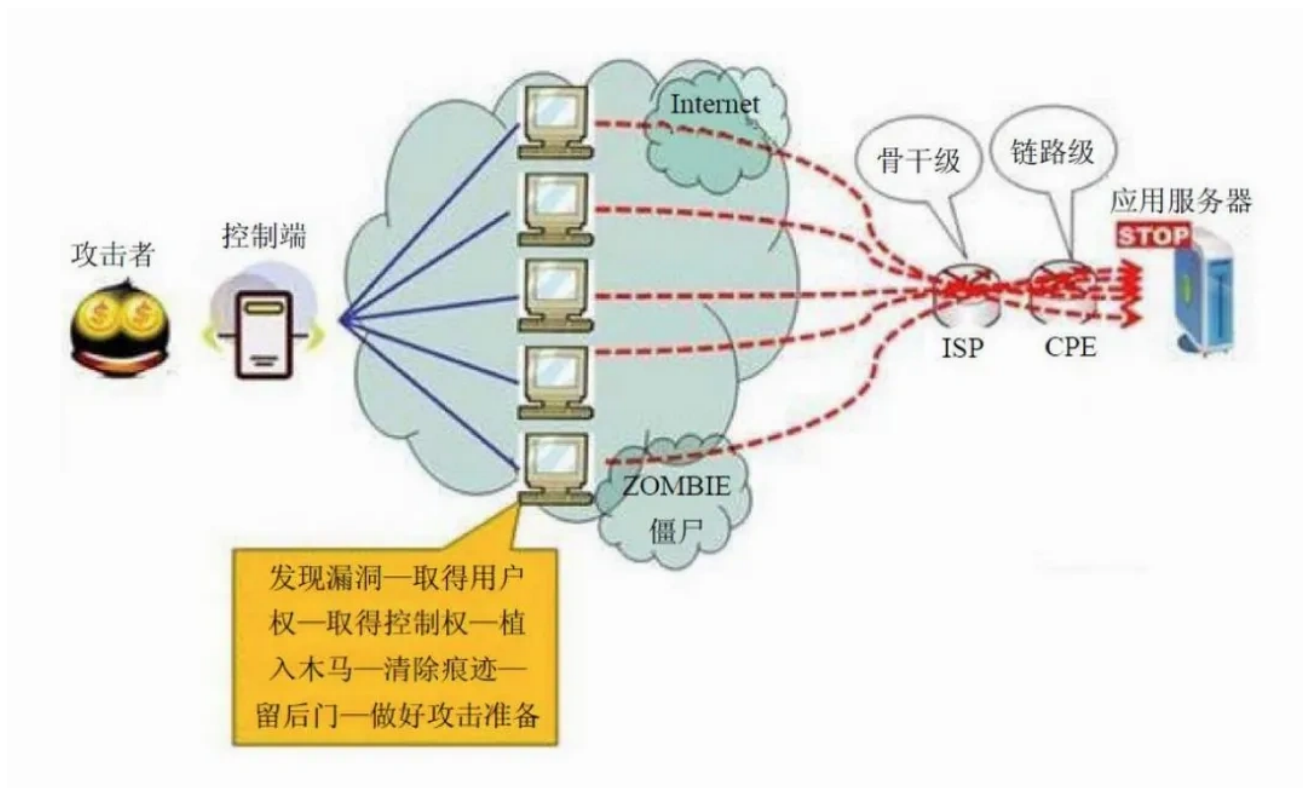
有可能服务器的防火墙禁止 icmp 协议，但是 tcp 协议没有禁止，就会出现服务器 ping 不通，但是 http 能请求成果。

网络攻击

什么是ddos攻击？怎么防范？

分布式拒绝服务（DDoS）攻击是通过大规模互联网流量淹没目标服务器或其周边基础设施，以破坏目标服务器、服务或网络正常流量的恶意行为。

DDoS 攻击是通过连接互联网的计算机网络进行的。这些网络由计算机和其他设备（例如 IoT 设备）组成，它们感染了恶意软件，从而被攻击者远程控制。这些个体设备称为机器人（或僵尸），一组机器人则称为僵尸网络。



一旦建立了僵尸网络，攻击者就可通过向每个机器人发送远程指令来发动攻击。当僵尸网络将受害者的服务器或网络作为目标时，每个机器人会将请求发送到目标的 IP 地址，这可能导致服务器或网络不堪重负，从而造成对正常流量的拒绝服务。由于每个机器人都是合法的互联网设备，因而可能很难区分攻击流量与正常流量。

常见的DDoS攻击包括以下几类：

- **网络层攻击**：比较典型的攻击类型是UDP反射攻击，例如：NTP Flood攻击，这类攻击主要利用大流量拥塞被攻击者的网络带宽，导致被攻击者的业务无法正常响应客户访问。
- **传输层攻击**：比较典型的攻击类型包括SYN Flood攻击、连接数攻击等，这类攻击通过占用服务器的连接池资源从而达到拒绝服务的目的。
- **会话层攻击**：比较典型的攻击类型是SSL连接攻击，这类攻击占用服务器的SSL会话资源从而达到拒绝服务的目的。
- **应用层攻击**：比较典型的攻击类型包括DNS flood攻击、HTTP flood攻击、游戏假人攻击等，这类攻击占用服务器的应用处理资源极大的消耗服务器处理性能从而达到拒绝服务的目的。

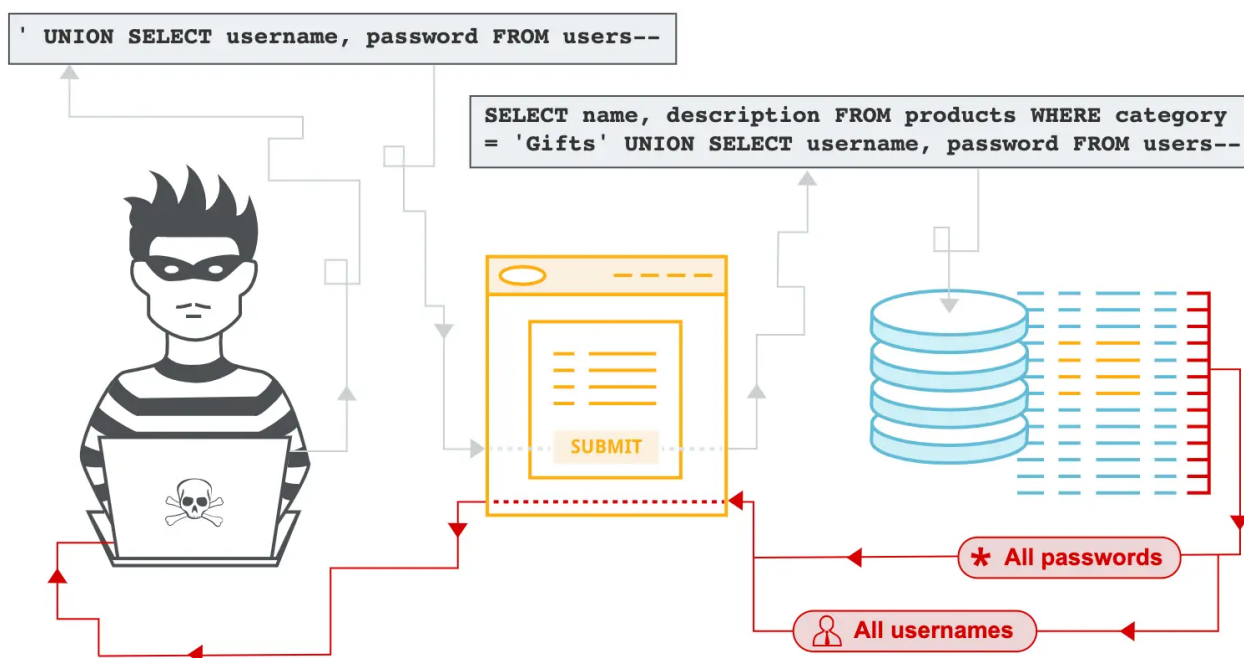
为了防范DDoS攻击，可以采取以下措施：

- **增强网络基础设施**：提升网络带宽、增加服务器的处理能力和承载能力，通过增强基础设施的能力来抵御攻击。
- **使用防火墙和入侵检测系统**：配置防火墙规则，限制不必要的网络流量，阻止来自可疑IP地址的流量。入侵检测系统可以帮助及时发现并响应DDoS攻击。
- **流量清洗和负载均衡**：使用专业的DDoS防护服务提供商，通过流量清洗技术过滤掉恶意流量，将合法流量转发给目标服务器。负载均衡可以将流量均匀地分发到多台服务器上，减轻单一服务器的压力。
- **配置访问控制策略**：限制特定IP地址或IP段的访问，设置访问频率限制，防止过多请求集中在单个IP上。

SQL注入问题是什么？

SQL注入发生在当应用程序直接使用用户提供的输入作为SQL查询的一部分时。当用户输入被错误地用作数据库查询的一部分，而应用程序没有对其进行适当的验证和转义，就可能会发生SQL注入。

例如，如果一个用户输入了一个字符串来查找特定用户的信息，但应用程序将此用户输入直接用作SQL查询的一部分（例如，作为SELECT语句的一部分），而不考虑可能的安全问题，那么攻击者可能会利用这一点来执行他们自己的恶意SQL查询。



解决SQL注入问题的方法主要有以下几种：

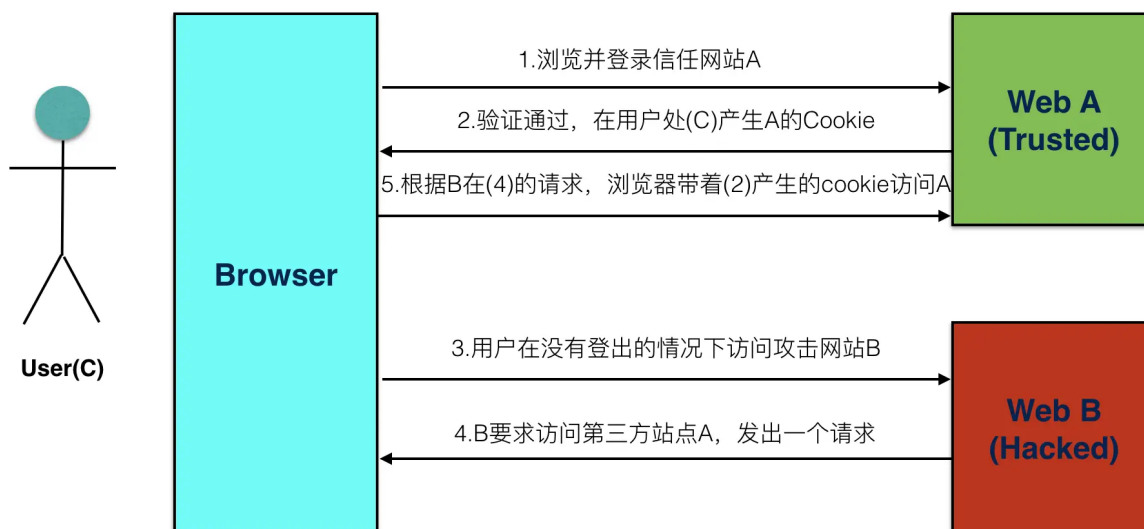
1. 输入验证和转义：在将用户输入用作SQL查询的一部分之前，对输入进行验证和转义。确保输入符合预期格式，并防止任何可能导致SQL注入的特殊字符。
2. 使用参数化查询：使用参数化查询可以避免直接将用户输入嵌入到SQL查询中。参数化查询使用预定义的变量来接收用户输入，并将其传递给数据库引擎，而不是直接将其用作查询的一部分。这样可以防止SQL注入攻击。
3. 限制数据库权限：限制数据库用户的权限，只授予他们执行所需操作所需的最低权限。攻击者可能具有比预期更多的权限，这可能会使攻击更加容易。
4. 实施输入过滤：在某些情况下，实施输入过滤可以进一步减少SQL注入的风险。这可能涉及检查和过滤用户输入中的特殊字符和词汇，以排除可能的恶意输入。

CSRF攻击是什么？

CSRF（跨站请求伪造）是一种攻击手段，攻击者通过诱导用户执行恶意操作，从而获取用户数据或执行恶意代码。CSRF攻击通常通过伪造一个合法的HTTP请求来实现，这个请求看起来是合法的，但实际上是为了执行一个攻击者控制的操作。

存在CSRF漏洞的网站：WebA
攻击者：WebB
受害者：User/WebA

6.A不知道(5)中的请求是C发出的还是B发出的，由于浏览器会自动带上用户C的Cookie，所以A会根据用户的权限处理(5)的请求，这样B就达到了模拟用户操作的目的

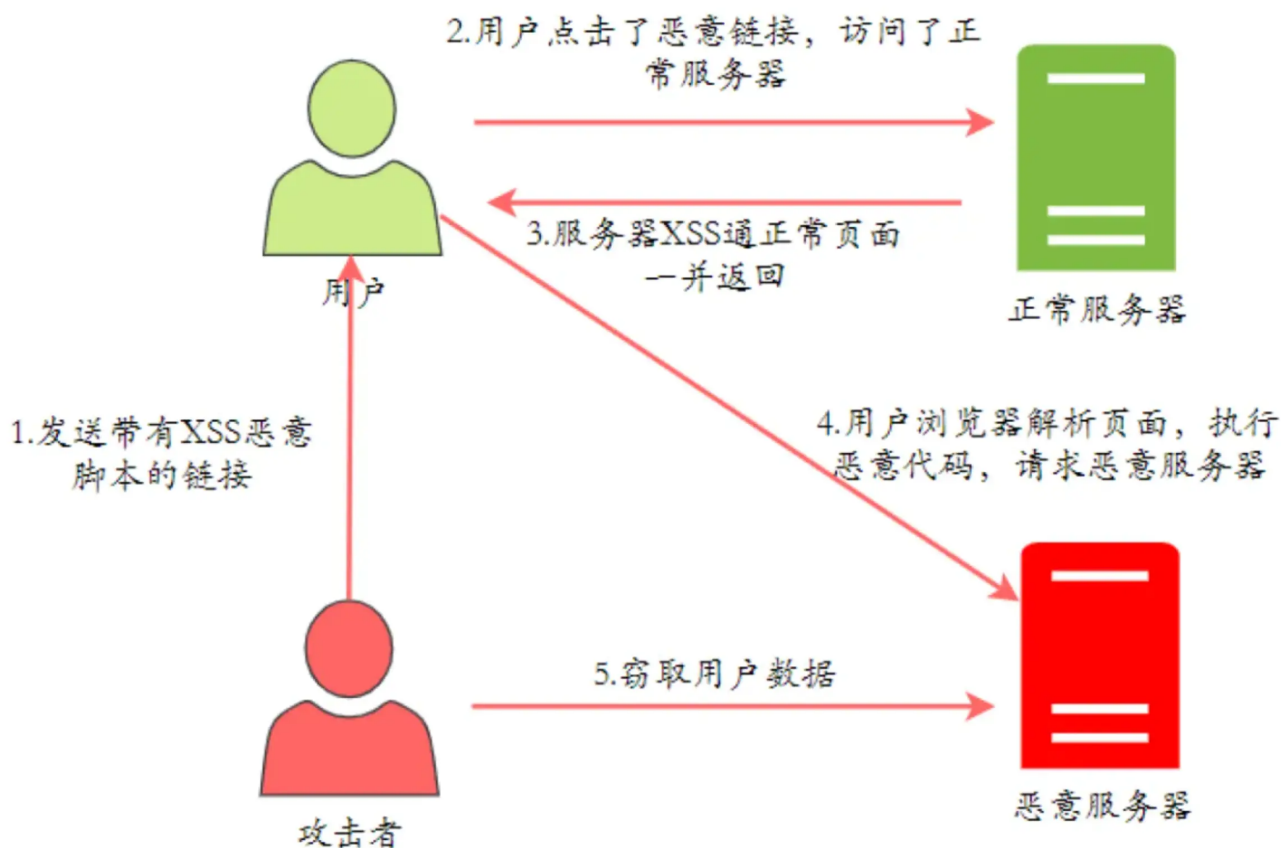


解决CSRF攻击的方法主要有以下几种：

1. 验证用户会话：在服务器端对用户会话进行验证，确保请求的会话标识符与当前会话标识符匹配。这样可以防止攻击者伪造会话标识符。
2. 使用双重验证：除了会话验证，还可以使用其他验证方式，例如验证码、签名验证等。这些验证方式可以增加攻击的难度。
3. 防止跨站请求：通过设置CSP（内容安全策略）来防止跨站请求，限制网页中可执行的脚本源，减少攻击者诱导用户执行恶意操作的可能性。
4. 避免使用自动提交表单：禁用默认的自动提交功能，要求用户在提交表单前确认操作，防止攻击者诱导用户在未经授权的情况下提交表单。
5. 强制Referer头部：在服务器端检查请求的Referer头部，确保请求来自可信来源。

XSS攻击是什么？

XSS是跨站脚本攻击，攻击者通过在Web页面中插入恶意脚本代码，然后诱使用户访问该页面，从而使得恶意脚本在用户浏览器中执行，从而盗取用户信息、会话信息等敏感数据，甚至控制用户账户。



XSS 攻击可以分为 3 类：存储型（持久型）、反射型（非持久型）、DOM 型。

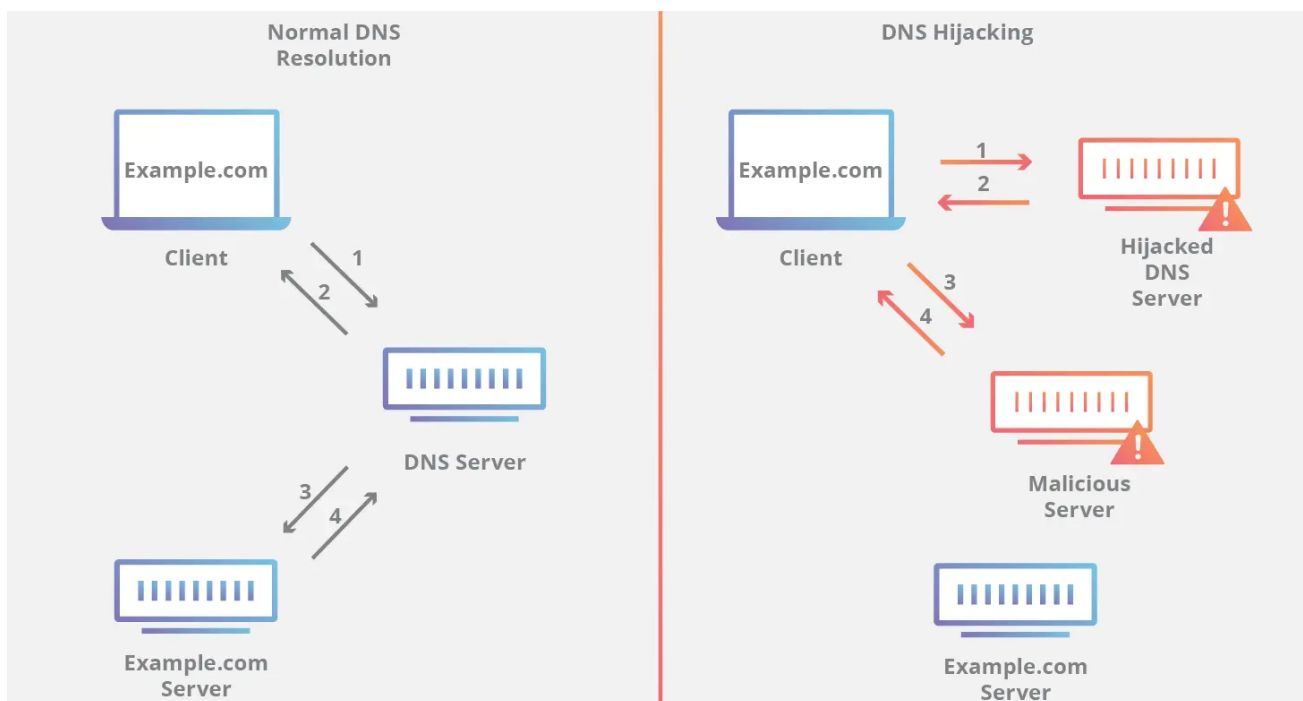
- [存储型 XSS \(opens new window\)](#): 注入型脚本永久存储在目标服务器上。当浏览器请求数据时，脚本从服务器上传回并执行。
- [反射型 XSS \(opens new window\)](#): 当用户点击一个恶意链接，或者提交一个表单，或者进入一个恶意网站时，注入脚本进入被攻击者的网站。Web 服务器将注入脚本，比如一个错误信息，搜索结果等 返回到用户的浏览器上。由于浏览器认为这个响应来自"可信任"的服务器，所以会执行这段脚本。
- [基于 DOM 的 XSS \(opens new window\)](#): 通过修改原始的客户端代码，受害者浏览器的 DOM 环境改变，导致有效载荷的执行。也就是说，页面本身并没有变化，但由于 DOM 环境被恶意修改，有客户端代码被包含进了页面，并且意外执行。

预防XSS攻击的方法主要包括以下几点：

- 输入验证：对所有用户输入的数据进行有效性检验，过滤或转义特殊字符。例如，禁止用户输入HTML标签和JavaScript代码。
- 输出编码：在网页输出用户输入内容时，使用合适的编码方式，如HTML转义、URL编码等，防止恶意脚本注入。
- Content Security Policy (CSP)：通过设置CSP策略，限制网页中可执行的脚本源，有效防范XSS攻击。
- 使用HttpOnly标记：在设置Cookie时，设置HttpOnly属性，使得Cookie无法被JavaScript代码读取，减少受到XSS攻击的可能。

了解过DNS劫持吗？

DNS劫持的原理是攻击者在用户查询DNS服务器时篡改响应，将用户请求的域名映射到攻击者控制的虚假IP地址上，使用户误以为访问的是正常网站，实际上被重定向到攻击者操控的恶意网站。这种劫持可以通过植入恶意的DNS记录或劫持用户的DNS流量来实现。



 面试鸭 mianshiya.com

程序员面试 刷题神器

八股文一网打尽 面试从未如此轻松

- ✓ 9000+ 中大厂高频面试题
- ✓ 大厂面试官团队原创优质题解
- ✓ 网页版+小程序+IDE、VSCode插件
- ✓ 真题命中率高，持续更新

微信扫一扫 >>

刷高频题拿高薪offer



最新的图解文章都在公众号首发，别忘记关注哦！！如果你想加入百人技术交流群，扫码下方二维码回复「加群」。



扫一扫，关注「小林coding」公众号

图解计算机基础
认准**小林coding**

每一张图都包含小林的认真
只为帮助大家能更好的理解

- ① 关注公众号回复「**图解**」
获取图解系列 PDF
- ② 关注公众号回复「**加群**」
拉你进百人技术交流群